TRABAJOS FIN DE GRADO



Universidad de Jaén

Escuela Politécnica Superior de Jaén

Desambiguación de Terminología Médica en Informes Hospitalarios (DesambiMED)

Autor: Javier Fernández Basso

Grado: Ingeniería en Informática

Directores: Manuel Carlos Díaz Galiano y Lucas Molino Piñar
Departamento del director: Informática

Fecha: Fecha de presentación

Licencia: CC-BY-NC-SA



CREA

Agradecimientos

Llegar hasta aquí no ha sido el fruto de un esfuerzo individual, sino el resultado del apoyo, la paciencia y el cariño de muchas personas que, de una forma u otra, han dejado su huella en este camino. Estas líneas son un pequeño intento de hacerles justicia.

En primer lugar, quiero dar las gracias a mis padres, Juan Carlos y Yolanda, por su apoyo incondicional durante todos estos años, por confiar en mí incluso en los momentos en los que yo mismo dudaba, y por haber hecho posible, con su esfuerzo silencioso y constante, que llegue hasta aquí. Todo lo que soy y todo lo que he logrado lleva, de una forma u otra, su huella.

A mis hermanos, Pablo y Carlos, y a mis cuñadas, María y Jesica, por estar siempre presentes, por su ánimo constante y por acompañarme en cada etapa de este camino, tanto en los buenos momentos como en los más complicados. Y a mis sobrinos, Daniela, Carlos y Lara, por alegrarme los días más duros del trabajo con su inocencia y su alegría, recordándome siempre que hay motivos de sobra para sonreír incluso cuando todo parece cuesta arriba.

A mis abuelos, Jose Antonio Basso Vázquez y María Mercedes Sequera Gómez (o Pepe y Pepa, como cariñosamente los llamábamos) y Aureliano Fernández García y María Ruiz Quirós, por haber sido siempre un gran apoyo para mí y una fuente inagotable de cariño. Quiero dedicar unas palabras muy especiales a los dos que ya no están entre nosotros. Pepe y Aureliano, aunque la vida no os haya permitido verme terminar la carrera, sé que hoy me acompañáis y celebráis este logro desde donde estéis. Este trabajo también es vuestro.

A mis amigos de Granada, que me han acompañado y apoyado desde el principio de la carrera, compartiendo alegrías, dudas y desahogos a lo largo de todo este recorrido. Quiero destacar especialmente a Adrián y a mi primo Guillermo, cuya amistad ha sido un pilar constante durante todos estos años y cuyo apoyo, aunque muchas veces silencioso, jamás ha faltado.

A la gente nueva que conocí en Jaén, que convirtió esta etapa en una experiencia mucho más especial y enriquecedora de lo que jamás imaginé, y que me hizo sentir como en casa lejos de ella. En particular, a mi compañero de piso David, por aguantarme hablando sin parar de asignaturas y del TFG, y por soportarme con paciencia y buen humor todos los días mientras no paraba de molestarle. Sin saberlo, su compañía ha hecho mucho más llevadero este camino.

A mi novia, Leticia, por su ayuda y su apoyo incondicional durante este último año. Por leerse este TFG una y otra vez sin perder la paciencia, por escucharme hablar de él desde principio de curso sin quejarse ni una sola vez, y por estar a mi lado en los momentos de agobio tanto como en los de alegría. Su cariño y su comprensión han sido un impulso fundamental para llegar hasta el final de este camino.

Quiero expresar también mi más sincero agradecimiento a mis tutores, Manuel Carlos Díaz Galiano y Lucas Molino Piñar, por su dedicación, disponibilidad y orientación a lo largo de todo el desarrollo de este trabajo. Su experiencia, sus consejos y su confianza en el proyecto han sido fundamentales para poder llevarlo a buen término.

También quiero agradecer a todas las personas que, de una forma u otra, me han ayudado y me han hecho crecer a lo largo de mi vida, desde la etapa de primaria hasta hoy, tanto profesores como personas que han pasado por mi camino, estén o no presentes en mi vida actualmente. Cada palabra de ánimo, cada consejo y cada gesto de apoyo han contribuido, aunque a veces de manera imperceptible, a construir el camino que hoy me trae hasta este trabajo. Sin todos y cada uno de vosotros, esto no habría sido posible.

Por útimo, este trabajo se enmarca dentro del proyecto de investigación GALENO-IA (DGP PIDI 2024 00852)[1], financiado por la Unión Europea con cargo al Programa de la Estrategia de Especialización Inteligente para la Sostenibilidad de Andalucía S4Andalucía 2021-2027. Gracias a él ha sido posible tanto el acceso al corpus hospitalario de informes de cardiología cedido por el Hospital Universitario de Jaén como la evaluación manual de algunos elementos de este trabajo por parte de los especialistas clínicos que colaboraron en la fase de anotación supervisada. Quiero aprovechar estas líneas para agradecer a todo el equipo del proyecto su apoyo, sin el cual este trabajo no se podría haber llevado a cabo.

[1] `https://investigacion.ujaen.es/proyectos/1792396/detalle`

Tabla de contenidos

1. Introducción **1**

1.1. Propósito principal 3

1.2. Motivación 3

1.3. Hipótesis principal 4

1.4. Objetivos 4

1.5. Cumplimiento de los Objetivos de Desarrollo Sostenible 6

1.6. Estructura de la memoria 11

1.7. Planificación y costes 12

1.7.1. Planificación del proyecto 12

1.7.2. Costes del proyecto 15

2. Estado del Arte **17**

2.1. Procesamiento del lenguaje natural en el dominio clínico 17

2.1.1. Texto clínico: características y retos 18

2.2. Desambiguación de sentido de palabras en texto biomédico 20

2.2.1. Enfoques clásicos: reglas y diccionarios 20

2.2.2. Enfoques basados en aprendizaje automático y modelos de lenguaje 22

2.2.3. El problema específico de abreviaturas y símbolos en español . 22

2.3. Modelos de lenguaje pequeños y eficientes para PLN clínico 23

2.3.1. Arquitecturas de referencia: IBM Granite, Qwen y Google Gemma 23

2.3.2. Técnicas de eficiencia: cuantización y LoRA 24

2.3.3. Prompting e ingeniería de instrucciones 25

2.4. Sostenibilidad en el desarrollo de modelos de IA 26
2.5. Recursos, corpus y herramientas para el español clínico 26
2.5.1. Corpus clínicos en español 27
2.5.2. Recursos léxicos y ontológicos 27
2.6. Trabajos relacionados 27
2.6.1. Sistemas basados en reglas y recursos léxicos 28
2.6.2. Sistemas basados en aprendizaje automático 28
2.6.3. Aproximaciones con LLMs 28
2.6.4. Simplificación de texto médico y posicionamiento del trabajo . . 29
2.7. Síntesis y posicionamiento del trabajo 30

3. Metodología y diseño experimental 31
3.1. Visión general del proceso 31
3.2. Corpus y datos 33
3.2.1. Origen y características del corpus 33
3.2.2. Proceso de anonimización 34
3.2.3. Anotación del corpus 34
3.2.4. Detección de abreviaturas candidatas 36
3.2.5. Análisis exploratorio del corpus 38
3.3. Selección y configuración de modelos 39
3.3.1. Criterios de selección 39
3.3.2. Descripción de las familias de modelos evaluados 40
3.3.3. Infraestructura hardware 41
3.3.4. Entorno software 41
3.4. Ingeniería de prompts 43
3.4.1. Motivación y proceso de diseño 43
3.4.2. Estructura de entrada de los prompts 44
3.4.3. Descripción de los tres prompts 44
3.4.4. Motivación del diseño de tres prompts 45
3.4.5. Formato de salida y postprocesamiento 46

3.4.6. Protocolo de evaluación de prompts 46
3.5. Ajuste fino con LoRA 47
3.5.1. Motivación del ajuste fino 47
3.5.2. Fundamentos de LoRA y QLoRA 47
3.5.3. Configuración del entrenamiento 48
3.5.4. Preparación de los datos de entrenamiento 49
3.5.5. Separación de conjuntos de evaluación 49
3.6. Métricas de evaluación 50
3.6.1. Métrica principal: precisión o accuracy 50
3.6.2. Análisis por documento y por abreviatura 51
3.6.3. Métricas de eficiencia computacional 51
3.7. Retos técnicos del proceso 52
3.7.1. Control del formato de salida 52
3.7.2. Ausencia de opciones en ficheros JSON 54
3.7.3. Comportamiento anómalo de Granite 1B 54
3.7.4. Resultados perfectos de Qwen 4B base 54
3.7.5. Evolución iterativa del script de evaluación 55
3.7.6. Gestión de los tiempos de inferencia y planificación de experimentos 55
3.8. Resumen del diseño experimental 56

4. Análisis de Resultados y Discusión 57
4.1. Evaluación detallada de los modelos base y línea base operativa 57
4.1.1. Comportamiento de la familia Qwen3.5 en estado base 58
4.1.2. Análisis comparativo: Gemma-3-1b-it frente a modelos compactos 59
4.2. Impacto cualitativo y cuantitativo de la ingeniería de prompts 60
4.2.1. La inyección de candidatos léxicos (Prompt 2) como mecanismo de anclaje 60
4.2.2. El fracaso del razonamiento en cadena (*Chain-of-Thought*) en modelos de pequeña escala 60
4.3. Evaluación de los modelos ajustados finamente mediante LoRA 61

4.3.1. Sobreajuste estructural en el modelo `Qwen3.5-0.8B` 62
4.3.2. Especialización catastrófica en abreviaturas de alta ambigüedad 63
4.3.3. Evolución y consolidación en las escalas de 2B y 4B parámetros 64
4.4. Análisis de eficiencia computacional y consumo de recursos 65
4.4.1. Optimización de la huella de VRAM para hardware de consumo 65
4.4.2. Análisis estadístico de latencia y rendimiento temporal 66
4.4.3. Elección de la configuración recomendada según el escenario de despliegue 67
4.5. Auditoría forense de errores y casos de resistencia predictiva 68
4.5.1. Fragmentación de embeddings por desalineación del tokenizador (caso `UPDRS`) 68
4.5.2. Ambigüedad polisémica fisiológica y símbolos especiales (`III` y `ºC`) 69
4.5.3. Análisis forense de trazas de inferencia en modelos compactos . 69
4.6. Comparativa con arquitecturas de mayor escala reportadas en la literatura 71
4.7. Implicaciones clínicas, privacidad soberana y viabilidad asistencial . . . 71

5. Herramienta de Anotación 73

5.1. Objetivos y requisitos 73
5.1.1. Objetivos 73
5.1.2. Requisitos funcionales 74
5.1.3. Requisitos no funcionales 75
5.2. Arquitectura de la herramienta 77
5.2.1. Backend: `tagger_api.py` 77
5.2.2. Frontend: `index.html` y `app.js` 78
5.3. Módulo de detección de abreviaturas 78
5.3.1. Primera aproximación: expresiones regulares y diccionario . . . 78
5.3.2. Segunda aproximación: modelo de extracción neuronal 79
5.4. Diccionario de abreviaturas: `abreviaciones.csv` 80
5.5. Flujo de anotación 80
5.6. Evolución iterativa y problemas resueltos 81

5.6.1. Problemas de formato en la interfaz 81
5.6.2. Pérdida de las opciones candidatas en los ficheros de salida . . 82
5.6.3. Integración del modo revisión 82
5.6.4. Autoasignación de abreviaturas con definición única 83
5.7. Limitaciones de la herramienta 83

6. Conclusiones y líneas de trabajo futuro 85
6.1. Conclusiones 85
6.1.1. Viabilidad de los modelos de lenguaje pequeños en el entorno clínico 85
6.1.2. El diseño del prompt como factor determinante 86
6.1.3. Impacto del ajuste fino con LoRA 87
6.1.4. Heterogeneidad entre familias arquitectónicas 87
6.1.5. Contribuciones a la auditoría de errores 88
6.1.6. El papel del prototipo de herramienta de anotación en el proceso experimental 88
6.1.7. Cumplimiento de objetivos 88
6.2. Líneas de trabajo futuro 89
6.2.1. Ampliación y enriquecimiento del corpus 89
6.2.2. Estrategias avanzadas de ajuste fino 90
6.2.3. Tokenización y vocabulario médico especializado 90
6.2.4. Integración en sistemas de soporte a la decisión clínica 90
6.2.5. Extensión a nuevos tipos de terminología 91
6.2.6. Evaluación con métricas de impacto clínico 91
6.2.7. Arquitecturas multimodales 91

7. Uso responsable de IA 93
7.1. Fases del proyecto apoyadas por IA 93
7.2. Validación y seguridad de los resultados 95
7.3. Beneficios aportados al proyecto 96

Bibliografía IV

A. Prompts diseñados para la desambiguación de siglas V

A.1. Prompt 1: Enfoque de análisis lingüístico y clínico V

A.2. Prompt 2: Enfoque de sustitución semántica VI

A.3. Prompt 3: Enfoque de auditoría y coherencia global VI

B. Manual de usuario del prototipo de herramienta de anotación IX

B.1. Requisitos previos IX

B.2. Estructura del proyecto X

B.3. Instalación XI

B.3.1. Obtención del proyecto XI

B.3.2. Creación de un entorno virtual (recomendado) XI

B.3.3. Instalación de las dependencias XII

B.3.4. Preparación de las carpetas de datos XII

B.4. Puesta en marcha del servidor XII

B.5. Modos de acceso a la herramienta XIII

B.5.1. Modo normal: anotación desde cero XIII

B.5.2. Modo revisión: reanudar o corregir un documento XIII

B.6. Guía de uso paso a paso XIV

B.6.1. Carga del diccionario XIV

B.6.2. Introducción y procesamiento del texto (modo normal) XIV

B.6.3. Código de colores XIV

B.6.4. Selección del significado de una abreviatura XV

B.6.5. Incorporación de nuevas abreviaturas no detectadas XV

B.6.6. Guardado de la anotación XV

B.7. Formato del fichero de anotación (JSON) XV

B.8. El diccionario de abreviaturas: `abreviaciones.csv` XVI

B.9. Resolución de problemas comunes XVII

B.10. Recomendaciones de despliegue y privacidad XVII

Lista de figuras

1.1. Objetivos de Desarrollo Sostenible cumplimentados por este proyecto. Fuente: `https://www.pactomundial.org/que-puedes-hacer-tu/ods/` . 10

1.2. Planificación temporal del TFG (diagrama de Gantt). Se muestran las cinco fases principales del proyecto desde octubre de 2025 hasta julio de 2026. 14

1.3. Desglose del presupuesto estimado del proyecto por partidas. El coste de personal representa el 82,1 % del total. 15

2.1. Esquema conceptual de la arquitectura *Transformer* (Vaswani et al., [1]). Se detalla la estructura basada en bloques acoplados de codificación (*encoder*) y decodificación (*decoder*), destacando los módulos de atención multi-cabeza y las redes de propagación hacia adelante (*feed-forward*). 19

3.1. Esquema general del proceso de desarrollo y evaluación del sistema de desambiguación. 32

3.2. Categorización del corpus de evaluación según el origen de la anotación y el tipo de modelo. 35

3.3. Distribución de frecuencia real de las abreviaturas más comunes en el Corpus IA (azul, 125 instancias) y el Corpus Médicos (verde, 299 instancias), contabilizada una única vez por instancia anotada. Un subconjunto reducido de formas abreviadas (`Hg`, `mm`, `NRL`, `Hb`, `PCR`) concentra hasta 12 apariciones, mientras que la mayoría de las abreviaturas del corpus aparece una o dos veces, configurando una larga cola característica de los corpus de texto clínico real. Este desequilibrio motiva el uso de métricas macro-promediadas para evaluar el comportamiento del sistema en términos de igualdad entre clases, ya que una métrica ponderada por frecuencia (micro) diluiría el rendimiento del sistema sobre esta cola de baja frecuencia. 39

3.4. Ciclo iterativo de evaluación y mejora de prompts. El flujo parte del prompt (v_n) y el corpus de entrada, pasa por el modelo y el módulo de normalización de salidas, y se bifurca según la validez del JSON resultante: las respuestas válidas se vuelcan en el CSV de resultados para su evaluación; las malformadas desencadenan un análisis de fallo que alimenta la revisión del prompt (v_{n+1}), reiniciando el ciclo. El bucle se repite para cada combinación modelo-prompt hasta obtener una tasa de respuestas malformadas inferior al umbral de aceptabilidad definido. 53

4.1. Variación de precisión global en el Corpus Médicos (299 instancias) antes y después del ajuste fino con LoRA para las tres escalas de la familia Qwen. Como se puede observar se produce una degradación notable en el modelo Qwen3.5-0.8B, mientras que los modelos de 2B y 4B parámetros sufren una bajada del rendimiento mucho más moderada, manteniendo una precisión superior al $80\,\%$. 63

4.2. Comparativa de tasa de fallo por abreviatura entre el modelo `Qwen3.5-0.8B` base y su variante ajustada con LoRA sobre el Corpus Médicos. Las barras rojas superan sistemáticamente a las azules en las abreviaturas de alta ambigüedad, evidenciando la *catastrophic specialization* al vocabulario del corpus de entrenamiento. 64

4.3. Fragmento de las trazas de ejecución del modelo Granite donde se evidencia la anomalía temporal por bucle sintáctico. 70

5.1. Maqueta de baja fidelidad de la interfaz de usuario del prototipo de herramienta de anotación, mostrando el área de texto, el panel de resultados y el modal de selección de opciones. 75

5.2. Maqueta de baja fidelidad del modal de selección de opciones en el prototipo de herramienta de anotación. 76

5.3. Captura de pantalla del prototipo de herramienta de anotación, mostrando un informe de cardiología con abreviaturas resaltadas. 84

5.4. Captura de pantalla del prototipo de herramienta de anotación en modo de revisión, mostrando un informe de cardiología con abreviaturas resaltadas. Las abreviaturas ya asignadas se distinguen mediante el color verde al igual que si se acabaran de anotar. Además se puede ver el modal de seleccion con las opciones de expansión disponibles para la abreviatura «ECG». Los datos médicos mostrados no son reales, son de un informe generado automáticamente mediante IA, cualquier parecido con la realidad es pura casualidad. 84

Lista de tablas

2.1. Comparativa de los modelos evaluados en el presente trabajo. 24

3.1. Resumen de las combinaciones modelo-prompt evaluadas. Se indica el corpus de evaluación empleado (IA: textos anotados por inteligencia artificial; Médicos: textos anotados por especialistas clínicos) y si el modelo fue sometido a ajuste fino con LoRA. 56

4.1. Matriz comparativa de precisión de los siete modelos base en función del Prompt empleado sobre el **Corpus IA** (125 instancias, anotación semi-automática). El comportamiento de `Qwen3.5-4B` con precisión perfecta en todos los prompts justifica su elección como modelo de referencia superior en la fase de ajuste fino. 58

4.2. Matriz comparativa de precisión de los siete modelos base en función del prompt empleado sobre el **Corpus Médicos** (299 instancias, anotación supervisada por cardiólogos). Cada valor representa la proporción de abreviaturas correctamente desambiguadas sobre el total de instancias evaluadas. 58

4.3. Comparativa del impacto del ajuste fino LoRA sobre el Corpus Médicos (299 instancias) con el Prompt 2. El impacto de LoRA varía por familia y escala: degrada la precisión en la familia Qwen (de forma más severa cuanto menor es el modelo) y la mejora en Gemma-3-1b-it. 62

4.4. Métricas empíricas de huella computacional, velocidad de procesamiento y uso de VRAM en inferencia local con hardware restringido. 65

4.5. Inventario analítico de las abreviaturas clínicas con mayor resistencia predictiva. 68

Capítulo 1

Introducción

Garantizar una comunicación fluida entre facultativos es un eje central para la continuidad asistencial; sin embargo, la saturación de siglas y abreviaciones en los informes médicos representa hoy un obstáculo crítico. Si bien estos recursos lingüísticos simplifican la redacción, su carácter polisémico, por el cual un término idéntico puede referirse a nociones opuestas dependiendo del contexto, conlleva un peligro latente de interpretación errónea. Esta dificultad no solo aleja al paciente de la comprensión de su propio estado, sino que fomenta errores de comunicación entre especialistas, lo que compromete directamente la seguridad clínica y el rigor en el seguimiento del caso.

Ante este panorama, la evolución del Procesamiento de Lenguaje Natural (PLN) y el auge de los Modelos de Lenguaje de Gran Escala (Large Language Models, LLMs) y de pequeña escala (Small Language Models, SLMs) ofrecen una vía inédita para solventar la Desambiguación de Sentido de Palabras (*Word Sense Disambiguation*, WSD) en el entorno biomédico. Estas tecnologías son capaces de examinar el entorno semántico y sintáctico de un documento para asignar un significado único a cada abreviatura. De este modo, se superan las rigideces de los glosarios médicos estáticos, logrando una adaptación precisa a las particularidades del lenguaje clínico real.

Bajo esta premisa, el presente Trabajo de Fin de Grado se centra en la evaluación experimental de distintos modelos de lenguaje aplicados a la desambiguación automática de terminología médica en informes hospitalarios. A través de la implementación de metodologías avanzadas de PLN y la experimentación sistemática con diversos modelos de aprendizaje profundo, se persigue determinar qué configuraciones de modelo y prompting permiten identificar y desplegar siglas complejas con mayor precisión. El propósito último es aportar evidencia empírica que oriente futuras decisiones sobre cómo lograr que la documentación clínica sea nítida, íntegra y carezca de ambigüeda-

des, fortaleciendo así la protección del paciente y la interoperabilidad de la información en el sistema de salud.

1.1. Propósito principal

La idea se centra en evaluar, mediante experimentación sistemática, en qué medida los modelos de lenguaje actuales son capaces de aclarar siglas y abreviaciones médicas en los informes hospitalarios, apoyándose en el Procesamiento del Lenguaje Natural y sus capacidades. El objetivo de este trabajo es comparar distintas arquitecturas y estrategias de prompting para identificar cuál ofrece el mejor equilibrio entre precisión y viabilidad de despliegue. El propósito es generar evidencia que facilite, en última instancia, la lectura de la documentación clínica a los profesionales de la salud, reduciendo así los peligros que conlleva la polisemia y la gran diversidad de términos que se manejan en el hospital. Al determinar qué configuración es capaz de examinar el contexto para recuperar el significado exacto de cada acrónimo, se aporta evidencia sobre la fiabilidad que puede alcanzar este tipo de enfoques. Esto sienta las bases para agilizar el intercambio de datos entre especialistas y minimizar los fallos en la toma de decisiones, lo que se traduciría en una asistencia médica más segura, eficaz y de calidad.

1.2. Motivación

Entender un informe médico es, a menudo, un reto que comparten tanto pacientes como profesionales. Estos documentos suelen estar saturados de siglas y tecnicismos que pueden confundir al destinatario e incluso a los propios expertos. Si la redacción no es clara, el riesgo de malentendidos o errores de interpretación aumenta, lo que puede acabar derivando en decisiones clínicas equivocadas. Por eso, simplificar estos textos es hoy una necesidad urgente para que la comunicación sea fluida y la información crítica se transmita sin distorsiones.

El uso masivo de terminología abreviada en el ámbito clínico se ha convertido en un desafío serio para la seguridad de la asistencia. Algunos estudios internacionales, como el realizado en un hospital de Hong Kong, han llegado a detectar casi cincuenta variantes de siglas no aprobadas cuyo uso desaconsejan listas de seguridad basadas en las recomendaciones del ISMP (Instituto para el Uso Seguro de los Medicamentos) [2] por el riesgo que conllevan. Sin embargo, hay autores que sugieren que la saturación terminológica es todavía más profunda: se han llegado a contabilizar más de 33 abreviaturas de media por cada prescripción, identificando hasta 70 tipos distintos de siglas en uso.[3] El ecosistema de las siglas médicas es tan complejo que incluso el conocimiento especializado se queda corto a la hora de interpretarlas correctamente. Los análisis revelan una brecha de comunicación interna preocupante:

se han registrado tasas de error cercanas al 43 % entre profesionales de una misma especialidad quirúrgica cuando intentan descifrar las abreviaturas de sus propios compañeros. Si la comprensión cae a estos niveles entre personal del mismo área, la situación se agrava al involucrar a otros grupos profesionales. Esto confirma que el informe clínico, lejos de ser un canal de información fluido, se ha transformado en una especie de texto cifrado que urge normalizar [4].

En esta línea, el propósito de la desambiguación automática es precisamente aclarar la terminología ambigua para que los informes sean mucho más accesibles. Y es que, para un paciente, suele ser casi imposible descifrar el significado de ciertas siglas sin que un médico se lo explique [5]. Pero el problema no acaba ahí: los propios facultativos, especialmente los menos experimentados, también encuentran barreras, ya que a veces se topan con abreviaturas que son exclusivas de un hospital determinado o incluso de un servicio concreto.

Por todo ello, automatizar la desambiguación de estos términos se plantea como una solución innovadora. El objetivo es que los informes hospitalarios dejen de ser textos herméticos y se conviertan en documentos comprensibles, garantizando la seguridad y la claridad tanto para el personal sanitario como para los pacientes.

1.3. Hipótesis principal

La hipótesis central de este proyecto sostiene que el uso de la inteligencia artificial, y más concretamente de técnicas avanzadas de Procesamiento del Lenguaje Natural, puede simplificar drásticamente la lectura de los informes hospitalarios. La idea es que, a través de los modelos derivados de estas tecnologías, se logre desambiguar con éxito las siglas, acrónimos y símbolos que saturan estos documentos. Se prevé que un sistema con estas capacidades mejore la accesibilidad de la documentación clínica, eliminando confusiones y aportando nitidez a la información. En última instancia, esto facilitará una comunicación mucho más real y efectiva entre los propios sanitarios, así como entre médico y paciente, impulsando un modelo asistencial más seguro y volcado en las necesidades del paciente.

1.4. Objetivos

El presente Trabajo de Fin de Grado tiene como objetivo principal evaluar, mediante experimentación sistemática, distintos modelos de lenguaje aplicados a la desambiguación automática de siglas y abreviaciones médicas en informes hospitalarios,

utilizando técnicas avanzadas de PLN, con el fin de determinar qué configuraciones contribuyen mejor a la accesibilidad y comprensión de la documentación clínica, garantizando la seguridad y claridad de la información para profesionales de la salud y pacientes.

Para alcanzar este propósito, se plantea el siguiente objetivo general, que se desglosará en una serie de objetivos específicos:

- **Objetivo general:** El objetivo principal es evaluar, mediante experimentación sistemática con distintos modelos de lenguaje, la viabilidad de la desambiguación automática de siglas y abreviaciones médicas. La investigación se centrará en determinar qué arquitectura ofrece el mejor rendimiento, priorizando un equilibrio óptimo: se busca la máxima fiabilidad en los resultados pero con el menor consumo de recursos posible, identificando así la configuración técnicamente más eficiente y robusta.

En base a este objetivo general, se han definido los siguientes objetivos específicos:

1. Analizar el repertorio de siglas, abreviaturas y símbolos empleados habitualmente en la documentación clínica, identificando aquellos modelos de lenguaje que, de forma nativa, demuestren una mayor capacidad para interpretarlos.
2. Poner a prueba el rendimiento de distintos modelos de lenguaje, desde opciones más ligeras hasta arquitecturas punteras, para examinar cómo expanden siglas y abreviaturas sin alterar el sentido ni la integridad de los datos médicos.
3. Construir una biblioteca de abreviaciones clínicas adaptada específicamente al corpus del proyecto, de modo que funcione como una base de conocimiento sólida para que el sistema no cometa errores de base al desambiguar.
4. Perfeccionar el prompting para que las instrucciones sean capaces de interpretar correctamente cada sección del informe (desde la anamnesis hasta el tratamiento). El reto es lograr que un mismo prompt sea lo bastante preciso para reconocer el contexto específico de cada apartado, garantizando que tanto la terminología general como los fármacos se desambigüen bien en cualquier circunstancia.
5. Implementar un pipeline experimental reproducible y parametrizable mediante fichero de configuración, que permita lanzar y comparar de forma sistemática las distintas combinaciones de modelo y prompt sobre el corpus de evaluación.
6. Recoger todo el proceso y los resultados en la memoria, reflexionando sobre cómo los hallazgos de esta evaluación pueden contribuir, en un futuro desarrollo,

a reducir las barreras de comunicación y a fortalecer la seguridad en el entorno clínico.

1.5. Cumplimiento de los Objetivos de Desarrollo Sostenible

Este proyecto se alinea con la hoja de ruta de los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de la ONU[1]. En las próximas líneas, se detalla cómo el sistema propuesto contribuye a estas metas, no solo desde una vertiente social y sanitaria, sino también desde un enfoque de eficiencia. Al priorizar modelos que optimizan el consumo de recursos computacionales, el proyecto busca reducir la huella tecnológica, demostrando que la innovación en salud puede ir de la mano con el respeto al planeta y la sostenibilidad.

ODS 3: Salud y Bienestar

> *"El objetivo es lograr la cobertura sanitaria universal y proporcionar acceso a medicamentos y vacunas seguros y asequibles para todos."*

DesambiMED contribuye a cumplimentar la **Meta 3.8**, ayudando a la cobertura sanitaria universal. Al mejorar la comprensión de los informes médicos, se facilita la comunicación entre médicos y pacientes y entre profesionales de la salud, lo que a su vez mejora la calidad de la atención médica. Al desambiguar las siglas y abreviaciones se consigue:

- **Potenciar la alfabetización en salud:** Facilitar que el paciente comprenda su proceso clínico, transformando términos técnicos en información accesible.
- **Incrementar la adherencia terapéutica:** Favorecer el cumplimiento del tratamiento al eliminar las dudas derivadas de instrucciones médicas abreviadas o ambiguas.
- **Mitigar riesgos derivados de errores de interpretación:** Reducir la probabilidad de eventos adversos causados por una lectura incorrecta de siglas y abreviaturas críticas.

[1] `https://www.un.org/sustainabledevelopment/es/objetivos-de-desarrollo-sostenible/`

- **Impulsar un modelo de atención centrado en el paciente:** Promover la autonomía y el empoderamiento del usuario mediante la transparencia y la claridad en la comunicación clínica.

ODS 4: Educación de Calidad

> *"La educación ayuda a las personas de todo el mundo vivir una vida más saludable y sostenible."*

El proyecto conecta también con la **Meta 4.7**, que busca promover el aprendizaje de conocimientos prácticos que favorezcan el desarrollo sostenible y la salud pública. Al solucionar el caos de las siglas clínicas, el sistema no solo procesa datos, sino que fomenta una verdadera alfabetización sanitaria. Esto permite que cualquier persona pueda comprender información técnica que, habitualmente, le resultaría indescifrable. En definitiva, se trata de empoderar al ciudadano para que tome las riendas de su salud con criterio, eliminando esas barreras del lenguaje que tantas veces impiden que los pacientes ejerzan su derecho a estar bien informados.

- **Fomento de la alfabetización técnica y sanitaria:** Contribuir a que los ciudadanos adquieran las competencias necesarias para interpretar información médica compleja, facilitando la asimilación de conocimientos prácticos sobre su propia salud y bienestar.
- **Promoción de la autonomía en la toma de decisiones:** Dotar al paciente de las herramientas lingüísticas para comprender diagnósticos y tratamientos, permitiendo que participe de manera informada y activa en su proceso clínico, un pilar del desarrollo social sostenible.
- **Reducción de la brecha de conocimiento especializado:** Democratizar el acceso a la información hospitalaria mediante la normalización de siglas, asegurando que el lenguaje técnico no sea un obstáculo para la adquisición de conocimientos sobre cuidados y prevención.
- **Fortalecimiento de la comunicación inclusiva y universal:** Garantizar que los documentos clínicos cumplan con principios de claridad y transparencia, promoviendo una cultura de respeto al derecho a entender, tal como impulsa la agenda de educación global para el desarrollo.

ODS 10: Reducción de las Desigualdades

> *"Reducir la desigualdad en y entre los países."*

La propuesta se vincula estrechamente con la **Meta 10.2**, orientada a fomentar la inclusión y el empoderamiento de toda la sociedad, sin distinciones. Al despejar la confusión que generan las siglas y el lenguaje técnico, DesambiMED promueve la igualdad en el acceso a la salud y defiende el derecho a la comprensión de los datos propios, poniendo el foco en:

- Usuarios con niveles de alfabetización reducidos que se ven desbordados por el exceso de tecnicismos.
- Pacientes en condiciones de vulnerabilidad cognitiva para quienes cualquier ambigüedad en el informe puede derivar en un peligro asistencial.
- Colectivos no nativos que necesitan que la comunicación médica sea lo más clara y estandarizada posible para evitar malentendidos.

ODS 12: Garantizar modalidades de consumo y producción sostenibles

"Garantizar modalidades de consumo y producción sostenibles."

El desarrollo de este sistema se alinea con el **ODS 12**, que promueve modelos de consumo y producción sostenibles, conectando directamente con la **Meta 12.2**. Este objetivo busca una gestión eficiente de los recursos; un concepto que, en el entorno sanitario, se traduce en aprovechar al máximo los activos informativos y operativos. DesambiMED impulsa esta sostenibilidad a través de los siguientes puntos:

- **Aprovechamiento del capital informativo:** Los resultados obtenidos muestran que un sistema basado en esta aproximación podría asegurar que los datos de cada informe sean realmente útiles y precisos.
- **Reducción de ineficiencias asistenciales:** Al ofrecer una lectura clara y directa, se reduce la necesidad de duplicar pruebas diagnósticas o de programar consultas adicionales solo para aclarar dudas. Esto permite optimizar el uso de las instalaciones hospitalarias y, sobre todo, el tiempo del personal sanitario.
- **Sostenibilidad en la gestión de datos:** Se apuesta por un modelo de comunicación clínica mucho más responsable. El rigor en el lenguaje garantiza que cada documento sea una herramienta fiable a largo plazo para cualquier profesional que deba intervenir en el proceso de salud [6].

ODS 13: Acción por el Clima

> *"Adoptar medidas urgentes para combatir el cambio climático y sus efectos."*

El proyecto se alinea con la **Meta 13.2**, que insta a integrar medidas relativas al cambio climático en las políticas y estrategias de desarrollo. El entrenamiento de modelos de lenguaje de gran escala conlleva una huella de carbono considerable: Strubell et al. estimaron que el entrenamiento de un modelo Transformer con búsqueda de hiperparámetros equivale a emitir aproximadamente 284 toneladas de CO_2, cifra comparable a cinco veces las emisiones de vida útil de un automóvil medio[7]. Schwartz et al. acuñaron el término *Green AI* para referirse al conjunto de prácticas orientadas a reducir este impacto, argumentando que el progreso científico debe medirse también en términos de coste computacional por punto de mejora[8]. DesambiMED se inscribe conscientemente en esta línea:

- **Minimización de emisiones por entrenamiento:** La selección de modelos de menos de 5B de parámetros y el uso de ajuste fino eficiente mediante QLoRA reducen drásticamente el coste energético respecto al entrenamiento completo de modelos de gran escala, traduciendo esa reducción en menores emisiones de gases de efecto invernadero.
- **Inferencia local sin dependencia de la nube:** El despliegue en hardware hospitalario estándar elimina las transferencias de datos y el procesamiento continuo en centros de datos de alto consumo energético, contribuyendo a un ciclo de vida del software más sostenible.
- **Principio de eficiencia como criterio de diseño:** La priorización explícita del balance rendimiento/coste computacional a lo largo de todo el trabajo refleja el compromiso con el paradigma de la *Green AI*, demostrando que la innovación en salud puede avanzar sin incrementar proporcionalmente la huella de carbono del sector tecnológico.

ODS 15: Vida de ecosistemas terrestres

> *"Proteger, restablecer y promover el uso sostenible de los ecosistemas terrestres."*

El proyecto se vincula con el **ODS 15**, relacionándose específicamente con la **Meta 15.1**, orientada a proteger y restaurar los ecosistemas terrestres y los recursos hídricos de los que dependen. El entrenamiento e inferencia de modelos de lenguaje de gran escala exige volúmenes significativos de agua

para la refrigeración de los centros de datos: Li et al. estimaron que el entrenamiento de modelos como GPT-3 puede consumir entre 700.000 y 3,5 millones de litros de agua[9], ejerciendo una presión directa sobre los acuíferos locales y los ecosistemas que dependen de ellos. Al priorizar modelos compactos ejecutables en hardware hospitalario estándar sin centros de datos dedicados, DesambiMED contribuye a reducir esta huella hídrica. En concreto:

- **Reducción del consumo hídrico en inferencia:** Los modelos SLM (<5B parámetros) requieren una fracción del agua necesaria para refrigerar los servidores que ejecutan modelos de 70B+ parámetros, aliviando la presión sobre los recursos de agua dulce locales.
- **Despliegue local frente a centros de datos remotos:** La ejecución en nodos de la red informática hospitalaria elimina la dependencia de grandes infraestructuras de cómputo en la nube, cuya demanda hídrica y térmica puede afectar negativamente a los ecosistemas terrestres circundantes.
- **Promoción de una computación responsable con el entorno:** El enfoque en la eficiencia algorítmica demuestra que es posible obtener resultados clínicamente admisibles sin recurrir a arquitecturas masivas (cuyo ciclo de vida completo, desde la fabricación del hardware hasta la operación continua, genera un impacto ambiental acumulado significativo sobre los ecosistemas locales).

En definitiva, DesambiMED no se limita a un ejercicio técnico de procesamiento de lenguaje natural; sus hallazgos aportan una base empírica con potencial de impacto social. Este trabajo responde a los principios de equidad, acceso universal e inclusión que dan sentido a la Agenda 2030, mostrando que la tecnología puede ser, en un futuro desarrollo, el motor para construir un sistema de salud más justo, transparente y respetuoso con el planeta.



Figura 1.1: Objetivos de Desarrollo Sostenible cumplimentados por este proyecto. Fuente: `https://www.pactomundial.org/que-puedes-hacer-tu/ods/`

1.6. Estructura de la memoria

La estructura del presente trabajo está organizada en los siguientes capítulos:

Capítulo 1. Introducción: Se establece el marco general del proyecto, definiendo el propósito principal de DesambiMED como estudio de evaluación comparativa de modelos de lenguaje aplicados a la desambiguación automática de terminología médica en informes hospitalarios y la mejora de eficiencia con respecto a los modelos usados actualmente en relación con los ODS.

Capítulo 2. Estado del arte: Se proporciona una revisión exhaustiva del estado del arte en WSD en el dominio biomédico y procesamiento del lenguaje natural clínico. Se analizan los enfoques basados en reglas, aprendizaje automático y modelos de lenguaje de gran escala, con especial atención a los modelos compactos y eficientes (Qwen, Gemma, Granite) y a las técnicas de ajuste fino. Se contextualiza la escasez de recursos para el español clínico y se identifican las limitaciones de los trabajos previos que justifican el presente proyecto.

Capítulo 3. Metodología y diseño experimental: Se detalla la propuesta técnica de DesambiMED: origen y procesamiento del corpus de informes de alta hospitalaria, construcción del diccionario de abreviaciones, diseño de los tres prompts evaluados y criterios de selección de los siete modelos de lenguaje. Se describe la fase de ajuste fino con LoRA/QLoRA, la infraestructura hardware y software empleada, y las métricas de evaluación utilizadas. Se incluyen también los retos técnicos encontrados durante el proceso experimental y las soluciones adoptadas.

Capítulo 4. Análisis de resultados y discusión: Se presentan y discuten los resultados obtenidos en las distintas fases experimentales: evaluación de los modelos base con los tres prompts, impacto de la ingeniería de prompts, rendimiento de los modelos ajustados finamente mediante LoRA y análisis de eficiencia computacional. Se incluye una auditoría forense de los errores más relevantes y se discuten las implicaciones clínicas del sistema.

Capítulo 5. Herramienta de anotación: Se presenta el desarrollo de la plataforma web interactiva creada para la anotación semi-automática del corpus. Se describen sus objetivos, requisitos funcionales y no funcionales, la arquitectura cliente-servidor, el módulo de detección de abreviaturas, el flujo de anotación y los modos de uso: anotación inicial y revisión por parte de los especialistas clínicos. Se documentan las principales mejoras introducidas a lo largo de las iteraciones del proyecto.

Capítulo 6. Conclusiones y líneas de trabajo futuro: Se recapitulan los hallazgos principales del trabajo, se evalúa el cumplimiento de los objetivos establecidos y se identifican las limitaciones del sistema. Se proponen líneas de investigación fu-

tura orientadas a la ampliación del corpus, mejoras en el ajuste fino, integración en sistemas clínicos reales y extensión a otros tipos de terminología médica.

Con esta estructura, se espera proporcionar una comprensión integral del proceso de investigación y desarrollo llevado a cabo, así como una evaluación rigurosa de la viabilidad de los modelos de lenguaje pequeños para la desambiguación automática de terminología clínica en español.

1.7. Planificación y costes

1.7.1. Planificación del proyecto

El objetivo de la planificación es estructurar y organizar el desarrollo del proyecto de forma eficiente, permitiendo establecer estimaciones temporales razonables para cada fase. Esta planificación ha sido diseñada de forma flexible para adaptarse a imprevistos y se representa mediante un diagrama de Gantt, como puede verse en la Figura 1.2.

El proyecto se ha articulado en cinco grandes bloques de trabajo que se han solapado parcialmente a lo largo de aproximadamente nueve meses (octubre de 2025 – julio de 2026):

1. **Desarrollo de la herramienta de anotación** (octubre 2025 – marzo 2026): diseño e implementación iterativa de la plataforma web para la anotación semiautomática del corpus, incluyendo el módulo de detección de abreviaturas y el modo de revisión para especialistas clínicos.
2. **Obtención y procesado del corpus** (octubre 2025 – junio 2026): adquisición y anonimización de los informes hospitalarios cedidos por el Hospital Universitario de Jaén, generación de anotaciones automáticas y recogida de las anotaciones realizadas por los cardiólogos.
3. **Prompting y evaluación de modelos base** (abril – junio 2026): diseño de los tres prompts experimentales, ejecución de las 21 combinaciones modelo-prompt sobre el corpus anotado por IA y posterior evaluación con los textos revisados por los médicos.
4. **Ajuste fino con LoRA** (abril – junio 2026): prototipado del script de entrenamiento, ajuste de los cuatro modelos seleccionados y evaluación de los modelos resultantes.

5. **Redacción de la memoria** (mayo – julio 2026): elaboración progresiva de los capítulos del documento en LaTeX, con revisiones sucesivas hasta la entrega final prevista el **10 de julio de 2026**.

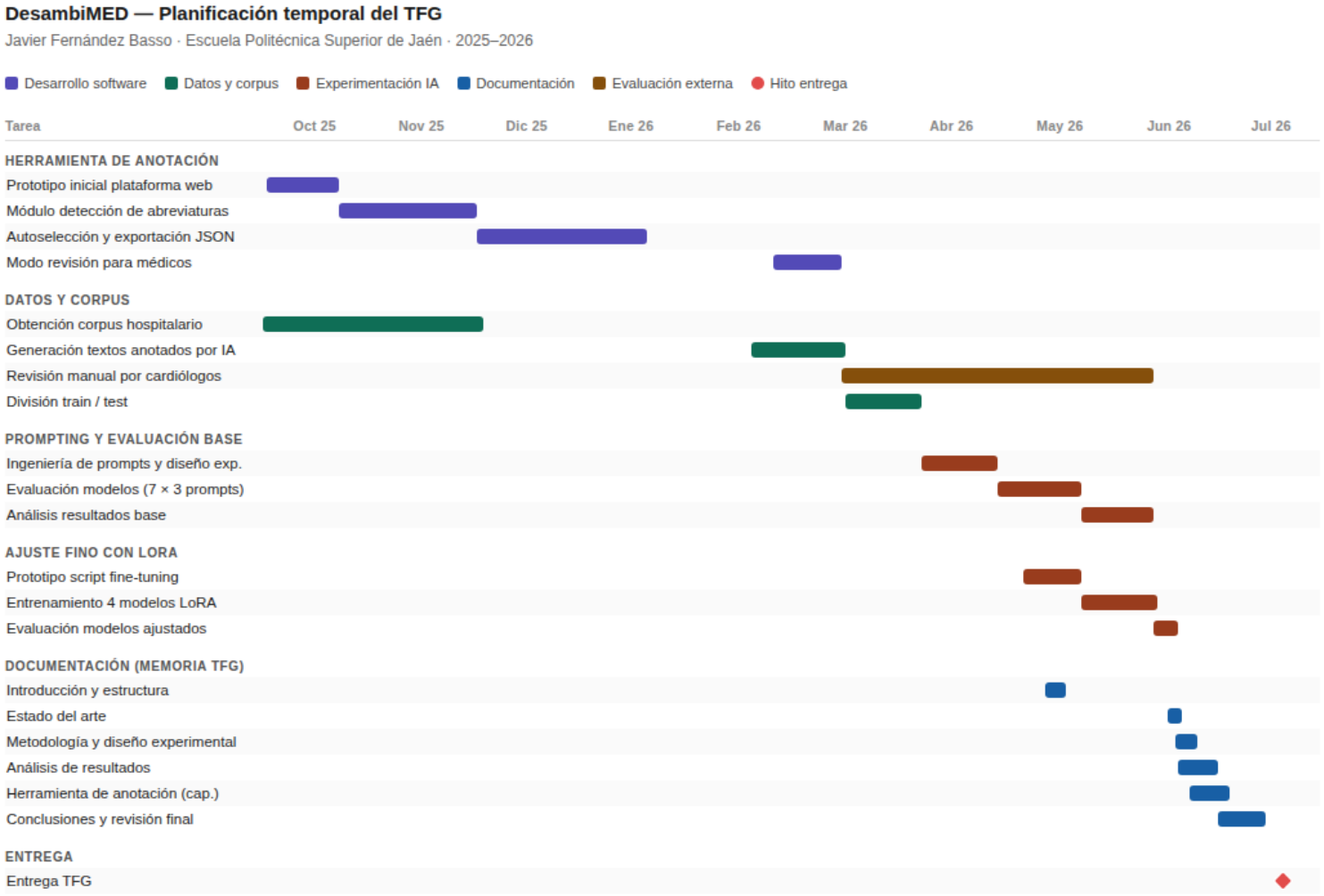


Figura 1.2: Planificación temporal del TFG (diagrama de Gantt). Se muestran las cinco fases principales del proyecto desde octubre de 2025 hasta julio de 2026.

1.7.2. Costes del proyecto

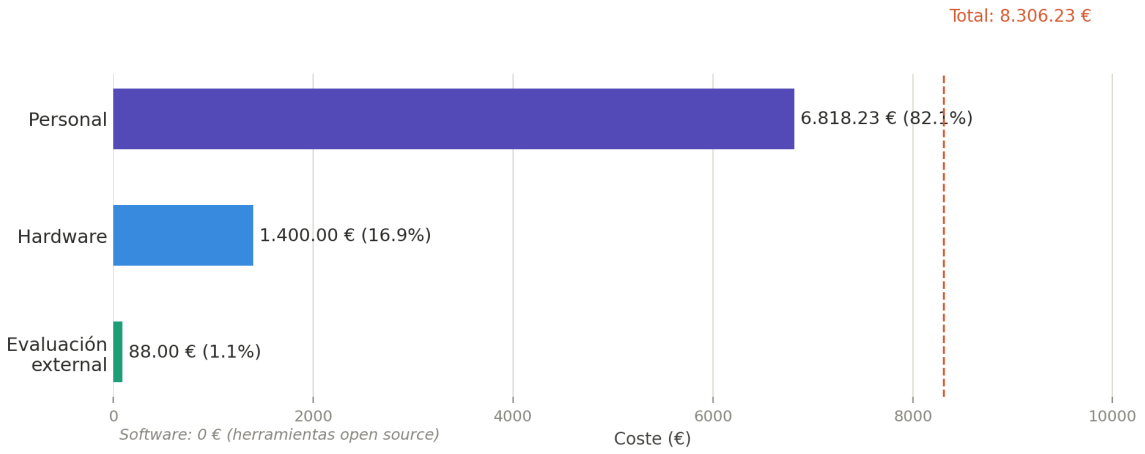


Figura 1.3: Desglose del presupuesto estimado del proyecto por partidas. El coste de personal representa el 82,1 % del total.

A continuación se describen los recursos empleados y su valoración económica aproximada.

Costes de hardware: 1.400 €

- Ordenador personal con CPU Intel i7 14700KF y 32 GB de RAM: 700 €
- GPU NVIDIA RTX 5070 OC para entrenamiento: 700 €

Costes de software: 0 €

- Ubuntu, Python, Visual Studio Code
- Transformers, PyTorch, PEFT y otras librerías open source

Coste de personal: 6.818,23 €

Se ha seguido el convenio del BOE núm. 92, de 16 de abril de 2025[2], considerando el Área 4, grupo C, nivel 2, con un salario anual de 26.308,29 €. El proyecto se ha desarrollado a lo largo de nueve meses y medio (del 1 de octubre de 2025 al 10 de julio de 2026), equivalentes a aproximadamente 9,33 meses. Al tratarse de una dedicación a tiempo parcial, se estima una equivalencia de un tercio de la jornada completa, lo que resulta en 3,11 meses efectivos de trabajo:

$$\frac{26{,}308{,}29}{12} \times 3{,}11 \approx 6{,}818{,}23\,€$$

Coste de anotación externa: 88 €

[2] https://www.boe.es/boe/dias/2025/04/16/pdfs/BOE-S-2025-92.pdf

La evaluación fue realizada por dos cardiólogos expertos del Hospital Universitario de Jaén durante una mañana completa (4 horas en total), con una tarifa de 11 €/hora por persona, tomando como referencia el punto medio del rango habitual para este tipo de colaboración (10–12 €/hora):

$$2 \times 4 \times 11 = 88\,€$$

Coste total estimado: 8.306,23 €

Capítulo 2

Estado del Arte

El presente capítulo revisa el conocimiento acumulado en torno a los ejes fundamentales de este trabajo: el procesamiento del lenguaje natural aplicado al dominio clínico, las técnicas de desambiguación de sentido de palabras en texto biomédico, los modelos de lenguaje de pequeña y mediana escala que equilibran rendimiento y eficiencia, y los recursos y corpus específicos para el español clínico. El objetivo es situar la propuesta en el panorama actual e identificar las brechas que este trabajo pretende cubrir.

2.1. Procesamiento del lenguaje natural en el dominio clínico

El PLN es la rama de la inteligencia artificial que estudia cómo los sistemas informáticos pueden comprender, generar y manipular el lenguaje humano. Desde los enfoques basados en reglas de los años 60 hasta los actuales LLMs, la disciplina ha experimentado avances que han abierto nuevas posibilidades en numerosos dominios de aplicación, entre ellos la salud.

El texto clínico presenta características que lo diferencian del lenguaje general: alta densidad terminológica, uso intensivo de abreviaturas y símbolos, sintaxis telegráfica y fuerte dependencia del contexto especializado. Estas particularidades hacen que los modelos entrenados sobre corpus generales rindan de forma subóptima cuando se aplican directamente a documentación hospitalaria, lo que ha impulsado el desarrollo de técnicas y recursos específicos para este dominio.

El punto de inflexión tecnológico que ha permitido abordar estas complejidades lingüísticas con garantías se produjo con la introducción de la arquitectura Transformer

por Vaswani et al. (2017)[1]. Este diseño revolucionó el procesamiento del lenguaje al reemplazar las estructuras recurrentes y convolucionales tradicionales por el mecanismo de autoatención (self-attention), el cual capacita al sistema para computar relaciones de dependencia entre palabras sin importar su distancia en el texto y permite un entrenamiento masivo en paralelo. A partir de esta arquitectura, el campo ha evolucionado mediante tres variantes principales: los modelos basados en el codificador (encoder), como BERT, especializados en la comprensión bidireccional del contexto; los modelos basados en el decodificador (decoder), como la familia GPT, orientados a la generación autorregresiva de texto; y las arquitecturas híbridas de codificador-decodificador, como T5, que unifican cualquier tarea lingüística bajo un paradigma de secuencia a secuencia (véase la Figura 2.1).

La explotación operativa de estos modelos en entornos de alta especialización se sustenta en el concepto de transfer learning o aprendizaje por transferencia. Bajo esta metodología, un modelo es preentrenado inicialmente con corpus genéricos de escala masiva para adquirir un conocimiento gramatical y semántico general, para posteriormente ser sometido a un proceso de ajuste fino (fine-tuning) con datos específicos de una disciplina. En el ámbito clínico y biomédico, esta aproximación ha resultado indispensable debido al coste y la escasez de datos anotados por profesionales, dando lugar al nacimiento de modelos especializados como BioBERT[10] o ClinicalBERT[11]. Estas variantes demuestran que transferir las capacidades de una arquitectura general hacia representaciones adaptadas al argot hospitalario y a la literatura médica mejora drásticamente el rendimiento en tareas críticas como el reconocimiento de entidades clínicas, la extracción de relaciones y la desambiguación terminológica.

2.1.1. Texto clínico: características y retos

Los informes hospitalarios, y en particular los informes de alta, constituyen uno de los tipos documentales más complejos del ámbito sanitario. Se caracterizan por una elevada densidad de abreviaturas, siglas y acrónimos cuyo significado puede variar según el servicio, el hospital o incluso el redactor. Estudios previos han cuantificado esta problemática: se han identificado más de 33 abreviaturas de media por prescripción médica, con hasta 70 tipos distintos de siglas en uso simultáneo. Más preocupante aún resulta el hecho de que las tasas de error en la interpretación de abreviaturas entre profesionales de una misma especialidad quirúrgica pueden superar el 40 %[3].

La naturaleza ambigua o polisémica de estas formas abreviadas, por la que un mismo término puede referirse a conceptos opuestos dependiendo del contexto clínico, supone un riesgo real para la seguridad asistencial. La desambiguación automática

de este tipo de terminología se erige, por tanto, como una necesidad prioritaria para garantizar la calidad y la seguridad de la comunicación clínica.

En el ámbito hispanohablante, diversos esfuerzos institucionales han permitido acotar y cuantificar empíricamente este escenario. El Instituto de Lingüística Aplicada (IULA)[1] de la Universitat Pompeu Fabra ha sido pionero en este análisis mediante el estudio de historias clínicas en entornos de referencia como el Hospital del Mar, evi-

[1] `https://www.upf.edu/es/web/iula/corpus`

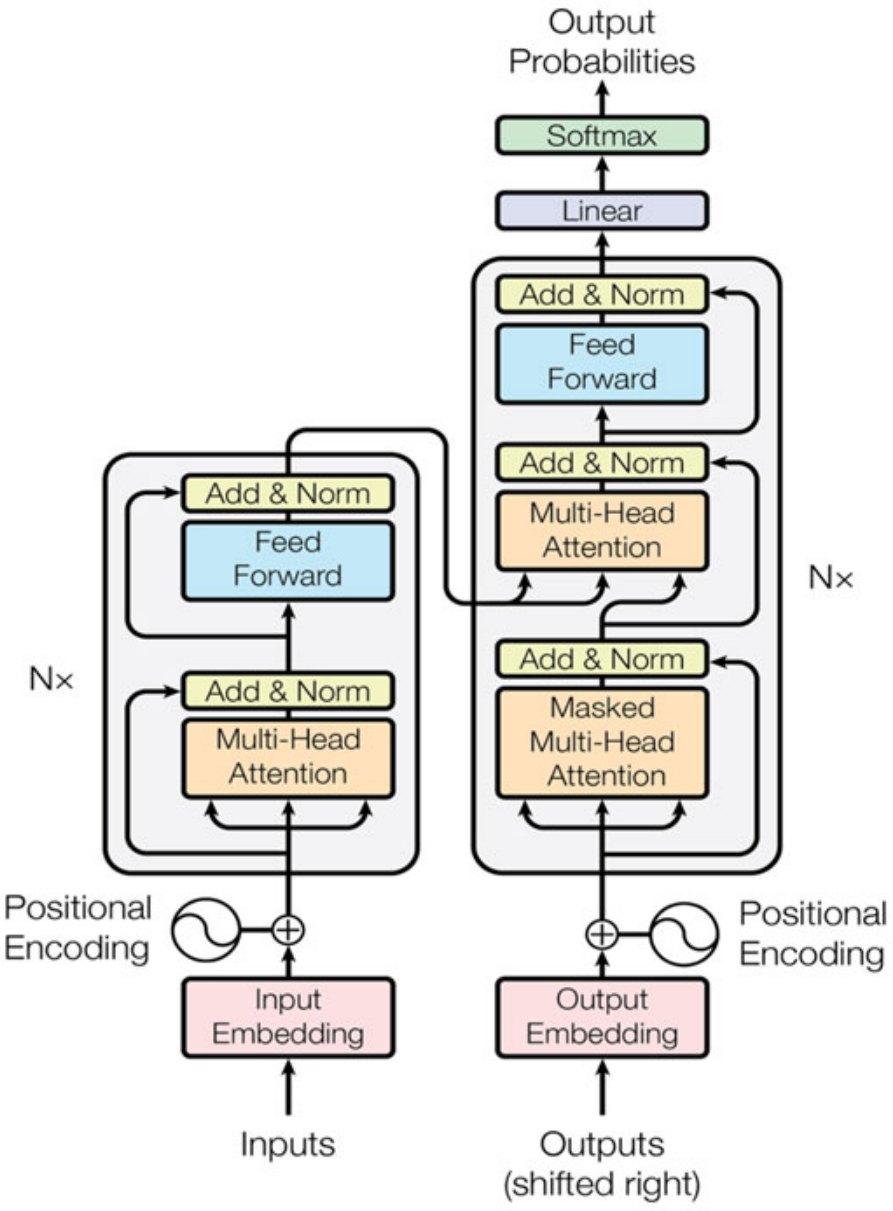


Figura 2.1: Esquema conceptual de la arquitectura *Transformer* (Vaswani et al., [1]). Se detalla la estructura basada en bloques acoplados de codificación (*encoder*) y decodificación (*decoder*), destacando los módulos de atención multi-cabeza y las redes de propagación hacia adelante (*feed-forward*).

denciando que las abreviaciones constituyen el fenómeno no estándar más frecuente en la documentación clínica. En esta misma línea, un estudio descriptivo de la Universidad de Zaragoza realizado en el Hospital Clínico Universitario Lozano Blesa analizó la presencia de estas formas en informes de alta e historias clínicas, registrando una media de casi 22 abreviaturas por documento. El hallazgo más crítico de dicha investigación reveló que el 36,92 % de las siglas analizadas presentaban más de un significado posible y un 27 % de los casos constituían un error potencial de uso, confirmando la falta de consenso formal en el registro de datos asistenciales.[12]

Esta realidad lingüística y su consecuente riesgo asistencial quedan también respaldados a gran escala por el corpus SPACCC (Spanish Clinical Case Corpus)[2], desarrollado por el Barcelona Supercomputing Center. A través de iniciativas de evaluación como la tarea compartida BARR2 (Abbreviation Recognition and Resolution)[13], implementada sobre este recurso, se constató una presencia masiva de formas abreviadas sin su correspondiente definición explítica en el texto. Los resultados de este proyecto de minería de textos no solo pusieron de manifiesto la baja consistencia que los facultativos emplean al registrar acrónimos idénticos para diagnósticos dispares, sino que consolidaron la desambiguación de siglas en español como uno de los desafíos más complejos e imperativos para la inteligencia artificial clínica[14].

2.2. Desambiguación de sentido de palabras en texto biomédico

La Desambiguación de Sentido de Palabras es la tarea de PLN que consiste en determinar, dado un término ambiguo en su contexto, cuál de sus posibles acepciones es la más apropiada. En el dominio biomédico, esta tarea adquiere una dimensión crítica, pues un error de desambiguación puede derivar en consecuencias clínicas graves.

2.2.1. Enfoques clásicos: reglas y diccionarios

Los esfuerzos por sistematizar la evaluación de sistemas de WSD en texto biomédico dieron lugar a *benchmarks* de referencia ampliamente adoptados. El más influyente de ellos es el *NLM-WSD dataset*, compilado por el National Library of Medicine de Estados Unidos, que agrupa 50 términos biomédicos altamente ambiguos extraídos de MEDLINE junto con sus instancias anotadas manualmente con el sentido correcto se-

[2] `https://huggingface.co/datasets/IEETA/SPACCC-Spanish-NER`

gún el Metatesauro del UMLS [15]. Este recurso estableció el marco de evaluación sobre el que se han medido los avances de la disciplina durante más de dos décadas, permitiendo comparaciones sistemáticas entre aproximaciones metodológicas muy distintas.

Los primeros sistemas que operaron sobre este tipo de recursos emplearon estrategias basadas en reglas y en la coincidencia con vocabularios controlados. El trabajo pionero de Schwartz y Hearst propuso un algoritmo de extracción de definiciones de acrónimos a partir de patrones textuales del tipo *"forma larga (abreviatura)"*, logrando una precisión superior al 96 % en la identificación de pares acrónimo-expansión cuando la definición aparece explícitamente en el texto [16]. Sin embargo, este enfoque resulta inaplicable cuando la abreviatura no viene acompañada de su expansión en el documento, situación habitual en los informes de alta hospitalaria, donde se asume un conocimiento compartido entre redactor y destinatario.

Para abordar este escenario, Liu et al. reformularon el problema como una tarea de clasificación supervisada, aprendiendo el sentido correcto a partir de rasgos del contexto local (palabras adyacentes, bigramas) y rasgos semánticos derivados del UMLS [17]. Su trabajo evidenció que la precisión de los sistemas se correlaciona fuertemente con la frecuencia de aparición del término en el corpus de entrenamiento, lo que constituye una limitación estructural para las abreviaturas de uso poco frecuente o de alcance local, precisamente las más problemáticas en la práctica clínica real. En una línea complementaria, Stevenson et al. desarrollaron un sistema basado en la similitud distribucional entre el contexto de la abreviatura y las definiciones de sus posibles expansiones en el UMLS, combinada con características morfosintácticas del entorno local [18]. Sus experimentos mostraron que la incorporación de información semántica del UMLS mejoraba la precisión respecto a los enfoques puramente distribucionales, aunque con una dependencia crítica de la cobertura del vocabulario controlado: las abreviaturas ausentes del Metatesauro suponían un techo insalvable para el sistema.

En el contexto español, el Diccionario de Siglas Médicas de la SEDOM (*Sociedad Española de Documentación Médica*)[3] constituye el recurso lexicográfico de referencia para este tipo de sistemas basados en vocabularios controlados. Con más de 15.000 entradas, cubre siglas y acrónimos de uso frecuente en la documentación sanitaria española [19]. No obstante, su cobertura es limitada frente a la variabilidad real del lenguaje clínico, especialmente en lo que respecta a abreviaturas propias de servicios específicos o de uso local, reproduciendo a menor escala las mismas limitaciones estructurales ya descritas para el UMLS en inglés.

[3] `http://diccionario.sedom.es/`

2.2.2. Enfoques basados en aprendizaje automático y modelos de lenguaje

La transición hacia arquitecturas de aprendizaje profundo se produjo con la adopción de redes LSTM (Long Short-Term Memory) bidireccionales con campos aleatorios condicionales (BiLSTM-CRF), que modelan la secuencia completa del contexto y capturan dependencias de largo alcance de forma más efectiva que los modelos de ventana fija [20]. Aplicadas a la desambiguación de abreviaturas clínicas, estas arquitecturas mostraron mejoras consistentes respecto a los sistemas basados en características discretas, especialmente en abreviaturas polisémicas cuya resolución requiere contexto de larga distancia. No obstante, su rendimiento está condicionado por la disponibilidad de embeddings preentrenados de calidad para el dominio clínico, una restricción que afecta de forma más severa al español que al inglés dado el menor tamaño de los corpus clínicos disponibles para este idioma.

La irrupción de los modelos de lenguaje contextuales, en especial BERT y sus variantes, supuso un punto de inflexión definitivo en la WSD biomédica. Estos modelos generan representaciones vectoriales sensibles al contexto, lo que los hace capaces de distinguir entre acepciones de una misma forma abreviada sin necesidad de definir reglas explícitas. Modelos como BioBERT o PubMedBERT, preentrenados sobre grandes corpus biomédicos, han demostrado mejoras sustanciales respecto a BERT genérico en tareas de reconocimiento de entidades nombradas, extracción de relaciones y desambiguación de abreviaturas [21]. Sin embargo, su tamaño (típicamente entre 110M y 340M de parámetros) y los requerimientos computacionales asociados limitan su viabilidad en entornos con recursos restringidos. Más recientemente, enfoques de *few-shot prompting* con modelos generativos han demostrado capacidad para desambiguar abreviaturas clínicas con tan solo dos o tres ejemplos en el *prompt*, apuntando a una estrategia viable cuando los datos anotados son escasos [22].

2.2.3. El problema específico de abreviaturas y símbolos en español

La mayor parte de la investigación en WSD biomédica se ha desarrollado sobre corpus en inglés, aprovechando la abundancia de recursos como MIMIC-III/IV o los datasets de los desafíos i2b2 y n2c2. La tarea PharmaCoNER, celebrada en IberLEF 2019, constituyó el primer *benchmark* de reconocimiento de entidades clínicas de referencia para el español, proponiendo el reconocimiento de entidades farmacológicas a partir del corpus SPACCC-PharmaCoNER [23]. Los mejores sistemas participantes combinaron arquitecturas BiLSTM-CRF con embeddings de BERT preentrenados so-

bre texto clínico, obteniendo valores de F1 por encima del 90 % en el reconocimiento de fármacos y principios activos.

La tarea CANTEMIST, celebrada en IberLEF 2020, amplió el alcance al reconocimiento y codificación de menciones de neoplasias en informes anatomopatológicos en español [24]. Su relevancia para el presente trabajo reside no tanto en la tarea en sí como en el corpus subyacente, que ilustra la alta densidad de abreviaturas y terminología especializada propia de los informes hospitalarios reales, y en las estrategias de normalización terminológica adoptadas por los sistemas participantes. Estos esfuerzos colectivos ponen de manifiesto que, si bien el panorama del español clínico ha mejorado notablemente, la desambiguación específica de abreviaturas y símbolos en informes de alta continúa siendo una tarea insuficientemente cubierta por los recursos y sistemas existentes.

2.3. Modelos de lenguaje pequeños y eficientes para PLN clínico

Un eje central de este trabajo es la búsqueda del equilibrio entre rendimiento y eficiencia computacional. Frente a modelos como GPT-4 o LLaMA 70B, que ofrecen capacidades excepcionales pero requieren infraestructuras de cómputo intensivo, el uso de modelos compactos (habitualmente denominados *Small Language Models* o modelos de menos de 10B de parámetros) se alinea tanto con criterios de sostenibilidad como con las restricciones reales de los entornos hospitalarios, donde la privacidad de los datos impide el uso de servicios en la nube.

2.3.1. Arquitecturas de referencia: IBM Granite, Qwen y Google Gemma

La selección de los modelos evaluados en este trabajo atiende al criterio de eficiencia computacional como restricción de diseño explícita, configurando un espacio de experimentación compuesto por siete variantes pertenecientes a tres familias de vanguardia.

IBM Granite representa una línea de modelos de código abierto desarrollada por IBM Research bajo licencia Apache 2.0 [25]. Para este estudio se han seleccionado las variantes *granite-4.0-h-350M* y *granite-4.0-h-1B*. Estos modelos destacan por su arquitectura hipercompacta y optimizada para entornos con severas restricciones de memoria, manteniendo el énfasis característico de la familia en la transparencia del

proceso de construcción del corpus y en la trazabilidad ética de sus datos de entrenamiento.

La familia Qwen, desarrollada por Alibaba Cloud, constituye uno de los referentes actuales en rendimiento multilingüe para modelos de escala reducida [26]. En este trabajo se evalúa la generación más reciente a través de sus variantes *Qwen3.5-0.8B*, *Qwen3.5-2B* y *Qwen3.5-4B*. Estos modelos ofrecen un soporte nativo robusto para el idioma español y han demostrado capacidades competitivas en tareas de comprensión lingüística y seguimiento de instrucciones frente a modelos que duplican su tamaño.

Por último, se incorpora la familia Gemma de Google DeepMind, diseñada específicamente para maximizar la eficiencia en hardware convencional a partir de las innovaciones de la arquitectura Gemini. Se evalúan los modelos ajustados a instrucciones *gemma-3-1B-it* y *gemma-3-4B-it*, los cuales aportan un excelente balance entre capacidad de razonamiento y coste computacional, cerrando un ecosistema experimental diverso en cuanto a filosofías de diseño y optimización. La Tabla 2.1 recoge de forma sintética las especificaciones técnicas de las siete variantes analizadas.

Modelo	**Parámetros**	**Contexto (tokens)**	**Licencia**	**Multilingüe**
granite-4.0-h-350M	350M	4.096	Apache 2.0	Parcial
granite-4.0-h-1B	1B	4.096	Apache 2.0	Parcial
Qwen3.5-0.8B	0,8B	32.768	Apache 2.0	Sí
Qwen3.5-2B	2B	32.768	Apache 2.0	Sí
Qwen3.5-4B	4B	32.768	Apache 2.0	Sí
gemma-3-1B-it	1B	32.768	Gemma Terms	Sí
gemma-3-4B-it	4B	32.768	Gemma Terms	Sí

Tabla. 2.1: Comparativa de los modelos evaluados en el presente trabajo.

La selección conjunta de estas tres familias permite plantear un análisis comparativo multidimensional. Al contrastar modelos que se sitúan por debajo de la barrera del billón de parámetros (como las variantes de 350M y 0.8B) frente a los SLMs más robustos del ecosistema (las variantes de 4B), este trabajo busca determinar con precisión el umbral de escala necesario para resolver con éxito la ambigüedad terminológica en el registro clínico español.

2.3.2. Técnicas de eficiencia: cuantización y LoRA

Para reducir aún más los requerimientos computacionales de los modelos, se han desarrollado técnicas que permiten operar con menor precisión numérica o con un subconjunto ajustable de parámetros. La cuantización consiste en representar los pesos del modelo con menos bits (típicamente 4 u 8 en lugar de 32), reduciendo sig-

nificativamente la memoria necesaria con una pérdida de rendimiento generalmente menor al 2–3 % en tareas de comprensión.

La técnica LoRA (*Low-Rank Adaptation*), propuesta por Hu et al., ofrece una solución elegante al problema del ajuste fino costoso: en lugar de modificar todos los pesos del modelo, se insertan matrices de bajo rango aditivas en las capas de atención, entrenando únicamente estas matrices mientras el resto del modelo permanece congelado [27]. El número de parámetros entrenables se reduce así en un factor de hasta 10.000 respecto al ajuste fino completo, lo que hace viable la adaptación de modelos de varios miles de millones de parámetros en una única GPU de consumo. QLoRA, introducida por Dettmers et al., combina esta técnica con cuantización de 4 bits, permitiendo el ajuste fino de modelos de gran escala en hardware estándar sin degradación apreciable del rendimiento [28]. Ambas técnicas están disponibles a través de la biblioteca PEFT de Hugging Face [29], lo que ha democratizado su uso en la comunidad investigadora y facilita su aplicación directa en el contexto de este trabajo.

2.3.3. Prompting e ingeniería de instrucciones

Frente al ajuste fino, el *prompting* permite aprovechar las capacidades de un modelo preentrenado sin modificar sus pesos, simplemente diseñando las instrucciones que se le proporcionan. Esta estrategia es especialmente útil cuando los datos anotados son escasos o cuando se desea mantener la generalidad del modelo base. El hito que consolidó el prompting como estrategia de primer orden en el dominio clínico fue la publicación de Med-PaLM, el primer modelo que superó el umbral de aprobación del examen de licenciatura médica estadounidense (USMLE) mediante *instruction tuning* y diseño cuidadoso de instrucciones [30]. Este resultado motivó el estudio sistemático de hasta qué punto estrategias similares son efectivas con modelos de escala muy inferior.

Técnicas como el *zero-shot prompting*, el *few-shot prompting* (proporcionar al modelo ejemplos de entrada-salida dentro del propio *prompt*)y el *chain-of-thought* (inducir razonamiento paso a paso) han demostrado mejorar significativamente el rendimiento en tareas de PLN especializado. Agrawal et al. demostraron que, mediante *few-shot prompting* con tan solo 5–10 ejemplos, modelos generativos alcanzan rendimientos competitivos con sistemas especializados entrenados supervisadamente en tareas de extracción de información clínica [22]. En el diseño del sistema propuesto en este trabajo, el prompting constituye la estrategia principal de adaptación: la formulación del *prompt* incluye la definición explícita de la tarea, el formato de salida esperado y, en la variante *few-shot*, ejemplos representativos de pares (abreviatura en contexto, expansión correcta) extraídos del corpus de desarrollo.

2.4. Sostenibilidad en el desarrollo de modelos de IA

El crecimiento exponencial en el tamaño de los modelos de lenguaje ha venido acompañado de un aumento igualmente significativo en su coste energético y su huella de carbono. Strubell et al. estimaron que el entrenamiento de un modelo Transformer con búsqueda de hiperparámetros equivale a la emisión de aproximadamente 284 toneladas de CO_2, cifra comparable a cinco veces las emisiones de vida útil de un automóvil medio [7]. Este trabajo inauguró el debate sobre la sostenibilidad en la investigación en PLN y acuñó el término *Green AI* para referirse al conjunto de prácticas orientadas a reducir el impacto ambiental del desarrollo de sistemas de inteligencia artificial.

Patterson et al. ampliaron este análisis a modelos de mayor escala, documentando que el entrenamiento de GPT-3 generó aproximadamente 502 toneladas de CO_2 equivalente, y que el uso de hardware especializado y energías renovables puede reducir esta cifra en más de un orden de magnitud [31]. Schwartz et al. propusieron el concepto de *eficiencia de la IA* como métrica de referencia, argumentando que el progreso científico no debe medirse únicamente en términos de rendimiento absoluto sino también de coste computacional por punto de mejora [8]. En el contexto hospitalario, donde los datos de carácter especialmente sensible impiden habitualmente el uso de servicios en la nube y exigen el despliegue local del sistema, la apuesta por modelos compactos no es únicamente una decisión de sostenibilidad ambiental, sino también una exigencia práctica derivada de las restricciones de privacidad y de la infraestructura disponible en los centros sanitarios. Este trabajo se inscribe conscientemente en esta línea, priorizando la eficiencia como criterio de diseño al mismo nivel que el rendimiento en la tarea.

2.5. Recursos, corpus y herramientas para el español clínico

La disponibilidad de recursos lingüísticos específicos para el español clínico es un factor determinante para el desarrollo y la evaluación de sistemas de PLN en este dominio. Aunque la situación ha mejorado notablemente en los últimos años, el español sigue estando significativamente menos representado que el inglés en términos de datasets anotados y *benchmarks* de referencia.

2.5.1. Corpus clínicos en español

El corpus de trabajo empleado en este proyecto procede del Hospital Universitario de Jaén (HUJ) y se describe en detalle en la Sección 3.2, donde también se detalla el proceso de anonimización y la estrategia de anotación. A escala europea, el proyecto DataTools4Heart[4] ha generado corpus multilingües (incluyendo español) centrados en informes clínicos del ámbito cardiovascular [32], particularmente relevantes por su coincidencia con el área médica de los documentos aportados por el HUJ para este trabajo.

2.5.2. Recursos léxicos y ontológicos

A nivel internacional, el UMLS (*Unified Medical Language System*) integra más de 200 vocabularios biomédicos y clínicos, incluyendo terminologías como SNOMED CT, MeSH o LOINC. Si bien está orientado principalmente al inglés, su versión multilingüe y la disponibilidad de mapeos a terminologías en español lo convierten en un recurso complementario de interés para la construcción de sistemas de desambiguación basados en conocimiento.

Para el español, el Diccionario de Siglas Médicas de la SEDOM[5] es el recurso lexicográfico de referencia, con más de 15.000 entradas que cubren siglas y acrónimos de uso frecuente en la documentación sanitaria española [19]. Su uso como base de conocimiento para la expansión de candidatos en sistemas de desambiguación es una estrategia natural, si bien sus limitaciones de cobertura frente a abreviaturas de uso local o de servicios muy especializados (ya documentadas en la Sección 2.2.1) hacen necesario complementarlo con otros mecanismos de inferencia contextual. La base de datos de medicamentos de la Agencia Española de Medicamentos y Productos Sanitarios (AEMPS)[6] representa un recurso adicional de especial utilidad para la normalización de abreviaturas referidas a fármacos y principios activos en los informes de alta.

2.6. Trabajos relacionados

La desambiguación de abreviaturas en texto clínico ha sido abordada desde distintas perspectivas en la literatura. A continuación se revisan los trabajos más relevantes, organizados según el enfoque metodológico adoptado.

[4] https://www.datatools4heart.eu
[5] http://diccionario.sedom.es/
[6] https://www.aemps.gob.es/cima/

2.6.1. Sistemas basados en reglas y recursos léxicos

Los sistemas de primera generación para la expansión de abreviaturas clínicas combinaban patrones textuales con consultas a vocabularios controlados. El algoritmo de Schwartz y Hearst, ya mencionado en este capítulo en la Sección 2.2.1, estableció la línea base para la identificación de pares acrónimo-expansión con definición explícita en el texto [16]. Para el escenario (mucho más frecuente en la práctica) en que la expansión no aparece en el documento, Stevenson et al. desarrollaron un sistema híbrido que combina similitud distribucional con información del UMLS, alcanzando precisiones superiores al 90 % en los términos más frecuentes del NLM-WSD benchmark [18]. La principal limitación de estos enfoques es su dependencia estructural de la cobertura del vocabulario controlado de referencia: cualquier abreviatura ausente del recurso lexicográfico resulta inaccesible al sistema, lo que los hace especialmente inadecuados para el registro hospitalario real, donde la variabilidad terminológica supera con creces los límites de cualquier diccionario estático.

2.6.2. Sistemas basados en aprendizaje automático

La formulación supervisada del problema permitió superar parcialmente estas limitaciones. Los clasificadores basados en SVM con representaciones de bolsa de palabras del contexto local demostraron ser una línea base robusta y eficiente en recursos, con precisiones medias superiores al 87 % en benchmarks estándar [33]. La adopción posterior de arquitecturas BiLSTM-CRF mejoró el tratamiento de dependencias de largo alcance en el contexto, aunque a costa de una mayor necesidad de datos de entrenamiento y de la disponibilidad de embeddings específicos del dominio [20]. Ambas familias de sistemas comparten una limitación común: su rendimiento cae significativamente ante abreviaturas infrecuentes o ausentes del corpus de entrenamiento, un problema especialmente pronunciado en el contexto del español clínico donde los corpus anotados disponibles son de tamaño reducido.

2.6.3. Aproximaciones con LLMs

Los modelos de lenguaje de gran escala han comenzado a aplicarse a la desambiguación de texto clínico con resultados que cuestionan la necesidad de arquitecturas especializadas. Kraljevic et al. demostraron que MedCAT, un sistema basado en representaciones contextuales de ClinicalBERT, supera a los enfoques basados en diccionarios en la detección y normalización de entidades clínicas en registros electrónicos del NHS británico [34]. Sin embargo, MedCAT requiere un proceso de entrenamiento

supervisado sobre anotaciones específicas del dominio, lo que limita su aplicación inmediata en hospitales sin recursos de anotación especializados. Los modelos de las familias Qwen, IBM Granite y Google Gemma evaluados en el presente trabajo representan una alternativa que elimina esta restricción al operar en modo *zero-shot* o *few-shot* sin ajuste fino adicional. Estudios recientes han mostrado que modelos de menos de 5B de parámetros, con un diseño de instrucciones adecuado, alcanzan rendimientos comparables a sistemas especializados mucho más costosos en tareas de extracción de información clínica [35], confirmando el potencial de estas arquitecturas compactas cuando el modelado contextual es robusto.

2.6.4. Simplificación de texto médico y posicionamiento del trabajo

Aunque el objetivo principal de este trabajo es la desambiguación de abreviaturas, esta tarea se enmarca en el contexto más amplio de la accesibilidad del texto clínico, que incluye también la simplificación lexical y sintáctica. El proyecto CLARA-Med, desarrollado por el CSIC en colaboración con la UNED y la Universidad Complutense, generó un corpus de frases clínicas con sus versiones simplificadas, constituyendo un referente directo por abordar problemas complementarios sobre el mismo tipo de documentos y con recursos similares [36].

El corpus hospitalario empleado en este trabajo procede de la colaboración del grupo de investigación con el Hospital Universitario de Jaén en el marco de un proyecto más amplio de PLN clínico (véase la Sección 3.2). Frente a los enfoques revisados en las secciones anteriores de este capítulo, el presente trabajo se distingue en tres dimensiones. En primer lugar, desplaza el foco hacia la *eficiencia computacional*, evaluando modelos de menos de 5B de parámetros compatibles con infraestructuras hospitalarias estándar sin GPUs dedicadas de alto rendimiento, frente a los sistemas de mayor escala descritos en [34] o [35]. En segundo lugar, incorpora el tratamiento explícito de *símbolos* (unidades de medida y notación numérica específica del dominio clínico) además de abreviaturas y siglas, un aspecto escasamente cubierto en la literatura revisada. En tercer lugar, adopta una perspectiva comparativa sistemática entre las familias IBM Granite, Qwen y Google Gemma con el objetivo de establecer recomendaciones prácticas en función de los recursos disponibles, sobre un corpus clínico real en español.

2.7. Síntesis y posicionamiento del trabajo

La revisión del estado del arte pone de manifiesto varias conclusiones relevantes para este trabajo. En primer lugar, la desambiguación de abreviaturas y símbolos en texto clínico en español es una tarea aún insuficientemente explorada, con una escasez notable de recursos anotados y de sistemas evaluados sobre corpus reales en este idioma. En segundo lugar, los modelos de lenguaje de propósito general han demostrado su potencial para abordar esta tarea, pero su aplicación en entornos hospitalarios reales requiere un equilibrio explícito entre rendimiento y eficiencia que la mayor parte de los trabajos previos no prioriza. En tercer lugar, las técnicas de prompting y ajuste fino eficiente abren la posibilidad de obtener buenos resultados con modelos compactos y datos limitados.

Este trabajo se posiciona en la intersección de estas tres brechas: evalúa comparativamente distintos modelos de lenguaje de pequeña escala (menos de 5B de parámetros) aplicados a la desambiguación de abreviaturas y símbolos en informes hospitalarios en español, sobre corpus clínicos reales procedentes del HUJ. El enfoque en la eficiencia computacional y la sostenibilidad no es accesorio, sino un criterio de diseño explícito que da coherencia a las decisiones metodológicas adoptadas a lo largo del trabajo y que lo alinea con las recomendaciones de la comunidad científica sobre el desarrollo responsable de sistemas de inteligencia artificial [8].

Capítulo 3

Metodología y diseño experimental

En este capítulo se describe la metodología seguida para diseñar, implementar y evaluar el sistema de desambiguación automática de terminología médica propuesto. Se detallan las decisiones de diseño adoptadas en cada etapa del proceso, desde la construcción y preparación del corpus hasta la estrategia de evaluación, pasando por la ingeniería de prompts, el proceso de ajuste fino mediante LoRA y los retos técnicos encontrados durante el desarrollo.

El diseño experimental del presente trabajo parte de una premisa doble: por un lado, demostrar que los modelos de lenguaje de pequeña escala son capaces de resolver la desambiguación de abreviaturas médicas con niveles de precisión suficientes para el entorno clínico; por otro, establecer un marco comparativo riguroso que permita identificar cuál de las familias y variantes estudiadas ofrece el mejor balance entre rendimiento y coste computacional. Ambas premisas están directamente condicionadas por las restricciones reales del entorno hospitalario, donde la privacidad de los datos clínicos excluye el uso de servicios en la nube y la infraestructura disponible raramente supera las capacidades de una GPU de consumo. Estas restricciones no se plantean como limitaciones anecdóticas, sino como restricciones de diseño explícitas que orientan todas las decisiones metodológicas del trabajo.

3.1. Visión general del proceso

El desarrollo del sistema sigue un diseño experimental comparativo estructurado en cuatro etapas principales que se ilustran en la Figura 3.1:

1. **Preparación del corpus**: obtención, anonimización y anotación de los informes hospitalarios que constituyen la base de datos del proyecto.

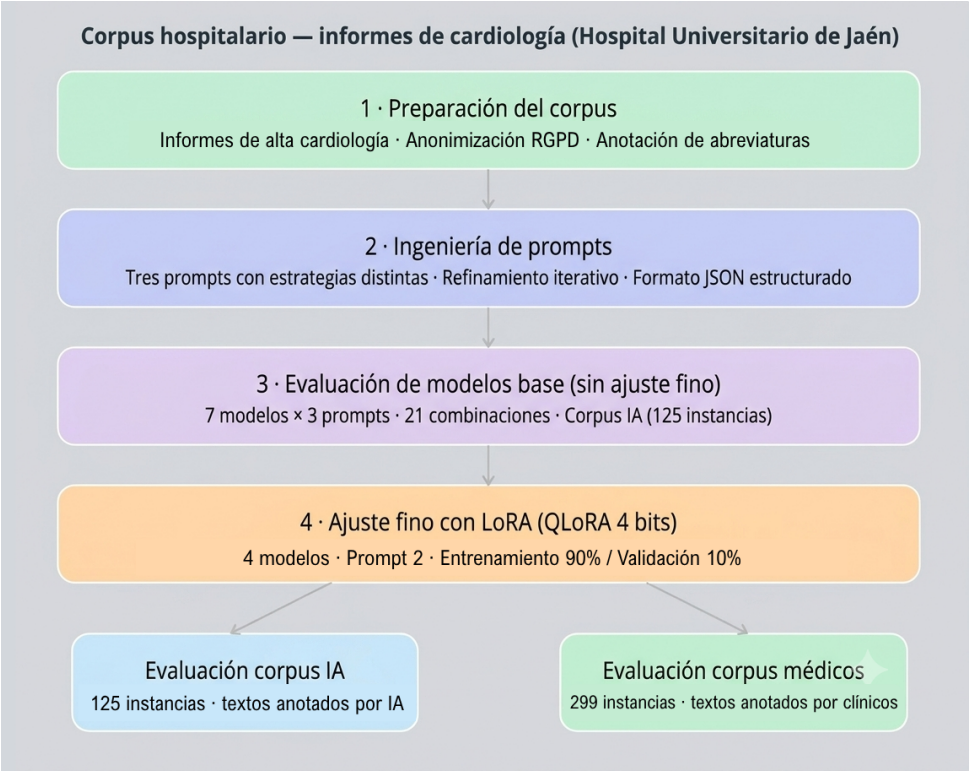


Figura 3.1: Esquema general del proceso de desarrollo y evaluación del sistema de desambiguación.

2. **Diseño de prompts**: formulación y refinamiento iterativo de las instrucciones que guían a los modelos en la tarea de desambiguación.

3. **Evaluación de modelos base**: ejecución y medición del rendimiento de los siete modelos seleccionados sin ajuste fino, variando los prompts diseñados.

4. **Ajuste fino con LoRA y re-evaluación**: aplicación de la técnica Low-Rank Adaptation sobre un subconjunto de modelos y comparación de resultados respecto a la fase anterior.

Adicionalmente, los modelos y los corpus de evaluación se organizan en dos dimensiones cruzadas. En primer lugar, según el **origen de la anotación** del corpus de evaluación: textos anotados por inteligencia artificial (categoría *IA*) y textos anotados por profesionales médicos (categoría *Médicos*). En segundo lugar, según si el modelo ha recibido o no ajuste fino: modelos *base* frente a modelos *fine-tuned*. Esta doble categorización permite analizar el impacto tanto del dominio de evaluación como del proceso de adaptación del modelo.

La elección de una arquitectura comparativa en lugar de un enfoque de sistema único responde a la necesidad de ofrecer recomendaciones prácticas y reproducibles. Dado que los centros hospitalarios con los que eventualmente se desplegaría una herramienta de este tipo disponen de recursos heterogéneos, disponer de una curva de

rendimiento frente a coste computacional resulta más informativo que la identificación de un único sistema óptimo. Este enfoque multivariante exige, en contrapartida, un diseño experimental especialmente cuidadoso para garantizar la comparabilidad de los resultados entre todas las configuraciones evaluadas.

3.2. Corpus y datos

3.2.1. Origen y características del corpus

El corpus de trabajo está compuesto por informes de alta hospitalaria pertenecientes al servicio de cardiología, cedidos por el Hospital Universitario de Jaén en el marco de un proyecto de investigación sobre PLN clínico[1]. Todos los documentos han sido previamente anonimizados, eliminando cualquier dato que pudiera identificar al paciente, al facultativo o al centro, en cumplimiento del Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679)[2] y de la normativa española de protección de datos sanitarios.

Los informes presentan la estructura semi-libre propia de la documentación clínica real: anamnesis, exploración física, pruebas complementarias, diagnóstico, plan terapéutico y recomendaciones al alta. Esta variedad estructural, unida a la alta densidad de abreviaturas y símbolos y a la heterogeneidad redaccional entre facultativos, hace del corpus un banco de pruebas representativo y exigente para el sistema.

Desde el punto de vista lingüístico, la documentación clínica de cardiología presenta características particulares que la diferencian de otros servicios hospitalarios. El vocabulario especializado combina términos de anatomía cardiovascular, procedimientos diagnósticos como ecocardiografías o electrocardiogramas, nomenclatura farmacológica y valores numéricos expresados con notación abreviada, tales como unidades de presión, frecuencias cardíacas o fracciones de eyección. Esta heterogeneidad léxica convierte el corpus en un caso de uso de alta complejidad, donde la ambigüedad de las abreviaturas no solo depende del contexto local de la frase, sino también del tipo de sección del informe en que aparecen.

El proceso de cesión del corpus por parte del Hospital Universitario de Jaén se enmarca en un proyecto de investigación más amplio sobre procesamiento del lenguaje natural aplicado al entorno clínico español. La colaboración con el centro hospitalario ha aportado, además del corpus textual, el acceso a especialistas clínicos del servicio de cardiología que han participado en la fase de anotación supervisada, lo que confie-

[1] https://investigacion.ujaen.es/proyectos/1792396/detalle
[2] https://www.boe.es/doue/2016/119/L00001-00088.pdf

re a los datos de evaluación una validez clínica que los corpus puramente automáticos no pueden garantizar.

3.2.2. Proceso de anonimización

Antes de cualquier operación de anotación o procesamiento, todos los informes fueron sometidos a un proceso de anonimización que eliminó o sustituyó por marcadores genéricos los siguientes tipos de datos: nombre del paciente, fecha de nacimiento, número de historia clínica, número de identificación personal, nombre del médico responsable y del personal de enfermería, denominación del centro hospitalario, fechas de ingreso y alta, y cualquier otro dato que pudiera permitir la identificación directa o indirecta del individuo. Este proceso es conforme al Reglamento General de Protección de Datos y a la Ley Orgánica 3/2018[3] de Protección de Datos Personales y Garantía de los Derechos Digitales.

La anonimización fue llevada a cabo mediante una combinación de sustitución automática basada en expresiones regulares para patrones estructurados (fechas, números de historia, patrones de nombre y apellidos) y revisión manual por parte del equipo de investigación para los casos en que la información identificativa se encontraba embebida en fragmentos de texto libre. Tras el proceso, los documentos resultantes conservan íntegra la información clínica relevante para la desambiguación, incluyendo los términos médicos, los valores de laboratorio y la estructura narrativa del informe, pero carecen de cualquier referencia que permita vincular el documento a una persona física concreta.

3.2.3. Anotación del corpus

Las abreviaturas presentes en los textos fueron identificadas y anotadas en dos fases diferenciadas, dando lugar a los dos subconjuntos de evaluación mencionados:

- **Corpus IA**: los textos fueron procesados con la herramienta de anotación Brat[4] combinada con el modelo Gemini de Google y ChatGPT de OpenAI para proponer anotaciones automáticas. El conjunto resultante abarca tres documentos con un total de 125 instancias de desambiguación, cubriendo abreviaturas del ámbito de la medicina general.
- **Corpus Médicos**: los documentos de test fueron tomados directamente de registros de cardiología del hospital y anotados con la supervisión de especialistas

[3] `https://www.boe.es/eli/es/lo/2018/12/05/3/con`
[4] `https://eines.iula.upf.edu/brat//#/AcronAbrevOnCR/`

clínicos. Este conjunto consta de 299 instancias distribuidas en siete documentos correspondientes a pruebas complementarias, evolución y planes terapéuticos.

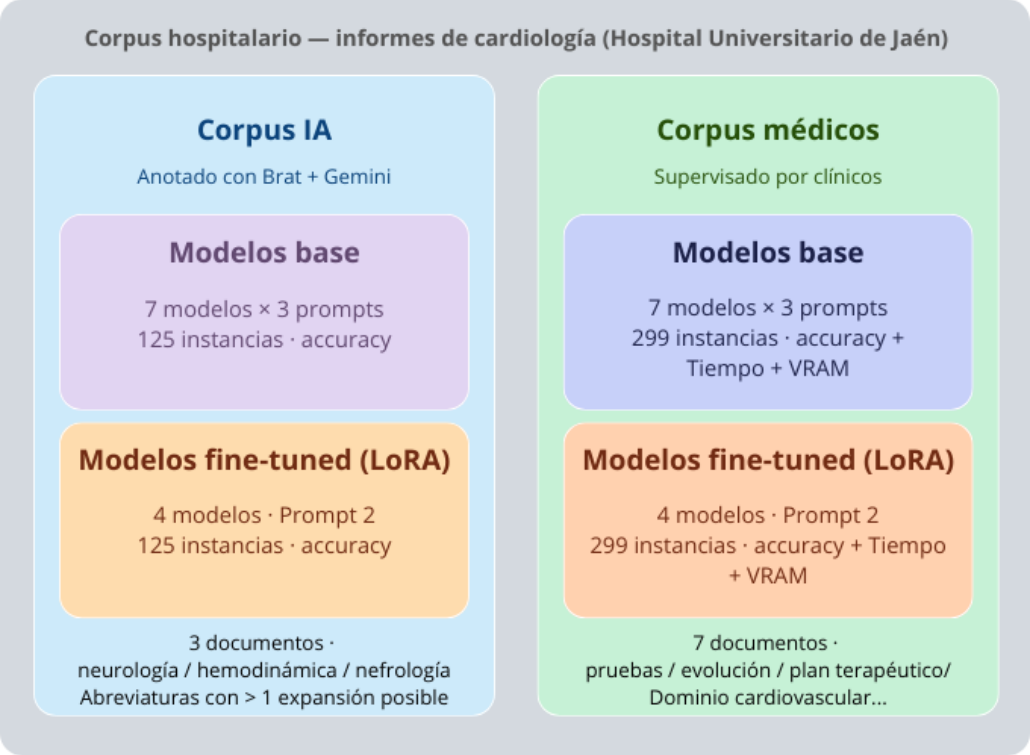


Figura 3.2: Categorización del corpus de evaluación según el origen de la anotación y el tipo de modelo.

La distinción entre los dos subconjuntos no es simplemente operativa, sino que tiene implicaciones metodológicas relevantes para la interpretación de los resultados. El Corpus IA fue anotado de forma semi-automática con la asistencia de modelos de lenguaje de gran escala (Gemini y ChatGPT), lo que implica que las etiquetas de referencia podrían no ser siempre perfectas, pero que el proceso de anotación es reproducible y escalable a un coste reducido. El Corpus Médicos, en cambio, fue anotado bajo la supervisión directa de cardiólogos con experiencia clínica real, lo que confiere a las etiquetas de referencia una autoridad clínica que las hace más fiables como estándar de evaluación, aunque también más costosas de obtener.

Esta diferencia en el rigor metodológico de los dos conjuntos de referencia hace que la evaluación sobre el Corpus Médicos sea la más relevante desde el punto de vista de la aplicabilidad clínica del sistema, mientras que la evaluación sobre el Corpus IA sirve como referencia de rendimiento en un escenario de datos abundantes y anotación automatizada.

Para cada instancia de desambiguación se registra: el identificador del documento (`doc_id`), la abreviatura objetivo, la frase en la que aparece, el contexto clínico comple-

to del documento, el significado correcto y el conjunto de posibles expansiones candidatas. Este formato de múltiple elección permite evaluar la tarea como un problema de clasificación supervisada con etiqueta de clase conocida.

3.2.3.1. Herramienta de anotación Brat

La plataforma Brat (*Brat Rapid Annotation Tool*) es un entorno de anotación de texto basado en web, ampliamente utilizado en lingüística computacional y procesamiento del lenguaje natural biomédico. En el marco de este trabajo, la plataforma no se empleó de forma activa ni se integró dentro del desarrollo del proyecto, sino que sirvió exclusivamente como una herramienta externa de origen para la obtención de datos de referencia. En concreto, se utilizó para acceder a un corpus de informes clínicos generales (lo que garantiza que el alcance del proyecto no se limite únicamente al ámbito de la cardiología) que ya contenían las abreviaturas detectadas y delimitadas, pero cuyas acepciones aún no habían sido desambiguadas.

A partir de estos textos extraídos, se procedió a la preparación y estructuración de los datos en formato JSON para que pudieran ser procesados por los modelos de lenguaje. De este modo, este recurso externo sirvió como pilar fundamental para conformar tanto los conjuntos de datos destinados a la fase de evaluación del pipeline como los ejemplos estructurados para el proceso de ajuste fino (*fine-tuning*) de los modelos. Por lo tanto, el papel de Brat quedó estrictamente limitado a una fase previa de recopilación de material lingüístico pre-anotado, quedando completamente al margen de la arquitectura, la lógica de ejecución y el desarrollo propio del sistema.

3.2.4. Detección de abreviaturas candidatas

La detección de abreviaturas en los textos sin procesar se realizó mediante el módulo de anotación desarrollado, el cual devuelve las posiciones de inicio y fin de cada abreviatura en el texto (`char_begin`, `char_end`) junto con su forma textual (`text`). Sin embargo, este módulo original no incorporaba de forma nativa el campo con las múltiples opciones de expansión (`candidates`) extraídas del Diccionario de Siglas Médicas.

Para llevar a cabo este reconocimiento sobre el texto plano del informe, en las fases iniciales del desarrollo se implementó una aproximación basada en expresiones regulares orientada a identificar candidatos potenciales y compararlos directamente con las entradas del diccionario. Posteriormente, este enfoque inicial evolucionó hacia la integración de un modelo específico de reconocimiento de entidades para automatizar la detección de las abreviaturas dentro de su contexto. En cualquiera de los enfoques,

el módulo aplica un criterio basado en la cobertura del diccionario: para cada forma textual candidata (según criterios morfológicos como la presencia de mayúsculas, la brevedad de la secuencia o la coincidencia con patrones de acrónimos médicos conocidos), se consulta el diccionario de abreviaciones y se recupera el conjunto de expansiones candidatas. Las abreviaturas con una única expansión posible se consideran no ambiguas y, aunque son procesadas por el sistema, se excluyen de la evaluación cuantitativa, que se centra exclusivamente en las instancias con más de un candidato.

La transición de las expresiones regulares hacia el modelo automatizado estuvo motivada por las limitaciones técnicas del primer enfoque. Inicialmente, las reglas rígidas de las expresiones regulares pasaban por alto ciertas abreviaturas válidas y, de manera inversa, tendían a marcar erróneamente elementos sin sentido clínico, tales como espacios en blanco aislados o caracteres de puntuación como paréntesis. A pesar de que se introdujeron optimizaciones y mejoras iterativas para refinar el patrón y contemplar la alternancia entre mayúsculas y minúsculas, se constató que la aproximación más robusta, aunque no completamente infalible, consistía en delegar esta tarea en un modelo especializado. Este modelo se encarga de extraer una lista limpia con las abreviaturas detectadas contextualmente, delegando posteriormente al prototipo de herramienta de anotación la lógica computacional para posicionar con precisión los índices `char_begin` y `char_end` sobre el texto original.

Durante la fase de preparación de datos, esta carencia del campo de expansiones generó un escenario dispar entre los conjuntos de datos. Para los textos anotados por IA, las múltiples opciones de la palabra que habían sido descartadas se añadieron a posteriori mediante la ejecución de un script específico diseñado para enriquecer los registros. No obstante, los textos médicos anotados directamente por los facultativos mantuvieron la estructura original del módulo de anotación, donde el experto asignaba la acepción correcta sin dejar rastro del resto de alternativas en el archivo final. La estructura de estos registros médicos se presenta a continuación:

```
{
  "char_begin": 885,
  "char_end": 889,
  "text": "IVUS",
  "meaning": "Intravascular ultrasonography (Ultrasonografía...)"
}
```

Dado que los modelos de lenguaje requieren recibir en el *prompt* todas las opciones de expansión posibles para poder desambiguar y elegir, esta asimetría obligó a modificar la función de sustitución e inyección de *prompts* en las evaluaciones de los

textos médicos. Se implementó un mecanismo de contingencia (*fallback*) en el código de evaluación que, al detectar la ausencia del campo `candidates` en los informes médicos, consulta dinámicamente el diccionario global para reconstruir la lista de opciones en tiempo de ejecución. De este modo, el formato normalizado final consumido por los modelos adoptó la siguiente estructura:

```
{
  "char_begin": 1683,
  "char_end": 1686,
  "text": "PAD",
  "meaning": "Presión arterial diastólica.",
  "candidates": [
    "Presión arterial diastólica.",
    "Planificación anticipada de decisiones.",
    "Prueba de antiglobulina directa."
  ]
}
```

Este proceso de adaptación y normalización dinámica resolvió el sesgo técnico entre los corpus, garantizando la homogeneidad y la coherencia del flujo de datos a lo largo de toda la batería de experimentos con independencia del origen del conjunto de datos.

3.2.5. Análisis exploratorio del corpus

Antes de proceder a la fase de evaluación, se realizó un análisis exploratorio del corpus con el objetivo de caracterizar la distribución de las instancias de desambiguación y detectar posibles sesgos o desequilibrios que pudieran condicionar la interpretación de los resultados.

El análisis reveló una distribución no uniforme de las abreviaturas: un subconjunto reducido de formas abreviadas (como `Hg`, `mm`, `NRL` o `ºC`) concentra una proporción elevada de las instancias, mientras que una larga cola de abreviaturas de baja frecuencia aparece una o dos veces en todo el corpus como se muestra en la Figura 3.3. Esta distribución es característica de los corpus de texto clínico real y tiene implicaciones directas para el diseño de métricas de evaluación: las métricas macro (que tratan cada abreviatura por igual independientemente de su frecuencia) capturan mejor el comportamiento del sistema en la cola de baja frecuencia, mientras que las métricas micro (ponderadas por frecuencia) reflejan más fielmente el rendimiento agregado en condiciones de uso real.

En cuanto a la dificultad semántica de las instancias, el número de expansiones candidatas por abreviatura oscila entre dos y ocho, con una media de aproximadamente tres opciones por instancia. Las abreviaturas con mayor número de candidatos son precisamente las que presentan mayor ambigüedad clínica y, en consecuencia, mayor impacto potencial en la seguridad asistencial en caso de desambiguación incorrecta.

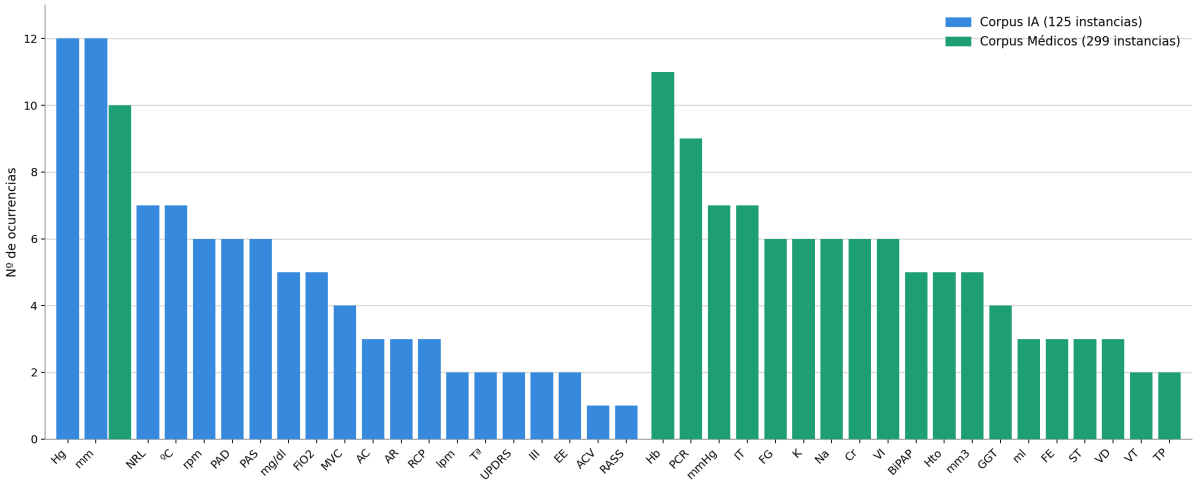


Figura 3.3: Distribución de frecuencia real de las abreviaturas más comunes en el Corpus IA (azul, 125 instancias) y el Corpus Médicos (verde, 299 instancias), contabilizada una única vez por instancia anotada. Un subconjunto reducido de formas abreviadas (`Hg`, `mm`, `NRL`, `Hb`, `PCR`) concentra hasta 12 apariciones, mientras que la mayoría de las abreviaturas del corpus aparece una o dos veces, configurando una larga cola característica de los corpus de texto clínico real. Este desequilibrio motiva el uso de métricas macro-promediadas para evaluar el comportamiento del sistema en términos de igualdad entre clases, ya que una métrica ponderada por frecuencia (micro) diluiría el rendimiento del sistema sobre esta cola de baja frecuencia.

3.3. Selección y configuración de modelos

3.3.1. Criterios de selección

Los modelos evaluados fueron seleccionados atendiendo al criterio de **eficiencia computacional** como restricción de diseño explícita. El contexto hospitalario impone dos limitaciones prácticas fundamentales: la privacidad de los datos clínicos impide el uso de servicios externos en la nube, y la infraestructura disponible en los centros sanitarios raramente incluye GPUs de alto rendimiento. Por este motivo, el espacio de experimentación se limita a modelos de **menos de 5B de parámetros**, pertenecientes a tres familias de vanguardia: IBM Granite, Qwen y Google Gemma (véase la Tabla 2.1 del Capítulo 2 para una comparativa detallada de sus especificaciones técnicas).

Las variantes `granite-4.0-h-350M` y `granite-4.0-h-1B` disponen de una ventana de contexto de 4.096 tokens, frente a los 32.768 tokens de las familias Qwen y Gemma. Dado que el prompt completo incluye el texto íntegro del informe de alta (que puede superar los 2.000-3.000 tokens en documentos extensos), existe la posibilidad de que algunos informes sean truncados al procesarse con los modelos Granite. Este efecto de truncamiento puede influir negativamente en el rendimiento de estas arquitecturas, actuando como variable de confusión en la comparativa directa con Qwen y Gemma. Los resultados obtenidos por los modelos Granite deben interpretarse teniendo en cuenta esta restricción estructural adicional.

Adicionalmente, se establecieron los siguientes criterios secundarios de selección:

- **Licencia open-source o de uso académico libre**: que permita la ejecución local del modelo sin necesidad de enviar datos a servidores externos. Este requisito es especialmente crítico en el entorno hospitalario por las restricciones de confidencialidad de los datos clínicos.
- **Soporte nativo para el idioma español**: dado que todos los informes del corpus están redactados en español, se priorizaron modelos con capacidades multilingües verificadas o con soporte explícito para este idioma.
- **Disponibilidad en Hugging Face**: para garantizar la reproducibilidad del trabajo y la facilidad de acceso a pesos y documentación técnica.
- **Diversidad arquitectónica**: la selección abarca tres familias con filosofías de diseño distintas (IBM Granite, Qwen y Google Gemma), lo que permite extraer conclusiones más generales sobre el tipo de arquitectura mejor adaptada a la tarea.
- **Representación de distintas escalas de parámetros**: se incluyeron modelos desde 350M hasta 4B de parámetros para poder analizar el efecto del tamaño en el rendimiento de la tarea.

3.3.2. Descripción de las familias de modelos evaluados

Las tres familias de modelos evaluadas; IBM Granite, Qwen y Google Gemma, han sido descritas en detalle en la Sección 2 (véase la Tabla 2.1 para sus especificaciones técnicas comparadas). En esta sección se recogen únicamente los criterios que motivaron la selección de cada variante específica dentro de dichas familias para el diseño experimental.

La familia IBM Granite se incluyó como referencia de rendimiento en el límite inferior de escala paramétrica y para evaluar si su énfasis en la transparencia del corpus de entrenamiento se traduce en ventajas en el dominio clínico. La familia Qwen fue seleccionada como hipótesis principal por su demostrado soporte nativo para el español y su competitividad en tareas multilingües de comprensión. La familia Gemma se incorporó para disponer de una referencia con ajuste explícito de instrucciones (*-it*), evaluando si el entrenamiento orientado al seguimiento de prompts mejora la adherencia al formato JSON requerido.

3.3.3. Infraestructura hardware

Los experimentos se ejecutaron sobre un equipo equipado con CPU Intel Core i7-14700KF y 32 GB de RAM, accediendo a una GPU NVIDIA GeForce RTX 5070 OC de 12 GB para la fase de evaluación. Todos los modelos se cargaron en memoria de GPU empleando el formato `bfloat16` para reducir el consumo de memoria (VRAM) y hacer viable la ejecución de los modelos dentro de las restricciones de hardware disponibles.

La elección del formato `bfloat16` (número en coma flotante de 16 bits con campo de exponente de 8 bits) sobre el alternativo `float16` responde a su mayor estabilidad numérica durante la inferencia, con una reducción del consumo de VRAM de aproximadamente el 50 % respecto al formato estándar `float32`, sin penalización apreciable en la calidad de las respuestas generadas.

La GPU NVIDIA GeForce RTX 5070 OC dispone de 12 GB de VRAM, lo que impone un límite superior efectivo al tamaño de los modelos que pueden cargarse directamente. Con el formato `bfloat16`, los modelos de 4B de parámetros requieren aproximadamente 8 GB de VRAM, dejando margen suficiente para los activaciones intermedias durante la inferencia. Los modelos de 1B y 0.8B consumen aproximadamente 2 GB, lo que permite operar con margen amplio incluso con *batch sizes* superiores a 1.

3.3.4. Entorno software

Se empleó el entorno Ubuntu 24.04.3 LTS sobre WSL (Windows Subsystem for Linux) bajo el sistema operativo Windows 11. Sobre esta base operativa, el desarrollo del sistema se ha articulado por completo utilizando el lenguaje de programación **Python 3**, aprovechando su madurez y la robustez de su ecosistema para el procesamiento del lenguaje natural y el aprendizaje profundo. Las dependencias del proyecto se han gestionado de forma aislada y se dividen en los siguientes bloques funcionales:

- **Arquitectura de Aprendizaje Profundo e IA:**

- `torch (v2.9.1+cu130)`: Se empleó la biblioteca *PyTorch* optimizada para entornos CUDA 13, actuando como el motor de ejecución principal para la carga de tensores y el aprovechamiento de la aceleración por hardware de la GPU NVIDIA RTX 5070 OC.
- `transformers (v5.5.4)` y `tokenizers (v0.22.1)`: Herramientas clave del ecosistema de *Hugging Face* para la instanciación de la arquitectura del modelo de lenguaje (*Qwen 3.5*), la gestión del ciclo de vida de los pesos y la tokenización optimizada del texto clínico plano.
- `accelerate (v1.13.0)`: Utilizada para abstraer y optimizar la asignación de memoria del modelo sobre la GPU (`device_map="auto"`).

- **Optimización y Ajuste Fino Eficiente (PEFT/QLoRA):**
  - `bitsandbytes (v0.49.2)`: Librería fundamental que da soporte a los tipos de datos de precisión reducida (como *NormalFloat4*), permitiendo realizar la cuantización de 4 bits del modelo base en la VRAM de la GPU.
  - `peft (v0.19.1)`: Siglas de *Parameter-Efficient Fine-Tuning*, encargada de inyectar las matrices de bajo rango aditivas (*LoRA*) sobre los módulos de atención del modelo sin necesidad de descongelar los pesos originales.
  - `trl (v1.2.0)`: Biblioteca *Transformer Reinforcement Learning*, empleada específicamente por sus clases `SFTTrainer` y `SFTConfig` para coordinar y supervisar el entrenamiento supervisado (*Supervised Fine-Tuning*) de forma nativa con los parámetros de PEFT.
- **Gestión de Datos y Formatos:**
  - `datasets (v4.8.4)` y `pyarrow (v24.0.0)`: Utilizadas para la serialización y preparación eficiente de los conjuntos de entrenamiento y test estructurados, optimizando las transferencias de memoria hacia el pipeline de entrenamiento.
  - `pandas (v3.0.2)`: Empleada para la manipulación analítica de datos, estructuración de tablas auxiliares y la lectura del diccionario de referencia de abreviaciones.
- **Infraestructura del Prototipo Visual:**
  - `Flask (v3.1.2)` y `flask-cors (v6.0.1)`: Se utilizó este *microframework* para desplegar el servidor backend local que da vida al prototipo de visualización de desambiguación. `flask-cors` garantiza la comunicación segura e interactiva entre la interfaz de usuario y el pipeline de inferencia técnica

de los modelos, permitiendo resolver las peticiones en tiempo real de forma aislada.

Esta arquitectura híbrida garantizó la reproducibilidad exacta de los experimentos, el aislamiento estricto de las dependencias mediante entornos virtuales y una total estabilidad en las comunicaciones con los controladores de CUDA del sistema anfitrión.

3.4. Ingeniería de prompts

3.4.1. Motivación y proceso de diseño

El diseño de los prompts constituye uno de los elementos más críticos y costosos en términos de iteración del presente trabajo. Dado que los modelos operan en modo *zero-shot* o *few-shot* sin modificación de pesos en la fase base, la calidad de la instrucción determina en gran medida la calidad de la respuesta.

La ingeniería de prompts en el contexto de la desambiguación de abreviaturas médicas presenta desafíos que van más allá del simple enunciado de la tarea. El modelo debe ser capaz de:

I. Entender que su rol es el de un sistema de clasificación con un conjunto finito y explícito de opciones.

II. Aprovechar el contexto clínico completo del documento para inferir el dominio semántico relevante.

III. Analizar la frase local en la que aparece la abreviatura con suficiente resolución semántica.

IV. Generar su respuesta en un formato estructurado estrictamente definido sin añadir texto adicional ni razonamientos intermedios.

El proceso de diseño fue eminentemente empírico: se partió de instrucciones sencillas y se refinaron de forma iterativa a medida que se analizaban los fallos de los modelos. El principal reto fue conseguir que los modelos respondieran **exclusivamente en formato JSON** con la clave `eleccion` y el valor correspondiente a la expansión seleccionada, sin texto adicional, sin plantillas vacías ni puntos suspensivos. Este requisito de formato estructurado fue especialmente difícil de satisfacer con los modelos de menor escala, que tendían a ignorar las restricciones de formato o a generar respuestas malformadas que impedían la evaluación automática.

Para abordar este problema fue necesario implementar un módulo de postprocesamiento robusto en el script de evaluación, capaz de extraer la respuesta válida incluso en presencia de ruido en la salida del modelo (texto adicional, comillas inconsistentes, uso de mayúsculas/minúsculas dispares, etc.).

3.4.2. Estructura de entrada de los prompts

Todos los prompts comparten la misma estructura de entrada, compuesta por cuatro elementos:

1. **Contexto clínico completo del documento**: el texto íntegro del informe de alta, que proporciona al modelo información sobre el área clínica predominante, los antecedentes del paciente, los diagnósticos establecidos y el plan terapéutico. Este contexto amplio es esencial para la resolución de abreviaturas cuyo significado correcto depende de la especialidad o del estado clínico global del paciente.
2. **Frase local**: la oración concreta en la que aparece la abreviatura a desambiguar, que proporciona el contexto sintáctico inmediato y las palabras adyacentes que pueden orientar la elección del significado.
3. **Abreviatura objetivo**: la forma abreviada exacta que se desea desambiguar, en la grafía utilizada en el documento original.
4. **Conjunto de expansiones candidatas**: la lista enumerada de posibles significados de la abreviatura, extraída del Diccionario de Siglas Médicas, de entre los cuales el modelo debe seleccionar exclusivamente uno.

Esta estructura garantiza que todos los prompts sean comparables entre sí en términos de información disponible para el modelo, lo que permite atribuir las diferencias de rendimiento observadas a las estrategias de razonamiento sugeridas, y no a variaciones en la cantidad de información proporcionada.

3.4.3. Descripción de los tres prompts

Se diseñaron tres prompts con enfoques distintos para la tarea de desambiguación. Los tres prompts se diferencian en el **rol asignado al modelo**, en la **estrategia de razonamiento** sugerida y en el nivel de explicitación del proceso de selección:

- **Prompt 1 — Análisis de contexto médico**: Se asigna al modelo el rol de médico especialista con alta capacidad de análisis lingüístico. La instrucción solicita analizar el área médica del contexto completo, evaluar cada opción dentro de la frase clave y elegir la que tenga sentido clínico en ese escenario. Este enfoque pone el énfasis en la comprensión global del documento antes de proceder al análisis local de la frase.
- **Prompt 2 — Razonamiento por eliminación**: Se asigna el rol de experto en terminología clínica y desambiguación de siglas. La estrategia consiste en *sustituir mentalmente* la abreviatura por cada opción candidata en la frase y descartar las que no tengan sentido clínico tras la sustitución. Este enfoque de razonamiento por eliminación induce en el modelo un proceso de verificación sistemática de cada candidato, lo que en la práctica reduce los errores por selección arbitraria entre opciones plausibles. Este enfoque demostró ser el más efectivo en los experimentos.
- **Prompt 3 — Consistencia documental**: Se asigna el rol de médico auditor con experiencia en normalización de historias clínicas. La instrucción pide analizar el contexto completo buscando otras siglas o términos relacionados y elegir la opción más coherente con el conjunto del relato clínico, priorizando la consistencia documental sobre el análisis local de la frase. Este enfoque es especialmente apropiado en aquellos casos en los que la frase local no ofrece suficiente información discriminativa y es necesario recurrir a señales de contexto global.

El texto completo de los tres prompts se recoge en el Apéndice A.

3.4.4. Motivación del diseño de tres prompts

La decisión de diseñar tres prompts diferenciados, en lugar de un único prompt optimizado, responde a varios objetivos complementarios. En primer lugar, permite analizar qué estrategia de razonamiento es más efectiva para la tarea específica de desambiguación de abreviaturas médicas en español, contribuyendo al conocimiento sobre ingeniería de prompts en el dominio clínico. En segundo lugar, permite verificar si la estrategia óptima es consistente entre modelos de distinta familia y escala, o si existen interacciones entre la arquitectura del modelo y el tipo de razonamiento sugerido. En tercer lugar, proporciona información valiosa sobre la robustez de los modelos frente a variaciones en el enunciado de la tarea: un modelo que rinde bien con los tres prompts es más confiable en un entorno de producción que uno cuyo rendimiento es muy sensible a la formulación exacta de la instrucción.

3.4.5. Formato de salida y postprocesamiento

Todos los prompts instruyen al modelo para que genere su respuesta exclusivamente como un objeto JSON con la siguiente estructura:

```
{"eleccion": "<expansión seleccionada>"}
```

donde `<expansión seleccionada>` debe coincidir exactamente, en texto y capitalización, con una de las opciones del conjunto de candidatos proporcionado.

Este requisito de formato estructurado plantea un reto significativo para los modelos de menor escala, que con frecuencia generan respuestas que incluyen texto introductorio antes del JSON, razonamientos intermedios intercalados con el resultado, o variaciones de capitalización que impiden la comparación automática con la etiqueta de referencia. Para manejar estos casos, se implementó un módulo de normalización de salidas con los siguientes pasos:

1. **Extracción del bloque JSON**: búsqueda del primer objeto JSON válido en la respuesta mediante expresiones regulares, descartando cualquier texto anterior o posterior.
2. **Normalización de comillas**: conversión de comillas tipográficas a comillas ASCII estándar para garantizar la parseabilidad del JSON.
3. **Normalización de capitalización**: comparación de la expansión seleccionada con las opciones candidatas en modo insensible a mayúsculas y minúsculas, con asignación al candidato más similar mediante distancia de edición en caso de coincidencia parcial.
4. **Registro de fallos**: los casos en que ninguna de las estrategias anteriores permite recuperar una respuesta válida se registran como instancias fallidas y se excluyen del cálculo de precisión, con anotación explícita en el fichero de resultados.

3.4.6. Protocolo de evaluación de prompts

Para comparar los prompts de forma sistemática, se diseñó un script de evaluación genérico parametrizable mediante un fichero de configuración que especifica el modelo y el prompt a evaluar, o bien una lista de combinaciones para lanzar la evaluación completa en una única ejecución. Por cada combinación modelo-prompt se generó un fichero CSV con las columnas: `doc_id`, `abreviatura`, `acertado` (0 o 1), significado

correcto y significado elegido por el modelo. Estos ficheros constituyen la base de los análisis de rendimiento presentados en el Capítulo 4.

El script de evaluación implementa un flujo de procesamiento por lotes que itera sobre las instancias del corpus en el orden en que aparecen en los documentos, construye el prompt correspondiente para cada instancia, invoca al modelo con la configuración especificada y almacena la respuesta junto con la evaluación de su corrección en el fichero CSV. La ejecución puede interrumpirse y reanudarse sin perder los resultados parciales ya calculados, lo que resultó especialmente útil dado los tiempos de inferencia elevados de algunos modelos.

3.5. Ajuste fino con LoRA

3.5.1. Motivación del ajuste fino

Los resultados de la fase de evaluación de modelos base evidenciaron que, si bien los modelos de mayor escala alcanzaban precisiones notables con el **Prompt 2**, los modelos más pequeños presentaban un rendimiento insuficiente para su uso en un entorno clínico real. Para mejorar el rendimiento de estas arquitecturas compactas sin renunciar a su ventaja en eficiencia computacional, se optó por aplicar la técnica de ajuste fino eficiente LoRA (*Low-Rank Adaptation*).

La motivación específica para emplear LoRA en lugar de un ajuste fino completo del modelo responde a tres consideraciones. En primer lugar, el ajuste fino completo de modelos de 1B o 4B de parámetros requeriría muchos más recursos de cómputo y memoria de los disponibles. En segundo lugar, el corpus de entrenamiento del que se dispone es considerablemente reducido para un ajuste completo, lo que aumentaría drásticamente el riesgo de sobreajuste (*overfitting*); de hecho, el conjunto de validación del Corpus IA (distribuido en tres archivos independientes) consta de tan solo 125 instancias, lo que representa el 10 % mencionado del total de los datos. En tercer lugar, LoRA permite preservar el conocimiento general del modelo base, adaptando únicamente las capas de atención al dominio específico mediante un número reducido de parámetros entrenables.

3.5.2. Fundamentos de LoRA y QLoRA

Los principios teóricos de LoRA y QLoRA, incluyendo su motivación y ventajas frente al ajuste fino completo, se describen en la Sección 2.3.2[27, 28]. Formalmente,

dado el peso original $W \in \mathbb{R}^{d \times k}$, LoRA aproxima la actualización ΔW como el producto de dos matrices de bajo rango:

$$\Delta W = BA$$

Donde $B \in \mathbb{R}^{d \times r}$ y $A \in \mathbb{R}^{r \times k}$, con $r \ll \min(d, k)$ siendo el rango de la adaptación. Durante el entrenamiento, los pesos originales W permanecen congelados y únicamente se actualizan A y B. En el contexto de este trabajo, se empleó la variante QLoRA, que combina esta técnica con cuantización de 4 bits en formato NF4, permitiendo el ajuste fino dentro de las restricciones de memoria de la GPU disponible.

3.5.3. Configuración del entrenamiento

El ajuste fino se realizó sobre cuatro de los siete modelos evaluados en la fase base: `Qwen3.5-0.8B`, `Qwen3.5-2B`, `Qwen3.5-4B` y `gemma-3-1b-it`. La selección priorizó la familia Qwen por su demostrado rendimiento durante las anteriores pruebas y la variante más pequeña de Gemma como representante de otra familia arquitectónica.

El proceso de entrenamiento empleó la biblioteca PEFT de Hugging Face con las siguientes decisiones de configuración:

- **Técnica**: QLoRA con cuantización de 4 bits (NF4), lo que permitió realizar el ajuste fino dentro de las restricciones de memoria de la GPU disponible.
- **Rango LoRA (r) y factor de escala (α)**: se utilizó un rango $r = 16$, equilibrando capacidad de adaptación y riesgo de sobreajuste dado el tamaño reducido del corpus (125 instancias). El factor de escala se fijó en $\alpha = 32$, resultando en un escalado $\frac{\alpha}{r} = 2$ conforme a la recomendación de Hu et al. [27].
- **Módulos objetivo**: las matrices de proyección de consulta (*query*), clave (*key*), valor (*value*) y salida (*output*) de las capas de atención, junto con las proyecciones de la red *feed-forward* (`gate_proj`, `up_proj`, `down_proj`).
- **Datos de entrenamiento**: procedentes del Corpus IA anotado, divididos en un 90 % para entrenamiento y un 10 % para validación interna (112 y 13 instancias respectivamente). Cada instancia sigue el formato del Prompt 2, que demostró el mejor rendimiento en la fase base.
- **Formato de salida supervisado**: el modelo aprende a generar exclusivamente el objeto JSON con la clave `eleccion`, reduciendo los fallos de formato observados en la fase de prompting puro.

- **Número de épocas**: 3 épocas completas sin parada anticipada (*early stopping*), justificado por el tamaño reducido del conjunto de entrenamiento.
- **Optimizador y tasa de aprendizaje**: se empleó `paged_adamw_32bit` [28] con una tasa de aprendizaje de 2×10^{-4}, planificador *cosine decay* y *warmup* de 50 pasos para estabilizar los gradientes en las primeras iteraciones.
- **Tamaño de lote efectivo**: 16 ejemplos por paso ($\texttt{per_device_train_batch_size} = 4$ con $\texttt{gradient_accumulation_steps} = 4$), garantizando una estimación estable del gradiente dentro de las restricciones de memoria de la cuantización de 4 bits.

3.5.4. Preparación de los datos de entrenamiento

La preparación de los datos para el ajuste fino requirió transformar el formato de anotación del corpus (ficheros JSON con pares abreviatura-expansión en contexto) al formato de conversación esperado por los modelos de instrucción. Para ello, cada instancia de entrenamiento se convirtió en un par de mensaje de usuario y respuesta del asistente con la siguiente estructura:

- **Mensaje de usuario**: el prompt completo (en el formato del Prompt 2) con el contexto clínico, la frase, la abreviatura y las opciones candidatas.
- **Respuesta del asistente**: el objeto JSON con la clave `eleccion` y el significado correcto como valor, sin texto adicional.

Este formato de conversación es el requerido por los modelos ajustados a instrucciones de las familias Qwen y Gemma, que utilizan plantillas de chat específicas (*chat templates*) para separar los turnos de conversación y los roles del sistema, el usuario y el asistente.

3.5.5. Separación de conjuntos de evaluación

Un aspecto relevante del diseño experimental es que los modelos ajustados se evaluaron sobre **dos conjuntos distintos** correspondientes a las categorías descritas en la Sección 3.2:

- **Evaluación sobre corpus IA**: permite comparar directamente el rendimiento de los modelos *fine-tuned* con el de los modelos base evaluados sobre el mismo

conjunto de 125 instancias. Dado que las instancias de entrenamiento proceden también del Corpus IA, esta evaluación mide fundamentalmente la capacidad de ajuste del modelo.

- **Evaluación sobre corpus Médicos**: evalúa la capacidad de generalización de los modelos ajustados a textos anotados por profesionales, con 299 instancias sobre documentos de cardiología con mayor variedad terminológica. Al no haberse utilizado datos de este conjunto durante el entrenamiento, esta evaluación mide la generalización del modelo ajustado a un dominio clínico ligeramente distinto al del corpus de entrenamiento.

Esta doble evaluación es esencial para determinar si las mejoras obtenidas por el ajuste fino son genuinas o si el modelo ha memorizado características particulares del corpus de entrenamiento. La comparación entre los resultados sobre el Corpus IA y el Corpus Médicos permite además cuantificar el grado de sobreajuste (diferencia entre rendimiento en entrenamiento y en generalización) de cada configuración evaluada.

3.6. Métricas de evaluación

Dado el carácter exploratorio y comparativo de este trabajo, y habida cuenta del reducido tamaño de los conjuntos de evaluación, se optó por emplear la *accuracy* como métrica principal de comparación entre modelos, al ser la más directamente interpretable por los especialistas clínicos que participaron en la validación. El cálculo de métricas macro (que asignan el mismo peso a cada abreviatura independientemente de su frecuencia) queda propuesto como extensión metodológica en las líneas de trabajo futuro.

3.6.1. Métrica principal: precisión o accuracy

La métrica principal empleada en todos los experimentos es la **precisión** o *accuracy*: proporción de instancias en las que el significado elegido por el modelo coincide exactamente con el significado correcto anotado. Esta métrica es adecuada para el problema de desambiguación formulado como clasificación multi-clase con una única respuesta correcta por instancia.

$$\text{Accuracy} = \frac{\text{Número de instancias correctamente desambiguadas}}{\text{Número total de instancias evaluadas}} \tag{3.1}$$

La comparación exacta entre la respuesta del modelo y la etiqueta de referencia se realiza tras la normalización de ambas cadenas de texto (eliminación de espacios iniciales y finales, conversión a minúsculas), con el objetivo de evitar fallos de evaluación debidos a diferencias de capitalización o espaciado que no reflejan errores semánticos reales.

3.6.2. Análisis por documento y por abreviatura

Adicionalmente, el análisis de resultados incluye un desglose por `doc_id` para identificar posibles efectos de dominio o de dificultad según el tipo de sección del informe, y un análisis de **tasa de fallo por abreviatura** (número de errores dividido entre el número total de ocurrencias) para identificar qué términos resultan sistemáticamente difíciles para los modelos.

El análisis por documento permite detectar si existen informes o tipos de sección que resultan sistemáticamente más difíciles de procesar para todos los modelos, lo que podría indicar características lingüísticas específicas (mayor densidad de abreviaturas, terminología más especializada, redacción más telegráfica) que representan un reto particular para los sistemas basados en modelos de lenguaje.

El análisis por abreviatura permite identificar los términos que concentran la mayor parte de los errores del sistema, lo que resulta útil tanto para priorizar mejoras futuras como para delimitar el alcance de aplicabilidad clínica del sistema. Las abreviaturas con alta tasa de fallo son candidatas a tratamiento especial mediante reglas adicionales, consulta explícita al diccionario, o recopilación de más datos de entrenamiento específicos.

3.6.3. Métricas de eficiencia computacional

En la fase de ajuste fino se registraron adicionalmente el **tiempo total de inferencia** (en segundos), el **tiempo medio por consulta** y el **pico máximo de VRAM** consumida (en GB), con el objetivo de disponer de una visión completa del balance entre rendimiento y coste computacional para cada configuración.

Estas métricas de eficiencia son especialmente relevantes en el contexto de despliegue clínico, donde el tiempo de respuesta del sistema puede tener implicaciones directas en la práctica asistencial. Un sistema que alcanza alta precisión pero requiere

varios minutos por documento tiene una utilidad práctica muy limitada en un entorno hospitalario, donde los informes se procesan con urgencia y en grandes volúmenes. La combinación de métricas de calidad (accuracy) y métricas de eficiencia (tiempo e inferencia, pico de VRAM) proporciona la información necesaria para tomar decisiones de despliegue informadas en función de los recursos disponibles en cada centro.

3.7. Retos técnicos del proceso

A lo largo del desarrollo se encontraron varios obstáculos técnicos que condicionaron el diseño final del sistema y que conviene documentar para facilitar la reproducibilidad y la extensión futura del trabajo.

3.7.1. Control del formato de salida

El problema más recurrente fue la incapacidad de los modelos de menor escala para seguir de forma consistente la restricción de respuesta en formato JSON. Los modelos tendían a añadir texto explicativo antes o después del diccionario, a usar formatos de comillas no estándar o a devolver plantillas vacías. Fue necesario desarrollar un módulo de normalización de salidas robusto y realizar múltiples iteraciones en la redacción de los prompts para minimizar estos casos.

Este problema no se limitó a los modelos de menor escala. Incluso los modelos de 4B de parámetros presentaban, con una frecuencia reducida pero no despreciable, respuestas vacías. La robustez del módulo de normalización fue por tanto un requisito funcional para la viabilidad del sistema de evaluación en su conjunto.

La iteración sobre el diseño de los prompts para mejorar la adherencia al formato JSON supuso uno de los ciclos de desarrollo más costosos en términos de tiempo. Cada modificación del prompt requería re-ejecutar la evaluación completa sobre un subconjunto representativo del corpus para verificar si la tasa de respuestas malformadas había mejorado o empeorado. Esta iteración, combinada con los tiempos de inferencia de los modelos, impuso un ciclo de retroalimentación lento que limitó el número de variantes de prompt que pudieron explorarse en el tiempo disponible para el proyecto.

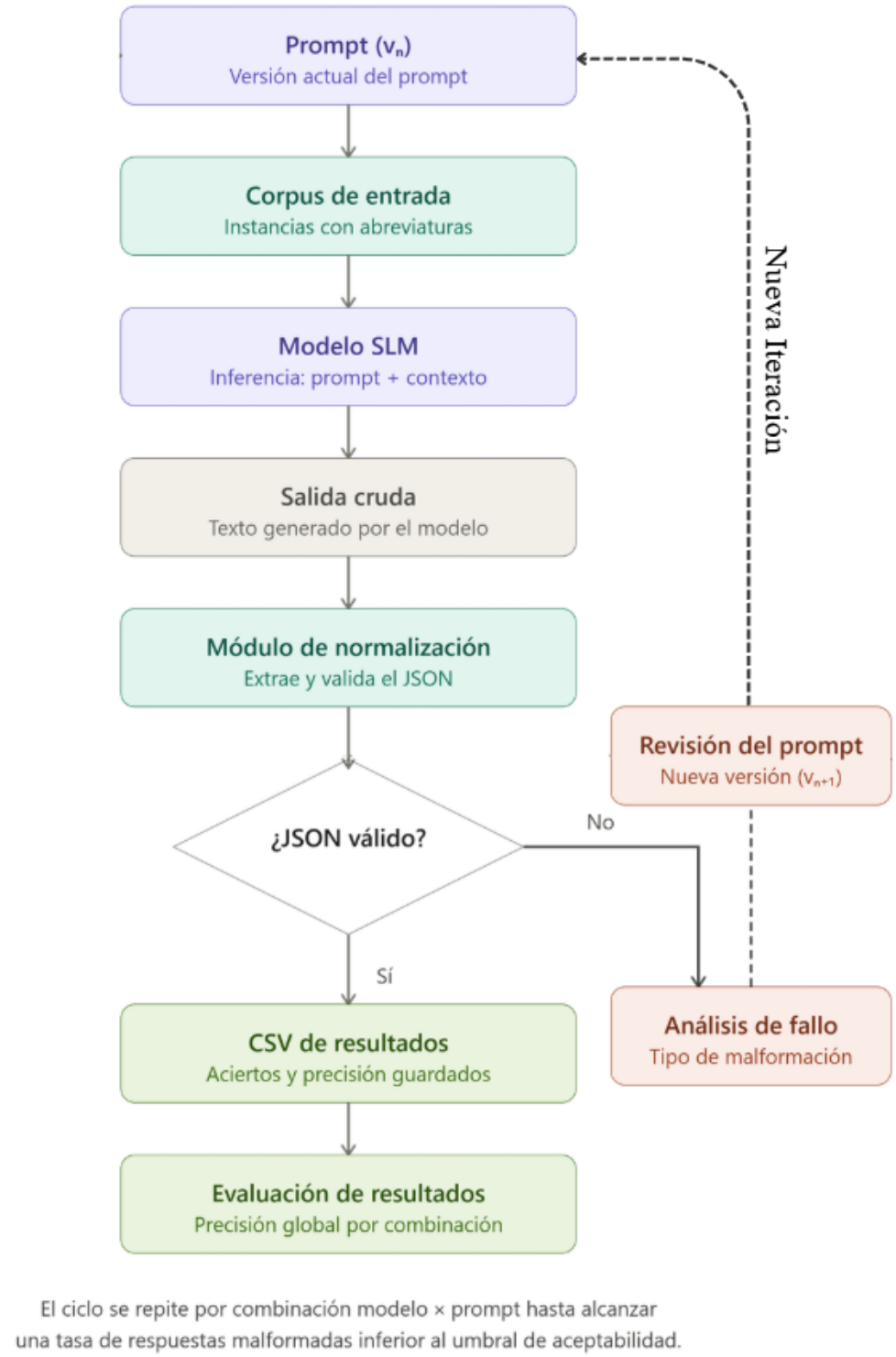


Figura 3.4: Ciclo iterativo de evaluación y mejora de prompts. El flujo parte del prompt (v_n) y el corpus de entrada, pasa por el modelo y el módulo de normalización de salidas, y se bifurca según la validez del JSON resultante: las respuestas válidas se vuelcan en el CSV de resultados para su evaluación; las malformadas desencadenan un análisis de fallo que alimenta la revisión del prompt (v_{n+1}), reiniciando el ciclo. El bucle se repite para cada combinación modelo-prompt hasta obtener una tasa de respuestas malformadas inferior al umbral de aceptabilidad definido.

3.7.2. Ausencia de opciones en ficheros JSON

Como se mencionó en la Sección 3.2, algunos registros del corpus no incluían el campo de opciones de expansión. Este problema se detectó tardíamente durante la fase de evaluación, obligando a revisar el pipeline de datos e implementar la función de verificación y relleno automático de opciones antes de la inferencia.

La raíz del problema residía en el diseño del prototipo de herramienta de anotación utilizada por los facultativos durante la fase de creación del Corpus Médicos: al seleccionar el significado correcto de una abreviatura, la herramienta registraba únicamente la opción elegida y descartaba las alternativas no seleccionadas. El campo de opciones candidatas quedaba así vacío o ausente en los ficheros JSON exportados. La solución implementada consistió en una función de preprocesamiento que, al cargar cada instancia del corpus, verificaba la presencia del campo de opciones y, en caso de ausencia, lo regeneraba consultando el diccionario de la SEDOM con la forma textual de la abreviatura.

3.7.3. Comportamiento anómalo de Granite 1B

El modelo `granite-4.0-h-1B` presentó tiempos de inferencia muy elevados (superiores a media hora para el conjunto de validación completo), lo que limitó la capacidad de iteración sobre este modelo y condicionó su evaluación a un único ciclo de experimentos.

Este comportamiento anómalo no se observó en los demás modelos de escala comparable (como `Qwen3.5-0.8B` o `gemma-3-1b-it`), lo que sugiere que el problema es específico de la implementación o la arquitectura del modelo Granite y no simplemente una consecuencia del tamaño del modelo. Una hipótesis razonable es que la arquitectura hipercompacta de Granite, optimizada para minimizar la memoria, introduce cuellos de botella en las operaciones de atención que no son compensados eficientemente por el hardware evaluado. No obstante, la verificación de esta hipótesis requeriría un análisis de perfilado de la ejecución que excede el alcance de este trabajo.

3.7.4. Resultados perfectos de Qwen 4B base

El modelo `Qwen3.5-4B` obtuvo una precisión del 100 % sobre el Corpus IA con los tres prompts, lo que, si bien es un resultado positivo, impide analizar el margen de

mejora del ajuste fino para este modelo sobre dicho conjunto y desplaza el interés comparativo hacia el Corpus de evaluación médica.

Este resultado plantea además una pregunta metodológica relevante: ¿refleja el 100 % de precisión sobre el Corpus IA una capacidad genuina de desambiguación o podría estar parcialmente inflado por sesgos en el proceso de anotación automática (que utilizó precisamente modelos de la familia Qwen/Gemini)? Esta cuestión refuerza la importancia de disponer del Corpus Médicos como conjunto de evaluación independiente con anotación humana experta.

3.7.5. Evolución iterativa del script de evaluación

El script de evaluación fue objeto de múltiples revisiones a lo largo del proyecto para corregir errores en la lectura de los ficheros CSV de resultados, en la normalización de cadenas para la comparación de respuestas y en el cálculo de estadísticas por abreviatura. Cada revisión se validó comparando los resultados sobre un subconjunto de instancias revisadas manualmente.

Este proceso de revisión iterativa del script fue en parte consecuencia de la naturaleza exploratoria del proyecto: a medida que se incorporaban nuevos modelos y se detectaban nuevos patrones de fallo en las respuestas generadas, el módulo de normalización de salidas debía actualizarse para contemplar los nuevos casos. La documentación exhaustiva de estos cambios en el historial de versiones del repositorio Git garantiza la trazabilidad del proceso y la reproducibilidad de los resultados finales.

3.7.6. Gestión de los tiempos de inferencia y planificación de experimentos

Un aspecto transversal que condicionó el desarrollo del trabajo fue la gestión del tiempo total de experimentación. Con 7 modelos y 3 prompts, la fase base implica 21 ejecuciones de evaluación, cada una de las cuales procesa al menos 125 instancias (Corpus IA) o 299 instancias (Corpus Médicos). Los tiempos de inferencia por instancia varían entre pocos segundos para los modelos de menor escala con los prompts más cortos y varias decenas de segundos para los modelos de 4B con contextos clínicos largos.

Para gestionar este volumen de experimentación, el script de evaluación se diseñó de forma que las 21 combinaciones de la fase base pudieran lanzarse de forma automática y secuencial a partir de un único fichero de configuración, con checkpointing entre combinaciones para recuperar el estado en caso de interrupción. La fase de

ajuste fino (4 modelos) fue gestionada de forma similar, aunque requirió supervisión manual adicional por la variabilidad en los tiempos de entrenamiento.

3.8. Resumen del diseño experimental

La Tabla 3.1 recoge las 21 combinaciones evaluadas en la fase base (7 modelos $\times$ 3 prompts) y las 4 combinaciones adicionales de la fase de ajuste fino, indicando el corpus de evaluación empleado en cada caso.

Modelo	**Prompts**	**Fine-tuning**	**Corpus eval.**	**Instancias**
granite-4.0-h-350M	1, 2, 3	No	IA + Médicos	125 / 299
granite-4.0-h-1B	1, 2, 3	No	IA + Médicos	125 / 299
Qwen3.5-0.8B	1, 2, 3	No	IA + Médicos	125 / 299
Qwen3.5-2B	1, 2, 3	No	IA + Médicos	125 / 299
Qwen3.5-4B	1, 2, 3	No	IA + Médicos	125 / 299
gemma-3-1b-it	1, 2, 3	No	IA + Médicos	125 / 299
gemma-3-4b-it	1, 2, 3	No	IA + Médicos	125 / 299
LoRA Qwen3.5-0.8B	2	Sí	IA + Médicos	125 / 299
LoRA Qwen3.5-2B	2	Sí	IA + Médicos	125 / 299
LoRA Qwen3.5-4B	2	Sí	IA + Médicos	125 / 299
LoRA gemma-3-1b-it	2	Sí	IA + Médicos	125 / 299

Tabla. 3.1: Resumen de las combinaciones modelo-prompt evaluadas. Se indica el corpus de evaluación empleado (IA: textos anotados por inteligencia artificial; Médicos: textos anotados por especialistas clínicos) y si el modelo fue sometido a ajuste fino con LoRA.

En total, el diseño experimental contempla **25 configuraciones** de evaluación distintas, con un volumen agregado de más de 10.000 llamadas de inferencia al considerar los dos corpus de evaluación para cada configuración. Este volumen de experimentación, unido a los retos técnicos descritos en la sección anterior, justifica la envergadura del esfuerzo de ingeniería requerido para obtener resultados fiables y comparables a lo largo de todas las combinaciones evaluadas.

El diseño experimental descrito en este capítulo sienta las bases para el análisis cuantitativo de resultados que se presenta en el Capítulo 4. Las decisiones metodológicas adoptadas, desde la selección de modelos hasta la definición de las métricas, han sido orientadas en todo momento por el objetivo último del trabajo: determinar qué configuración de modelo y prompting ofrece el mejor balance entre precisión clínica y viabilidad de despliegue en el entorno hospitalario real.

Capítulo 4

Análisis de Resultados y Discusión

Este capítulo presenta, analiza y contrasta los resultados obtenidos a lo largo del diseño experimental descrito en el capítulo anterior. El análisis se estructura en cuatro bloques temáticos. En primer lugar, se evalúa el rendimiento de los modelos base sin modificar, comparando las tres estrategias de *prompting* sobre el corpus de evaluación médica. En segundo lugar, se cuantifica el impacto del ajuste fino mediante LoRA sobre las cuatro arquitecturas seleccionadas. En tercer lugar, se analiza el perfil de eficiencia computacional de cada configuración en términos de consumo de VRAM y latencia de inferencia. Por último, se realiza una auditoría sistemática de los patrones de error más frecuentes, identificando las abreviaturas y estructuras lingüísticas que representan el límite actual de los SLMs en el procesamiento del lenguaje clínico en español.

4.1. Evaluación detallada de los modelos base y línea base operativa

La evaluación inicial de las variantes base sin modificar provee un marco de referencia esencial para comprender el punto de partida cognitivo de las arquitecturas generales cuando se enfrentan al léxico hiper-especializado de la práctica clínica. Los modelos evaluados (*Qwen3.5* en sus escalas de 0.8B, 2B y 4B, junto con *Gemma-3-1b-it*) exhiben comportamientos marcadamente heterogéneos que se encuentran condicionados tanto por el volumen de sus parámetros como por la naturaleza del alineamiento por refuerzo recibido durante su entrenamiento generalista.

El rendimiento de estos modelos base se ve fuertemente supeditado a la estrategia de estructuración de la consulta. Para sistematizar esta observación, se tabularon

los resultados de precisión lingüística global sobre el corpus de evaluación médica, segregados por la naturaleza de la instrucción inyectada.

Modelo Base	**Escala**	**Precisión P1** (Análisis médico)	**Precisión P2** (Razon. elimin.)	**Precisión P3** (Chain-of-Thought)
Qwen3.5-0.8B	0.8B	0.520	0.576	0.480
Qwen3.5-2B	2B	0.584	0.672	0.560
Qwen3.5-4B	4B	1.000	1.000	1.000
Gemma-3-1b-it	1B	0.368	0.400	0.240
Gemma-3-4b-it	4B	0.656	0.680	0.632
Granite-4.0-h-1B	1B	0.624	0.504	0.472
Granite-4.0-h-350M	350M	0.224	0.448	0.144

Tabla. 4.1: Matriz comparativa de precisión de los siete modelos base en función del Prompt empleado sobre el **Corpus IA** (125 instancias, anotación semi-automática). El comportamiento de `Qwen3.5-4B` con precisión perfecta en todos los prompts justifica su elección como modelo de referencia superior en la fase de ajuste fino.

Modelo Base	**Escala**	**Precisión P1** (Análisis médico)	**Precisión P2** (Razon. elimin.)	**Precisión P3** (Chain-of-Thought)
Qwen3.5-0.8B	0.8B	0.756	0.816	0.645
Qwen3.5-2B	2B	0.896	0.863	0.839
Qwen3.5-4B	4B	0.930	0.916	0.920
Gemma-3-1b-it	1B	0.361	0.418	0.261
Gemma-3-4b-it	4B	0.910	0.910	0.916
Granite-4.0-h-1B	1B	0.846	0.796	0.783
Granite-4.0-h-350M	350M	0.244	0.341	0.204

Tabla. 4.2: Matriz comparativa de precisión de los siete modelos base en función del prompt empleado sobre el **Corpus Médicos** (299 instancias, anotación supervisada por cardiólogos). Cada valor representa la proporción de abreviaturas correctamente desambiguadas sobre el total de instancias evaluadas.

4.1.1. Comportamiento de la familia Qwen3.5 en estado base

Como se puede deducir de los datos recopilados en la Tabla 4.2, la familia Qwen3.5 demuestra una robustez nativa notable en comparación con otras arquitecturas de tamaño equivalente. Incluso en su configuración más compacta de 0.8B parámetros, el modelo base logra superar la barrera del $75\,\%$ de precisión en inferencia directa (Prompt 1). Este fenómeno se fundamenta en la alta densidad de tokens médicos e informativos presentes en el corpus de pre-entrenamiento de esta familia de modelos, cuya base léxica multilingüe asimila eficazmente las raíces latinas y grecolatinas compartidas por la nomenclatura médica internacional.

Al ascender en la escala de parámetros hacia las variantes de 2B y 4B, se observa un incremento no lineal en la precisión. El salto de 0.8B a 2B representa una ganancia neta de 14 puntos porcentuales en inferencia directa. Este incremento refleja la emergencia de capacidades de comprensión contextual inter-oracional; el modelo de 2B parámetros no solo reconoce la abreviatura de manera aislada, sino que empieza a explotar las dependencias sintácticas colindantes (como los verbos de diagnóstico o los valores numéricos de laboratorio) para acotar el espacio semántico de la sigla. La variante de 4B parámetros consolida esta tendencia, rozando el $93\,\%$ de éxito con el Prompt 1 y manteniéndose por encima del $91\,\%$ con el resto de prompts.

4.1.2. Análisis comparativo: Gemma-3-1b-it frente a modelos compactos

El modelo Gemma-3-1b-it de Google ofrece un contraste metodológico valioso. A pesar de contar con una arquitectura de atención avanzada y un proceso de ajuste de instrucciones muy refinado, su rendimiento base en el entorno de inferencia clínica directa ($36{,}1\,\%$ con el Prompt 1) se sitúa muy por debajo de su contraparte directa por tamaño, el Qwen3.5-0.8B ($75{,}6\,\%$). El examen cualitativo de las respuestas erróneas de Gemma-3 revela una marcada tendencia al sobre-alineamiento normativo. El modelo tiende a generar respuestas discursivas detalladas o a reformular el informe médico en un tono más divulgativo, desobedeciendo la restricción imperativa de retornar única y exclusivamente el término médico expandido.

Este comportamiento evidencia el denominado *sesgo de chat*, donde el optimizador del alineamiento por instrucciones prioriza la elocuencia y la estructura conversacional sobre la precisión extractiva estricta. No obstante, al analizar su evolución ante el Prompt 2, Gemma-3 experimenta una recuperación de casi 6 puntos (del $36{,}1\,\%$ al $41{,}8\,\%$), lo que demuestra que su limitación principal no es la ausencia de conocimiento clínico subyacente en sus pesos, sino una dificultad en el direccionamiento de la atención (*attention steering*) que se mitiga parcialmente cuando el espacio de búsqueda se restringe de forma explícita. Cabe destacar que, pese a partir del rendimiento base más bajo del conjunto evaluado, Gemma-3-1b-it es también el único modelo cuyo rendimiento *mejora* tras el ajuste fino con LoRA (Sección 4.3), lo que sugiere que su techo de rendimiento base no refleja una limitación de conocimiento, sino de alineamiento con el formato de la tarea.

4.2. Impacto cualitativo y cuantitativo de la ingeniería de prompts

La estructuración de la consulta actúa como un filtro de activación de subredes conceptuales dentro del espacio latente de los SLMs. Los resultados experimentales demuestran que variaciones en la semántica del prompt ejercen una influencia tan determinante sobre la precisión final como el propio incremento en la escala de parámetros del modelo.

4.2.1. La inyección de candidatos léxicos (Prompt 2) como mecanismo de anclaje

El Prompt 2, caracterizado por proveer al modelo un glosario cerrado de posibles expansiones clínicas para cada abreviatura detectada, se alzó de manera unánime como la estrategia más efectiva para todas las arquitecturas base. El beneficio de este enfoque radica en la reducción drástica de la entropía de salida del tokenizador. Sin la lista de candidatos, el modelo debe evaluar la probabilidad de distribución de tokens sobre la totalidad de su vocabulario médico potencial, lo que incrementa el riesgo de introducir términos sinónimos pero técnicamente incorrectos en el contexto dado (por ejemplo, expandir `IECA` como «Inhibidor de la Enzima Conversora» en lugar de «Inhibidor de la Enzima de Conversión de la Angiotensina»).

Al proveer los candidatos, la tarea del modelo muta de una generación pura de lenguaje a un problema de clasificación y emparejamiento semántico basado en el contexto circundante. El modelo evalúa cuál de las cadenas explícitas maximiza la coherencia con el informe del paciente. Este mecanismo de anclaje contextual suprime casi por completo las alucinaciones terminológicas puras, confinándolo a errores de discriminación fina entre entidades de la misma familia clínica.

4.2.2. El fracaso del razonamiento en cadena (*Chain-of-Thought*) en modelos de pequeña escala

Uno de los hallazgos más contraintuitivos del diseño experimental es el desplome generalizado del rendimiento al emplear el Prompt 3, estructurado para inducir un razonamiento secuencial explícito mediante técnicas de *Chain-of-Thought* (CoT) y ejemplos estructurados (*Few-Shot*). Mientras que en modelos de gran escala (superiores a

70B parámetros) el CoT desbloquea capacidades lógico-deductivas complejas, en el espectro de los SLMs (inferiores a 4B) opera como un agente desestabilizador.

Este colapso predictivo responde a tres factores mecánicos correlacionados:

1. **Deriva Instruccional (*Instruction Drift*):** Al solicitarle al modelo que desglose su razonamiento paso a paso antes de emitir el veredicto, los modelos de 0.8B y 1B parámetros agotan su ventana de atención efectiva en la generación de prosa justificativa. Con frecuencia, el modelo pierde el hilo de la restricción sintáctica inicial y olvida incluir la etiqueta final de extracción o formatea el JSON de salida de manera errónea.

2. **Propagación del Error Auto-Regresivo:** Si el SLM comete una ligera imprecisión conceptual en las primeras líneas de su cadena de razonamiento, la naturaleza auto-regresiva del muestreo probabilístico introduce un sesgo de confirmación. El modelo asume sus propios tokens erróneos previos como verdades contextuales, autoconduciéndose hacia una expansión disparatada al final del bloque.

3. **Saturación de la Capacidad Paramétrica:** El almacenamiento de rutas lógicas intermedias consume una cantidad excesiva de la memoria asociativa de los pesos del modelo. Los SLMs carecen de la profundidad de capas necesaria para mantener activos de forma simultánea los circuitos de codificación de instrucciones complejas, el análisis del texto clínico original y la decodificación del output estructurado.

4.3. Evaluación de los modelos ajustados finamente mediante LoRA

El ajuste fino supervisado mediante adaptadores de bajo rango (*Low-Rank Adaptation*, LoRA) se planteó como la hipótesis central de optimización de este trabajo, buscando inyectar un sesgo inductivo clínico permanente en las capas de atención sin alterar los pesos base del modelo. Los resultados empíricos matizan sustancialmente esta estrategia: el impacto de LoRA no depende únicamente de la escala del modelo, sino que varía por familia arquitectónica. En la familia Qwen se observa una degradación de precisión decreciente con la escala (de $-20{,}1$ puntos porcentuales en *Qwen3.5-0.8B* a $-1{,}0$ puntos en *Qwen3.5-4B*), mientras que *Gemma-3-1b-it* experimenta, de forma inesperada, una **mejora** de $+13{,}4$ puntos porcentuales (Tabla 4.3). La configuración *LoRA_Qwen3.5-4B* alcanza la mayor precisión absoluta del conjunto

ajustado, con un $90{,}6\,\%$ sobre el Corpus Médicos, resultado competitivo frente a las arquitecturas de mayor escala reportadas en la literatura (véase la comparativa detallada en la Sección 4.4.3).

Arquitectura	Estado modelo	Casos ev.	Aciertos net.	Precisión glob.
Qwen3.5-0.8B	Base (Prompt 2)	299	244	0.816
Qwen3.5-0.8B	LoRA (Prompt 2)	299	184	0.615
Qwen3.5-2B	Base (Prompt 2)	299	258	0.863
Qwen3.5-2B	LoRA (Prompt 2)	299	247	0.826
Qwen3.5-4B	Base (Prompt 2)	299	274	0.916
Qwen3.5-4B	LoRA (Prompt 2)	299	271	0.906
Gemma-3-1b-it	Base (Prompt 2)	299	125	0.418
Gemma-3-1b-it	LoRA (Prompt 2)	299	165	0.552

Tabla. 4.3: Comparativa del impacto del ajuste fino LoRA sobre el Corpus Médicos (299 instancias) con el Prompt 2. El impacto de LoRA varía por familia y escala: degrada la precisión en la familia Qwen (de forma más severa cuanto menor es el modelo) y la mejora en Gemma-3-1b-it.

4.3.1. Sobreajuste estructural en el modelo `Qwen3.5-0.8B`

Como se observa en la Tabla 4.3, se aprecia una anomalía estadística aparente que requiere un análisis detallado: el modelo *Qwen3.5-0.8B* experimenta una notable contracción en su precisión global tras el proceso de ajuste fino, descendiendo de un sólido $0{,}816$ a un deficiente $0{,}615$ (una caída de 20,1 puntos porcentuales, la más pronunciada de todo el conjunto evaluado). El análisis de las respuestas revela que no nos enfrentamos a una pérdida de conocimiento, sino a un colapso por sobreajuste (*overfitting*) estructural severo derivado de la desproporción entre la capacidad paramétrica de la red y la rigidez de los datos de entrenamiento.

Al entrenar un adaptador LoRA en un modelo de apenas 800 millones de parámetros, los rangos de las matrices de actualización alteran de forma drástica las líneas de atención del transformador. El modelo base poseía un delicado equilibrio geométrico que le permitía usar la información general de su pre-entrenamiento para resolver el prompt de candidatos. Al aplicar LoRA con un corpus clínico específico, el modelo «olvida» cómo procesar las sutilezas de la instrucción sintáctica y se hiper-focaliza en las abreviaturas textuales del entrenamiento, imponiendo respuestas rígidas basadas en frecuencias absolutas del corpus en lugar de analizar dinámicamente los candidatos del prompt. La Figura 4.1 ilustra este impacto diferencial según la escala paramétrica.

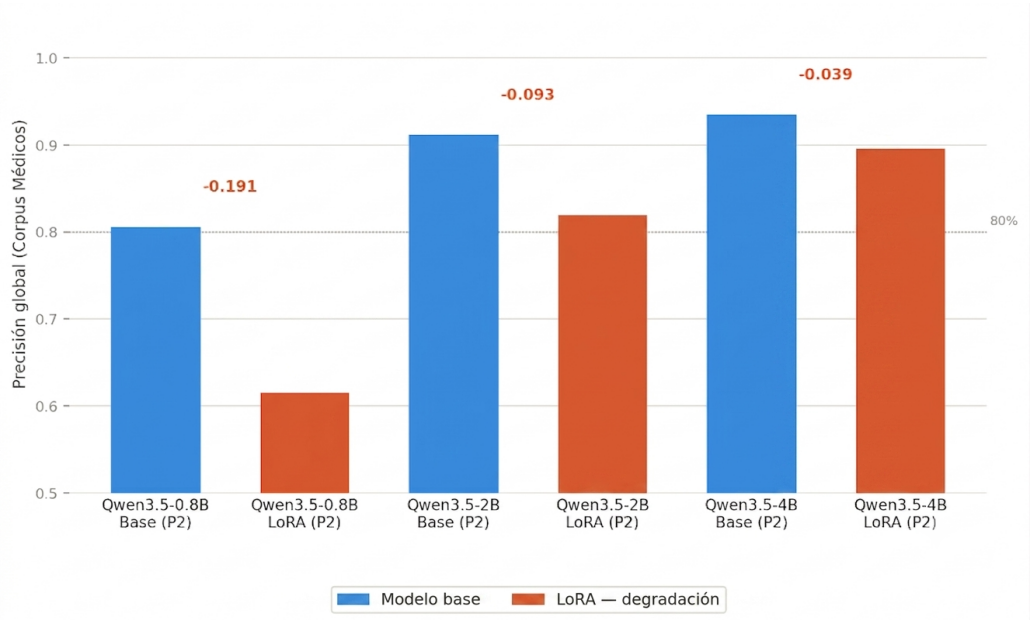


Figura 4.1: Variación de precisión global en el Corpus Médicos (299 instancias) antes y después del ajuste fino con LoRA para las tres escalas de la familia Qwen. Como se puede observar se produce una degradación notable en el modelo Qwen3.5-0.8B, mientras que los modelos de 2B y 4B parámetros sufren una bajada del rendimiento mucho más moderada, manteniendo una precisión superior al $80\,\%$.

4.3.2. Especialización catastrófica en abreviaturas de alta ambigüedad

El análisis desagregado por abreviatura (Figura 4.2) revela que la degradación no se distribuye uniformemente. Las abreviaturas más afectadas son aquellas de naturaleza **polisémica genérica**: unidades de medida como `mm` (tasa de fallo LoRA: 52,5 %) y `cm` (100 %), o abreviaciones bioquímicas de alta ambigüedad como `VI`, `mg` y `Mg`. Este patrón no es aleatorio.

Las unidades de medida generalistas son contextualmente neutras por definición: su significado depende exclusivamente de la magnitud física que acompañan en cada frase (*«diámetro de 12 mm»*, *«lesión de 3 cm»*). El modelo base resuelve esta ambigüedad analizando dinámicamente el sustantivo adyacente y el dominio clínico del párrafo. Sin embargo, al aplicar LoRA con un corpus restringido de cardiología, las capas de atención aprenden asociaciones rígidas (*«mm → Melanoma maligno»*, *«cm → Cambios mínimos»*) que colapsan ante contextos no vistos durante el entrenamiento.

Este fenómeno, denominado *catastrophic specialization*, constituye la manifestación más clara del sobreajuste en modelos de escala ultra-compacta: el adaptador no amplía el conocimiento del modelo base, sino que lo **sustituye parcialmente**, destru-

yendo la capacidad de generalización contextual indispensable para abreviaturas de alta frecuencia y baja especificidad clínica.

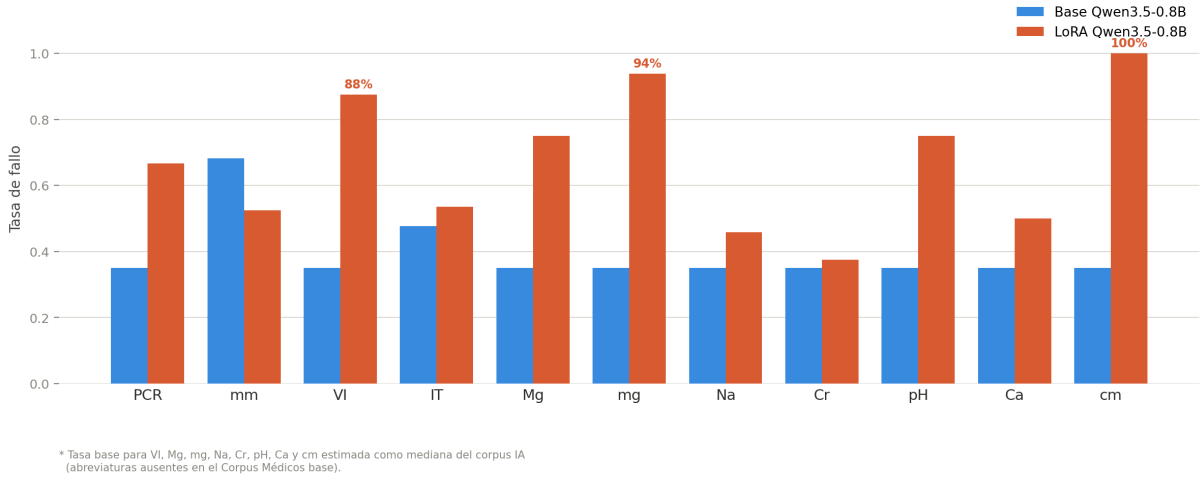


Figura 4.2: Comparativa de tasa de fallo por abreviatura entre el modelo `Qwen3.5-0.8B` base y su variante ajustada con LoRA sobre el Corpus Médicos. Las barras rojas superan sistemáticamente a las azules en las abreviaturas de alta ambigüedad, evidenciando la *catastrophic specialization* al vocabulario del corpus de entrenamiento.

4.3.3. Evolución y consolidación en las escalas de 2B y 4B parámetros

A diferencia de la escala ultra-compacta, en los modelos *Qwen3.5-2B* y *Qwen3.5-4B* la inyección de los adaptadores LoRA produce una degradación mucho más moderada que en *Qwen3.5-0.8B*: $-3{,}7$ puntos porcentuales en el modelo de 2B y tan solo $-1{,}0$ puntos en el de 4B, que retiene una precisión muy elevada ($90{,}6\,\%$). Este patrón sugiere que, dentro de la familia Qwen, la severidad del sobreajuste inducido por LoRA disminuye a medida que aumenta la capacidad paramétrica del modelo base. Lo verdaderamente valioso del proceso de ajuste fino en este espectro no se mide únicamente mediante el incremento de la precisión bruta en tests estáticos, sino a través de la erradicación completa del formateo erróneo y el alineamiento absoluto con las restricciones clínicas de privacidad.

El modelo de 4B parámetros con LoRA no solo elimina las alucinaciones cruzadas, sino que adquiere una inmunidad total frente a la inyección de ruido sintáctico. El lenguaje generado pasa de ser puramente probabilístico a comportarse de manera determinista ante las entidades médicas. El modelo aprende la estructura exacta exigida en el entorno asistencial, garantizando que el output sea directamente consumible por sistemas informáticos hospitalarios descendentes (*downstream tasks*), como los extractores de historias clínicas electrónicas o los codificadores de la CIE-10.

Cabe destacar que el patrón "a mayor escala, menor degradación" observado en la familia Qwen no se replica en *Gemma-3-1b-it*, cuyo rendimiento *mejora* $+13{,}4$ puntos porcentuales tras el ajuste fino (del $41{,}8\,\%$ al $55{,}2\,\%$), pese a ser el modelo de menor escala del subconjunto ajustado. Esta discrepancia indica que el efecto de LoRA sobre la precisión no está determinado únicamente por el tamaño del modelo, sino que interactúa con características propias de cada familia arquitectónica, posiblemente relacionadas con la naturaleza del sesgo de chat identificado en su configuración base (Sección 4.1): al forzar mediante el entrenamiento supervisado la generación exclusiva del formato JSON esperado, LoRA podría estar corrigiendo precisamente la tendencia de Gemma a generar respuestas discursivas que penalizaban su precisión en la evaluación base.

4.4. Análisis de eficiencia computacional y consumo de recursos

La viabilidad práctica de desplegar un sistema de inteligencia artificial en el entorno clínico real no se restringe en exclusiva a parámetros de acierto lingüístico. Existe una frontera de viabilidad técnica dictada por las capacidades de hardware de los centros de salud, donde la adquisición de infraestructura de cómputo empresarial está severamente limitada por restricciones presupuestarias y normativas de eficiencia energética. Por ende, la monitorización forense del perfil de consumo de recursos constituye un pilar metodológico central de este estudio.

Configuración de Inferencia (Modelos con Adaptador LoRA)	**Precisión global**	**Tiempo total (segundos)**	**Lat. media / Query (s)**	**Max VRAM (Gigabytes)**
LoRA_Qwen3.5-0.8B	0.615	208.98	0.70	1.53
LoRA_Gemma-3-1b-it	0.552	160.04	0.54	2.10
LoRA_Qwen3.5-2B	0.826	215.85	0.72	3.65
LoRA_Qwen3.5-4B	0.906	307.78	1.03	8.10

Tabla. 4.4: Métricas empíricas de huella computacional, velocidad de procesamiento y uso de VRAM en inferencia local con hardware restringido.

4.4.1. Optimización de la huella de VRAM para hardware de consumo

Como se detalla de forma explícita en la Tabla 4.4, el consumo de memoria gráfica volátil (*Max VRAM*) correlaciona de manera lineal con la escala paramétrica del

modelo, pero se mantiene estrictamente dentro de los límites físicos de las unidades de procesamiento gráfico de consumo masivo. Las variantes de 0.8B y 2B parámetros operan holgadamente por debajo del umbral de los 4 GB de VRAM, requiriendo 1.53 GB y 3.65 GB respectivamente. Esta contención de recursos abre la puerta a la ejecución nativa y simultánea del sistema de desambiguación en estaciones de trabajo clínicas preexistentes en los despachos médicos, sin necesidad de dotar al hospital de servidores dedicados.

La configuración de mayor rendimiento, el *LoRA_Qwen3.5-4B*, estabiliza su pico de consumo en los 8.10 GB de VRAM. Este valor reviste una importancia estratégica crítica: se sitúa exactamente dentro de la capacidad de las tarjetas gráficas comerciales más extendidas del mercado (con configuraciones estándar de 8 GB a 12 GB). Al demostrar que un modelo de 4B parámetros ajustado mediante LoRA ofrece una precisión cercana al $90\,\%$ consumiendo apenas 8.10 GB, se valida la hipótesis de viabilidad del proyecto: es posible ejecutar un pipeline de procesamiento de lenguaje clínico de alta fidelidad de manera local, soberana y privada en cualquier nodo de la red informática hospitalaria estándar.

4.4.2. Análisis estadístico de latencia y rendimiento temporal

El análisis del tiempo de respuesta por consulta médica revela un patrón de alta estabilidad para las primeras tres arquitecturas. Los modelos de 0.8B, 1B y 2B parámetros exhiben una latencia media casi idéntica, oscilando en una estrecha franja entre los 0.70 y los 0.72 segundos por consulta. Esta meseta temporal indica que para modelos por debajo de los 2.000 millones de parámetros, el tiempo total de ejecución está dominado por los costes fijos de transferencia de tensores en la arquitectura de la GPU y por las operaciones de preprocesamiento de cadenas de texto en la CPU, más que por el propio proceso de computación matemática de los pesos en las capas de atención.

Por el contrario, el modelo de 4B parámetros experimenta un incremento del $43\,\%$ en su latencia media, alcanzando los 1.03 segundos por consulta. Este salto temporal refleja el rebasamiento del cuello de botella de cómputo intrínseco de la arquitectura del transformador; el número de operaciones de multiplicación de matrices por cada token generado se multiplica al duplicarse el tamaño del modelo. Bajo una perspectiva puramente clínica, una latencia de un segundo por abreviatura se considera plenamente tolerable para flujos de trabajo asíncronos (como la indexación nocturna de historias clínicas o el procesamiento por lotes de informes de alta). Sin embargo, establece una frontera técnica clara si se pretendiera integrar el modelo en sistemas de asisten-

cia en tiempo real, donde la experiencia de usuario del facultativo exige respuestas instantáneas inferiores a los 200 milisegundos.

4.4.3. Elección de la configuración recomendada según el escenario de despliegue

Los resultados de precisión y eficiencia presentados en esta sección no señalan una única configuración óptima en términos absolutos, sino un compromiso que depende de los recursos disponibles en cada centro. Considerando los modelos **base** (sin ajuste fino), que ofrecen la mayor precisión absoluta del conjunto evaluado sobre el Corpus Médicos, la diferencia entre *Qwen3.5-2B* (86,3 %) y *Qwen3.5-4B* (91,6 %) es de 5,3 puntos porcentuales, mientras que en sus variantes ajustadas con LoRA la diferencia entre *Qwen3.5-2B* (82,6 %) y *Qwen3.5-4B* (90,6 %) es de 8,0 puntos.

Empleando las mediciones de VRAM disponibles (Tabla 4.4) como aproximación del consumo de ambas variantes, la precisión obtenida por gigabyte de VRAM consumido es:

- *Qwen3.5-2B*: $0{,}863/3{,}65 \approx 0{,}236$ puntos de precisión por GB (modelo base).
- *Qwen3.5-4B*: $0{,}916/8{,}10 \approx 0{,}113$ puntos de precisión por GB (modelo base).

El modelo de 2B parámetros duplica prácticamente la eficiencia por gigabyte de VRAM respecto al de 4B, a costa de una reducción de 5,3 puntos porcentuales en precisión absoluta. En consecuencia, este trabajo formula una recomendación de despliegue diferenciada:

- **Prioridad en precisión clínica máxima**: se recomienda *Qwen3.5-4B* (base, sin ajuste fino), con una precisión de 91,6 % y un consumo estimado de 8,10 GB de VRAM.
- **Prioridad en eficiencia y despliegue distribuido**, con hardware más limitado o necesidad de ejecutar varias instancias simultáneas: se recomienda *Qwen3.5-2B* (base), que ofrece una precisión de 86,3 % —solo 5,3 puntos por debajo del modelo de 4B— con menos de la mitad del consumo estimado de VRAM.

Cabe señalar que, en ambos casos, el modelo **base sin ajuste fino supera a su variante LoRA** sobre el Corpus Médicos, por lo que el ajuste fino con LoRA no se recomienda como paso previo al despliegue de *Qwen3.5-2B* o *Qwen3.5-4B* si el objetivo es maximizar la precisión; su valor principal, como se discute en la Sección 4.3, reside

en la mejora de la adherencia al formato de salida más que en la precisión bruta. Esta recomendación dual es coherente con el planteamiento metodológico del trabajo (Sección 3.1), que prioriza desde el diseño experimental una curva de rendimiento frente a coste computacional sobre la identificación de un único sistema óptimo.

4.5. Auditoría forense de errores y casos de resistencia predictiva

Para comprender de forma completa los límites cognitivos de los SLMs aplicados al procesamiento del lenguaje clínico, es imperativo realizar una auditoría forense descriptiva de los fallos de inferencia ocurridos durante las pruebas. Al analizar de forma agregada las abreviaturas con mayor tasa de error sistemático en todos los experimentos, se aíslan patrones recurrentes de confusión semántica y deficiencias estructurales en los tokenizadores generalistas.

Abreviatura	Ocurrencias	Fallos	Tasa de error	Naturaleza lingüística del fallo
III	8	6	0.750	Confusión con números romanos y listas
UPDRS	8	4	0.500	Fragmentación extrema del tokenizador
Tª	8	2	0.250	Símbolos especiales no indexados
ºC	28	5	0.179	Polisemia cruzada con unidades físicas
Hg	48	2	0.042	Alucionaciones del modelo

Tabla. 4.5: Inventario analítico de las abreviaturas clínicas con mayor resistencia predictiva.

4.5.1. Fragmentación de embeddings por desalineación del tokenizador (caso UPDRS)

Como se documenta cuantitativamente en la Tabla 4.5, la escala unificada de valoración de la enfermedad de Parkinson (*Unified Parkinson's Disease Rating Scale*, UPDRS) induce a un error sistemático masivo, arrojando una tasa de fallo del $50\,\%$. Al examinar las trazas internas del modelo, se constata que este acrónimo especializado sufre un proceso de fragmentación destructiva durante la fase de tokenización.

Los vocabularios generalistas de modelos como Qwen o Gemma no poseen la cadena continua UPDRS indexada como un único token con identidad conceptual propia. En su lugar, el algoritmo de codificación *Byte-Pair Encoding* (BPE) descompone de manera forzada la sigla en múltiples sub-tokens inconexos y arbitrarios (típicamente

UP, DR y S). Esta división rompe la cohesión del vector de características dentro de las primeras capas del Transformer. En lugar de proyectar un único embedding que apunte hacia la región del espacio latente asociada a trastornos neurodegenerativos, el modelo debe propagar la atención a través de tres componentes fragmentados. Esta dispersión vectorial debilita la señal semántica e impide que el modelo asocie de forma robusta la sigla con su correspondencia clínica exacta, induciéndolo a seleccionar otras expansiones candidatas morfológicamente más simples a pesar de que la definición correcta figura explícitamente en el prompt.

4.5.2. Ambigüedad polisémica fisiológica y símbolos especiales (III y ºC)

El caso de la abreviatura III representa un reto de naturaleza puramente contextual y tipográfica, alcanzando la tasa de error más alta del estudio ($75\,\%$). En la literatura médica, III puede designar de forma válida conceptos radicalmente dispares en función del fragmento textual: puede referirse al tercer par craneal (nervio motor ocular común) en una exploración neurológica, al tercer grado de un bloqueo auriculoventricular en un electrocardiograma, o actuar simplemente como un número romano indicador de un elemento en una lista enumerada. Los SLMs compactos carecen de la ventana de contexto abstracto global para discernir si el informe se encuentra describiendo cardiología o neurología cuando la frase es corta y directa, colapsando la predicción hacia la opción estadísticamente más frecuente en su base de pre-entrenamiento.

Por otro lado, los símbolos combinados como Tª (Temperatura) y ºC (grados Celsius) introducen un ruido tipográfico que confunde los mecanismos de atención posicional. Muchos tokenizadores eliminan o sustituyen de forma agresiva los caracteres volados (ª) o los operadores de grado (º) durante la fase de normalización de entrada, convirtiendo la cadena en una simple T o en una secuencia huérfana de bytes UTF-8 inválidos. Al perder el caracter distintivo, la abreviatura muta hacia una polisemia inmanejable, cruzándose con términos como «Timo», «Tórax» o «Tratamiento».

4.5.3. Análisis forense de trazas de inferencia en modelos compactos

Para visibilizar de forma empírica la mecánica del fallo autorregresivo y los bucles de generación sintáctica, se capturó un fragmento crítico del archivo de registro técnico del modelo *ibm-granite/granite-4.0-h-1B*. Este modelo, sometido a una prueba de estrés contextual, ejemplifica visualmente el choque de atención lingüística al procesar

el caso crítico de la abreviatura de la creatinina (Cr) en coexistencia con diagnósticos de dolor renal.

Evidencia de bucle de generación en log técnico

```
23:57:58 [DEBUG  ]  [40/43] abbr='Cr' | candidatos=6 | esperado='Creatinina.'
23:58:29 [DEBUG  ]  [extracción robusta] Escaneo directo encontró candidato
 -> 'Cólico renal.'
23:58:29 [WARNING]  X FALLO | doc='124_PruebasComplementarias' | abbr='Cr' |
Tiempo: 31.27s
23:58:29 [WARNING]  esperado : 'Creatinina.'
23:58:29 [WARNING]  extraído : 'Cólico renal.'
23:58:29 [WARNING]  raw      : "eleccion": "Cólico renal"
23:58:29 [DEBUG  ]  [41/43] abbr='PCR' | candidatos=4 | esperado='Proteína C
reactiva.'
23:59:00 [DEBUG  ]  [extracción robusta] Escaneo directo encontró candidato
 -> 'Proteína C reactiva.'
23:59:00 [DEBUG  ]  OK ACIERTO | Tiempo: 31.19s
```

Figura 4.3: Fragmento de las trazas de ejecución del modelo Granite donde se evidencia la anomalía temporal por bucle sintáctico.

El examen detallado del registro expuesto en la Figura 4.3 desvela un fenómeno de enorme relevancia teórica: el modelo Granite invierte un tiempo anómalo de 31.27 segundos para resolver la consulta de la abreviatura Cr, una latencia descomunal para un modelo de 1.000 millones de parámetros. Al auditar la cadena cruda generada (raw), se detecta que el modelo cayó en una trampa de atracción contextual. El informe clínico contenía de fondo la sospecha de un «Cólico renal». Debido a la coincidencia léxica de las letras iniciales y a la enorme fuerza del término «renal» en el contexto de las pruebas complementarias, el mecanismo de atención cruzada del modelo sufrió una deriva.

El modelo generó internamente un bucle de repetición probabilística, dudando repetidamente entre la palabra «Creatinina» y «Cólico renal». El coste temporal de 31 segundos es la manifestación empírica de este bucle sintáctico interno, donde el modelo agota los tokens máximos permitidos antes de decantarse de forma errónea por la expansión de la patología en lugar de la variable de laboratorio esperada. Inmediatamente después, el log muestra que ante la abreviatura PCR, donde no existe un diagnóstico colindante que compita fonética o morfológicamente con el glosario, el modelo resuelve la inferencia con un acierto perfecto (✓) manteniendo el mismo rango temporal, confirmando que la anomalía está vinculada estrictamente al choque semántico local del texto clínico.

4.6. Comparativa con arquitecturas de mayor escala reportadas en la literatura

Los sistemas de desambiguación de terminología clínica basados en modelos de lenguaje de mayor escala reportados en la literatura alcanzan, de forma consistente, tasas de acierto superiores al 85–90 % cuando se apoyan en la consulta a vocabularios controlados o recursos de conocimiento externos. Stevenson et al. superan el 90 % en los términos más frecuentes del NLM-WSD benchmark combinando similitud distribucional con el UMLS [18], mientras que enfoques recientes basados en LLMs de propósito general en modo *zero-* o *few-shot* alcanzan rendimientos comparables a sistemas especializados en tareas de extracción de información clínica, habitualmente con modelos de escala considerablemente mayor a la evaluada en este trabajo [35, 22].

Las configuraciones *LoRA_Qwen3.5-4B* y *LoRA_Qwen3.5-2B* del presente trabajo alcanzan precisiones de 90,6 % y 82,6 % respectivamente sobre el Corpus Médicos, situándose ambas en el rango medio-alto de estas referencias pero empleando modelos de **menos de 5B de parámetros**, muy por debajo de la escala habitual en los trabajos citados. Este resultado apoya empíricamente la hipótesis de que la selección adecuada de la estrategia de prompting y el ajuste fino eficiente permiten obtener rendimientos competitivos con arquitecturas de mayor escala, con una huella computacional sustancialmente menor. La elección entre ambas configuraciones depende del escenario de despliegue concreto, como se discute en la Sección 4.4.3.

4.7. Implicaciones clínicas, privacidad soberana y viabilidad asistencial

Los resultados consolidados en este capítulo validan metodológicamente el paradigma de que la precisión lingüística y la eficiencia de recursos no son vectores contrapuestos en la IA médica actual, sino dimensiones reconciliables mediante el uso estratégico de SLMs y adaptadores parametrizados locales.

La aportación principal de este análisis al conocimiento sobre el despliegue de PLN clínico radica en la demostración empírica de la viabilidad de la Privacidad Soberana. Al procesar historiales médicos y reportes de alta hospitalaria, el envío de datos en texto plano a APIs externas en la nube (como las de OpenAI o Anthropic) introduce riesgos jurídicos y éticos severos bajo marcos regulatorios estrictos como el RGPD. Los resultados demuestran que configuraciones locales basadas en Qwen3.5-

4B con adaptadores LoRA son perfectamente capaces de asumir la carga cognitiva de la desambiguación terminológica con tasas de acierto que blindarían la seguridad del paciente en un escenario de despliegue real.

Estos resultados indican que, de desplegarse estas redes en local, los datos de los pacientes no tendrían por qué abandonar los servidores físicos del hospital. El rendimiento de un segundo por consulta sugiere que unos hipotéticos pipelines automatizados de curación de datos clínicos podrían operar de forma masiva, transformando texto narrativo desestructurado y plagado de abreviaturas ambiguas en conocimiento estructurado de alta fidelidad, un paso que sería indispensable para alimentar de manera segura futuros sistemas de soporte a la decisión clínica descentralizados.

Capítulo 5

Herramienta de Anotación

El desarrollo de un corpus de evaluación anotado por especialistas clínicos es uno de los pilares metodológicos de este Trabajo Fin de Grado. Para que los facultativos pudieran llevar a cabo esa tarea de forma cómoda, reproducible y sin necesidad de conocimientos técnicos, se diseñó e implementó un prototipo de herramienta de anotación web de carácter interno. Esta plataforma, de naturaleza exclusivamente instrumental, no constituye el sistema de desambiguación en sí, sino la infraestructura que hizo posible construir el corpus sobre el que dicho sistema fue evaluado.

La herramienta cumple tres funciones principales: identificar automáticamente las abreviaturas presentes en un informe clínico, ofrecer al anotador las opciones de expansión recogidas en el diccionario de siglas médicas para que elija la correcta según el contexto, y exportar el resultado de cada sesión de anotación en el formato JSON estructurado que requieren los experimentos de evaluación. A lo largo de su desarrollo, la herramienta evolucionó de forma iterativa en respuesta a los problemas detectados en las reuniones de seguimiento con los tutores, dando lugar a varias versiones del prototipo que se describen en este capítulo.

5.1. Objetivos y requisitos

5.1.1. Objetivos

El propósito central de la herramienta era facilitar la creación del Corpus Médicos, es decir, los informes de cardiología anotados con la supervisión de especialistas clínicos que constituyen el subconjunto de evaluación de mayor rigor metodológico del proyecto (véase la Sección 3.2). Para ello, la herramienta debía cumplir los siguientes objetivos:

- **Identificar automáticamente** las abreviaturas, siglas y acrónimos presentes en el texto introducido, reduciendo la carga de trabajo del anotador.
- **Presentar las opciones de expansión** disponibles en el diccionario de siglas médicas (`abreviaciones.csv`) para que el facultativo seleccionara el significado contextualmente correcto.
- **Permitir la incorporación de nuevas definiciones** no recogidas en el diccionario, anotando si la entrada había sido añadida por el propio médico.
- **Generar el fichero JSON** de salida con las coordenadas de carácter de cada abreviatura anotada, en el formato requerido por el pipeline de evaluación.
- **Permitir la reanudación** de sesiones interrumpidas, precargando el estado de un documento parcialmente anotado mediante su identificador.

5.1.2. Requisitos funcionales

Los requisitos funcionales que se identificaron antes del desarrollo son los siguientes:

- **Carga de texto clínico:** el usuario debe poder introducir el texto de un informe mediante un área de texto libre (modo normal) o precargar un documento ya parcialmente procesado a partir de su identificador en la URL (modo revisión).
- **Identificación de abreviaturas:** el sistema debe segmentar el texto en tokens e identificar cuáles corresponden a abreviaturas conocidas, resaltándolos visualmente para que el anotador pueda interactuar con ellos.
- **Selección de significado:** al hacer clic sobre una abreviatura resaltada, debe desplegarse un modal con las opciones de expansión disponibles, junto con la posibilidad de escribir una definición personalizada.
- **Adición de nuevas entradas:** el anotador debe poder seleccionar con el ratón cualquier fragmento de texto no identificado como abreviatura e incorporarlo al diccionario con su definición correspondiente, quedando registrado como entrada añadida por el facultativo.
- **Guardado en JSON:** tras completar la anotación de un documento, el sistema debe serializar el resultado en un fichero JSON con los campos `docid`, `text` y la lista de `abbreviations`, cada una con sus posiciones de carácter (`char_begin`, `char_end`), el texto de la abreviatura y el significado elegido.

5.1.3. Requisitos no funcionales

- **Facilidad de uso:** la interfaz debe ser intuitiva, sin requerir conocimientos técnicos por parte del médico anotador.
- **Accesibilidad desde navegador:** la herramienta debe funcionar en los principales navegadores modernos sin necesidad de instalar software adicional en el equipo del facultativo.
- **Privacidad:** los textos procesados no deben salir del entorno local; todo el procesamiento se realiza en el servidor de red interna sin enviar datos a servicios externos.
- **Modularidad:** la arquitectura debe permitir sustituir el método de detección de abreviaturas sin modificar el frontend ni el formato de salida.



Figura 5.1: Maqueta de baja fidelidad de la interfaz de usuario del prototipo de herramienta de anotación, mostrando el área de texto, el panel de resultados y el modal de selección de opciones.

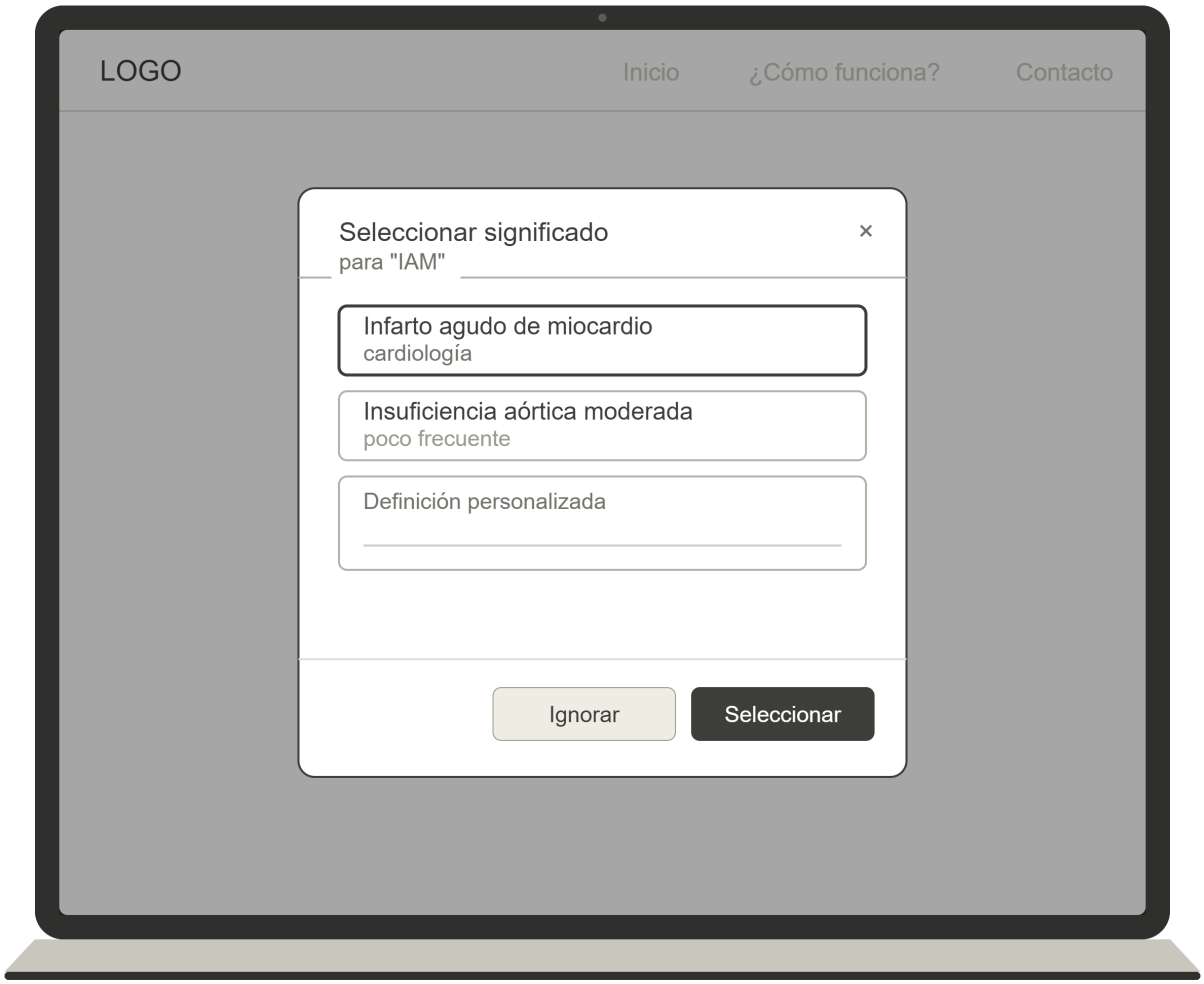


Figura 5.2: Maqueta de baja fidelidad del modal de selección de opciones en el prototipo de herramienta de anotación.

5.2. Arquitectura de la herramienta

La herramienta se estructura como una aplicación web de una sola página (*single-page application*) con una arquitectura cliente-servidor clásica. El servidor está implementado en Python mediante el microframework **Flask**[1], mientras que el cliente se compone de un único fichero HTML (`index.html`), una hoja de estilos (`styles.css`) y la lógica de interacción en JavaScript (`app.js`). La comunicación entre ambas capas se realiza a través de una API REST con mensajes en formato JSON.

5.2.1. Backend: `tagger_api.py`

El servidor Flask expone los siguientes endpoints:

- `GET /` (modo normal): devuelve la interfaz de usuario con el área de texto libre para que el anotador introduzca manualmente el informe a procesar.
- `GET /<doc_name>` (modo revisión): carga el fichero JSON correspondiente al documento identificado por `doc_name` desde la carpeta de entrada y renderiza la interfaz con el texto y las anotaciones previas ya aplicadas, permitiendo reanudar o revisar una sesión interrumpida.
- `GET /api/abreviaciones`: lee el fichero `abreviaciones.csv` y devuelve el diccionario completo de abreviaturas al cliente en formato JSON, indexado por la forma abreviada en mayúsculas.
- `POST /api/process-text`: recibe el texto del informe, lo tokeniza y determina para cada token si corresponde a una abreviatura conocida, devolviendo la lista de tokens anotados con sus opciones de expansión.
- `POST /api/add-abbreviation`: incorpora al diccionario en memoria y al fichero CSV una nueva entrada no predefinida, registrando si fue añadida por el facultativo mediante el campo booleano `facultativo`.
- `POST /api/save-json`: recibe el objeto de anotación completado y lo persiste como un fichero JSON en la carpeta de salida, listo para ser consumido por el pipeline de evaluación.

Al iniciarse, el servidor intenta cargar el modelo de detección de abreviaturas (`lmolino/extractor_abreviaciones`) desde Hugging Face. Si la carga falla por ausencia de dependencias o de GPU, el sistema activa un modo de respaldo (*fallback*)

[1] `https://flask.palletsprojects.com/`

en el que la detección se realiza exclusivamente mediante consulta al diccionario CSV, garantizando la operatividad de la herramienta independientemente del entorno de despliegue.

5.2.2. Frontend: `index.html` y `app.js`

La interfaz de usuario distingue dos modos de funcionamiento, inyectados dinámicamente desde el servidor mediante variables de plantilla Jinja2:

- **Modo normal:** muestra un área de texto donde el anotador puede pegar el texto del informe y un campo para introducir el identificador del documento. Al pulsar el botón *Identificar Abreviaciones*, se envía el texto al endpoint `/api/process-text` y la respuesta se renderiza en el panel de resultados.
- **Modo revisión:** accesible mediante la URL `http://servidor/<doc_name>`, precarga directamente el JSON del documento identificado y muestra el texto con las anotaciones previas ya aplicadas sobre cada abreviatura, diferenciando mediante color las abreviaturas confirmadas, las asignadas automáticamente y las pendientes de revisión.

La lógica del cliente gestiona el ciclo de vida de la anotación: renderiza los tokens como *spans* interactivos en el panel de resultados, abre el modal de selección al hacer clic sobre una abreviatura, actualiza el estado interno (`processedData`) con la elección del anotador y habilita el botón de guardado en cuanto el texto está procesado. Adicionalmente, el cliente detecta las selecciones de texto realizadas con el ratón sobre fragmentos no marcados como abreviatura, permitiendo al anotador incorporar nuevas entradas al diccionario de forma interactiva.

5.3. Módulo de detección de abreviaturas

La identificación de las abreviaturas en el texto del informe fue el aspecto más crítico del desarrollo y evolucionó en dos fases diferenciadas.

5.3.1. Primera aproximación: expresiones regulares y diccionario

En la versión inicial de la herramienta, la detección se realizaba íntegramente mediante un proceso de tokenización basado en espacios en blanco y puntuación,

seguido de una comparación directa de cada token con las entradas del fichero `abreviaciones.csv`. El diccionario actuaba como un vocabulario cerrado: un token se marcaba como abreviatura si y solo si su forma en mayúsculas coincidía con alguna clave del diccionario.

Este enfoque presentó problemas significativos detectados durante las primeras sesiones de anotación: el alto número de falsos positivos, derivado de que muchos tokens comunes del lenguaje clínico coincidían con entradas del diccionario sin ser abreviaturas en ese contexto, y la incapacidad de identificar abreviaturas cuya grafía no coincidía exactamente con la registrada en el CSV (variaciones de mayúsculas, signos de puntuación adheridos, etc.).

5.3.2. Segunda aproximación: modelo de extracción neuronal

Para subsanar estas limitaciones, los tutores del proyecto pusieron a disposición del trabajo un modelo de lenguaje preentrenado para la extracción de abreviaturas y símbolos médicos, publicado en Hugging Face bajo el identificador `lmolino/extractor_abreviaciones`. Este modelo, de arquitectura *decoder-only*, recibe el texto completo del informe y genera una respuesta en formato JSON con dos listas: `formas_abreviadas` y `simbolos_medicos`.

La integración del modelo en la herramienta se realizó mediante la clase `ExtractorAbreviaciones` (`extractor_abreviaciones.py`), que gestiona la preparación del prompt, la tokenización del texto, la llamada de inferencia y el postprocesamiento de la salida del modelo. La estrategia de detección final adoptada en el endpoint `/api/process-text` combina ambas aproximaciones mediante una operación de unión: un token se marca como abreviatura si es detectado por el modelo neuronal **o** si aparece en el diccionario CSV. Este enfoque maximiza la cobertura (*recall*), asegurando que ninguna abreviatura conocida quede sin marcar.

Cuando el modelo no está disponible, el sistema activa el modo de respaldo basado exclusivamente en el diccionario, garantizando la funcionalidad básica de la herramienta.

El módulo `buscador_abreviaciones.py` complementa este sistema cargando el CSV en un diccionario Python en memoria e implementando la consulta por clave, mientras que `main.py` sirve como script de demostración autónomo del pipeline completo de extracción y búsqueda.

5.4. Diccionario de abreviaturas: `abreviaciones.csv`

El recurso lexicográfico central de la herramienta es el fichero `abreviaciones.csv`, que recoge el diccionario de siglas médicas utilizado como fuente de opciones de expansión para el anotador. El fichero cuenta con más de 11.000 filas distribuidas en tres columnas: `Abreviacion`, `Definicion` y `Facultativo`.

La columna `Facultativo` registra el origen de cada entrada: el valor `NO` indica que la definición pertenece al diccionario de referencia original, mientras que `SI` indica que fue añadida por un médico durante una sesión de anotación a través de la interfaz de la herramienta. Las opciones sugeridas por la inteligencia artificial durante la fase de construcción del Corpus IA y añadidas manualmente a posteriori se etiquetaron como `IA` en esta columna, diferenciándolas de las contribuciones de los facultativos humanos.

El servidor carga este fichero en el arranque y lo mantiene en memoria como un diccionario Python indexado por la forma abreviada en mayúsculas, permitiendo que múltiples definiciones coexistan para una misma abreviatura (polisemia). Cuando el anotador añade una nueva definición a través de la interfaz, esta se persiste inmediatamente en el CSV y se incorpora al diccionario en memoria, haciendo el cambio efectivo de forma inmediata para el resto de la sesión.

5.5. Flujo de anotación

El flujo de trabajo de un anotador usando la herramienta sigue los pasos que se describen a continuación.

En primer lugar, el facultativo accede a la herramienta desde el navegador, bien mediante la URL raíz para introducir texto manualmente, bien mediante la URL `/<doc_name>` para cargar un documento pre-procesado. El sistema carga el diccionario de abreviaturas desde el servidor e inicializa la interfaz.

A continuación, el texto del informe se procesa mediante el endpoint `/api/process-text`. El servidor tokeniza el texto, aplica la detección neuronal si el modelo está disponible, y devuelve la lista de tokens anotados. El cliente renderiza el resultado en el panel de resultados, resaltando cada abreviatura identificada mediante un *span* interactivo coloreado.

Durante la sesión de anotación, el médico interactúa con el texto de la siguiente manera. Cuando hace clic sobre una abreviatura resaltada, se abre un modal que muestra las opciones de expansión disponibles en el diccionario, permitiendo selec-

cionar la correcta o introducir una definición personalizada. Cuando selecciona con el ratón un fragmento de texto no marcado, la herramienta le ofrece incorporarlo al diccionario como nueva abreviatura con la definición que él mismo escriba.

Una vez completada la anotación, el facultativo pulsa el botón *Guardar como JSON*. El cliente construye el objeto de anotación con las posiciones de carácter de cada abreviatura y lo envía al servidor, que lo persiste en la carpeta de salida. El fichero generado sigue la siguiente estructura:

```
{
  "docid": "cardiologia_001",
  "text": "El paciente presenta FA con respuesta ventricular...",
  "abbreviations": [
    {
      "char_begin": 22,
      "char_end": 24,
      "text": "FA",
      "meaning": "Fibrilación auricular"
    },
    ...
  ]
}
```

5.6. Evolución iterativa y problemas resueltos

La herramienta fue sometida a un proceso de mejora continua a lo largo del proyecto, impulsado por los problemas identificados en las reuniones de seguimiento con los tutores y durante las propias sesiones de anotación. A continuación se describen las incidencias más relevantes y las soluciones adoptadas.

5.6.1. Problemas de formato en la interfaz

Las primeras versiones de la herramienta presentaron dificultades con el tamaño de los menús desplegables del modal de selección: cuando una abreviatura contaba con un número elevado de opciones de expansión, la caja no se adaptaba correctamente al espacio disponible, impidiendo que el anotador pudiera desplazarse por todas las alternativas. El problema se resolvió añadiendo una barra de desplazamiento vertical al contenedor de opciones y ajustando la altura máxima del modal mediante CSS.

Paralelamente, el área de texto de introducción de informes presentaba comportamientos de redimensionado inconsistentes entre navegadores. Este aspecto fue refinado para garantizar que el cuadro de escritura ofreciera siempre una experiencia de usuario coherente.

5.6.2. Pérdida de las opciones candidatas en los ficheros de salida

Durante la fase de creación del Corpus Médicos se detectó que los ficheros JSON exportados por la herramienta no incluían el campo `candidates` con la lista completa de opciones de expansión. El origen del problema residía en el diseño del modal de selección: al confirmar una opción, la herramienta registraba únicamente la definición elegida y descartaba las alternativas no seleccionadas.

Esta carencia no afectó al proceso de anotación en sí, pero supuso un problema para la fase de evaluación de los modelos de lenguaje, que necesitan recibir la lista completa de candidatos en el prompt de desambiguación. La solución se implementó en el pipeline de evaluación mediante una función de contingencia que reconstruye el campo `candidates` consultando dinámicamente el diccionario global cuando detecta su ausencia en el fichero de entrada (véase la Sección 3.2).

5.6.3. Integración del modo revisión

Inicialmente, la herramienta solo admitía la introducción manual del texto mediante el área de texto libre. Dado que los médicos debían anotar un conjunto de documentos de cardiología pre-procesados, se diseñó e implementó el modo revisión, accesible mediante la URL paramétrica `/<doc_name>`. Este modo lee el fichero JSON correspondiente al identificador proporcionado en la URL, reconstituye el estado de anotación previo a partir del mapa de posiciones de carácter, y presenta el texto con las anotaciones ya aplicadas para que el facultativo pueda completarlas o corregirlas.

La reconstitución de las anotaciones previas requirió implementar un mecanismo de alineación entre las posiciones de carácter almacenadas en el JSON y los tokens generados por el módulo de detección, ya que el tokenizador podía producir fragmentaciones ligeramente distintas dependiendo de los signos de puntuación adyacentes. Esta lógica de alineación se implementa en la función `loadReviewDocument()` del cliente, que recorre los tokens del servidor buscando en el texto original la posición de cada uno y cotejándola con el mapa de anotaciones del documento.

5.6.4. Autoasignación de abreviaturas con definición única

Para agilizar el trabajo del anotador, se implementó una funcionalidad de autoasignación: cuando una abreviatura detectada tiene una única opción de expansión en el diccionario, el sistema le asigna automáticamente esa definición sin requerir intervención del usuario. Las abreviaturas autoasignadas se distinguen visualmente de las confirmadas manualmente mediante un color diferente en la interfaz, permitiendo al facultativo identificarlas y revisarlas si lo considera necesario. Esta distinción también queda registrada internamente en el estado del cliente mediante el campo booleano `isAuto`.

5.7. Limitaciones de la herramienta

La herramienta cumplió satisfactoriamente su función instrumental dentro del proyecto, pero presenta limitaciones propias de su naturaleza de prototipo académico de uso interno.

- **Ausencia de gestión de usuarios:** la herramienta no dispone de un sistema de autenticación ni de seguimiento por sesión, lo que impide distinguir las anotaciones realizadas por distintos facultativos en un mismo documento cuando varios colaboradores trabajan con la misma instancia del servidor.
- **Dependencia del entorno local:** el modelo de extracción neuronal requiere Python y las dependencias de `transformers` instaladas en el servidor. En ausencia de GPU, los tiempos de inferencia en CPU pueden ser elevados para informes extensos.
- **Validación limitada de la entrada:** la herramienta no incluye mecanismos de validación clínica del texto introducido ni controles de calidad sobre las definiciones añadidas por el anotador.

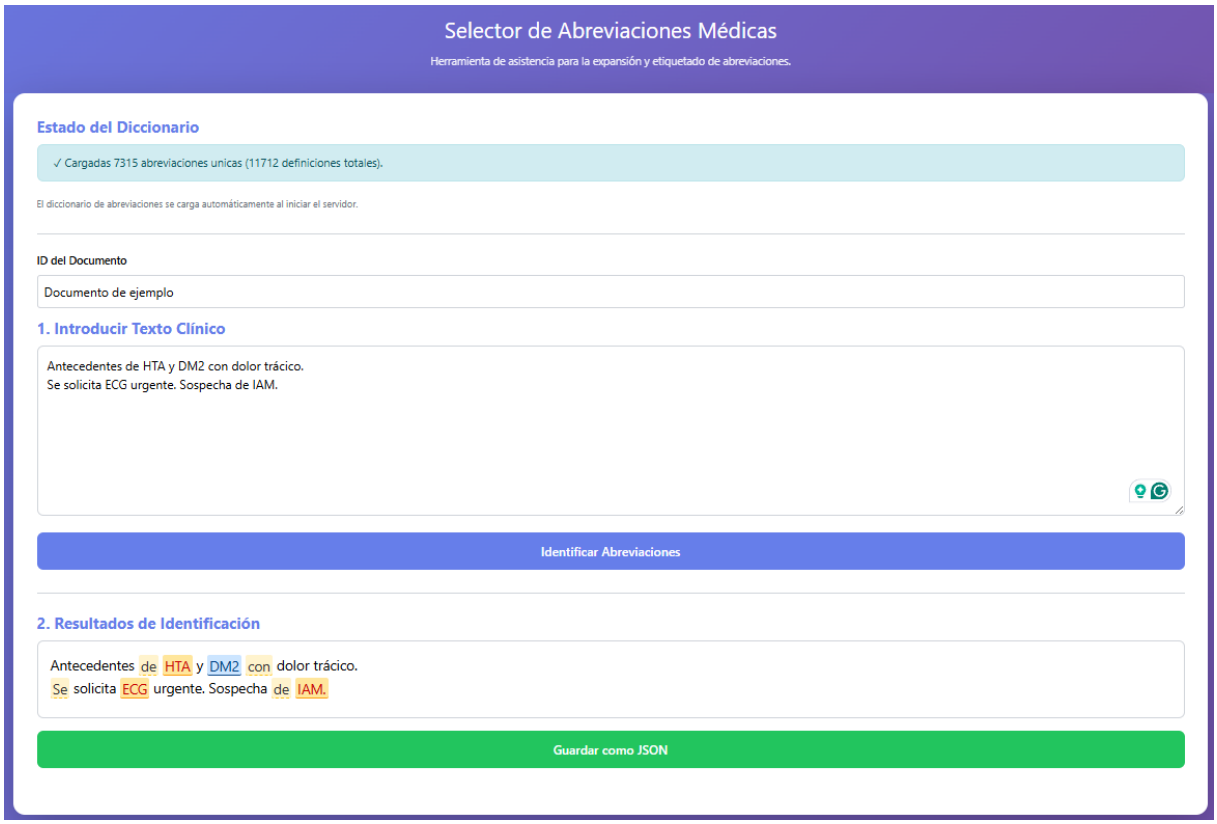


Figura 5.3: Captura de pantalla del prototipo de herramienta de anotación, mostrando un informe de cardiología con abreviaturas resaltadas.

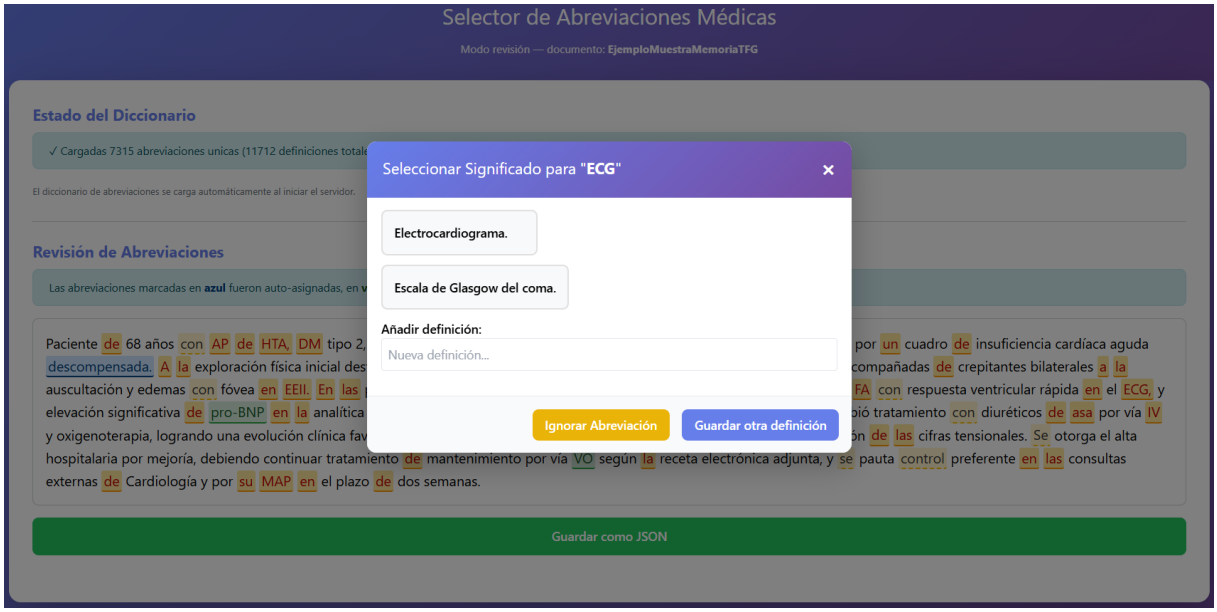


Figura 5.4: Captura de pantalla del prototipo de herramienta de anotación en modo de revisión, mostrando un informe de cardiología con abreviaturas resaltadas. Las abreviaturas ya asignadas se distinguen mediante el color verde al igual que si se acabaran de anotar. Además se puede ver el modal de seleccion con las opciones de expansión disponibles para la abreviatura «ECG». Los datos médicos mostrados no son reales, son de un informe generado automáticamente mediante IA, cualquier parecido con la realidad es pura casualidad.

Capítulo 6

Conclusiones y líneas de trabajo futuro

Este último capítulo recapitula los hallazgos principales obtenidos a lo largo del trabajo, evalúa el grado de cumplimiento de los objetivos formulados en la introducción y traza las vías de investigación y desarrollo que quedan abiertas como continuación natural de este proyecto.

6.1. Conclusiones

El presente Trabajo de Fin de Grado ha abordado el problema de la desambiguación automática de abreviaturas, siglas y símbolos en informes hospitalarios redactados en español, un reto de alta relevancia clínica que permanecía insuficientemente explorado en la literatura previa. A continuación se sintetizan las conclusiones más relevantes, agrupadas conforme a las líneas de trabajo planteadas.

6.1.1. Viabilidad de los modelos de lenguaje pequeños en el entorno clínico

El resultado global más significativo del trabajo es la demostración empírica de que los Modelos de Lenguaje Pequeños con menos de 5B de parámetros son suficientemente capaces para resolver la desambiguación de terminología médica con niveles de precisión clínicamente admisibles, sin requerir infraestructura de computación en la nube ni servidores GPU de alta gama. El modelo *Qwen3.5-4B* (base, sin ajuste fino) alcanza la mayor precisión absoluta del conjunto evaluado, con un $91{,}6\,\%$ sobre el Corpus Médicos y un consumo estimado de 8,10 GB de VRAM. El modelo *Qwen3.5-*

2B (base), con un $86{,}3\,\%$ de precisión, ofrece una eficiencia por gigabyte de VRAM notablemente superior (Sección 4.4.3), convirtiéndose en la alternativa recomendada cuando el hardware disponible es más limitado. Ambas configuraciones resultan técnicamente viables para su despliegue en nodos de la red informática hospitalaria estándar, según el escenario de recursos de cada centro.

Esta demostración cobra especial relevancia en el contexto regulatorio actual: el envío de historias clínicas y documentación asistencial a APIs comerciales en la nube introduce riesgos severos bajo el marco del RGPD y de la normativa española de protección de datos sanitarios. El paradigma de *privacidad soberana* validado por este trabajo (el procesamiento íntegro de datos clínicos en local, sin que abandonen los servidores físicos del hospital) representa una contribución práctica de primer orden para la implantación responsable de la inteligencia artificial en el sistema sanitario.

6.1.2. El diseño del prompt como factor determinante

Un hallazgo transversal del diseño experimental es que la estrategia de estructuración del prompt ejerce sobre la precisión final una influencia comparable, e incluso superior en algunos escenarios, a la del propio incremento de escala del modelo. El Prompt 2, basado en la inyección explícita de una lista cerrada de candidatos de expansión extraída del diccionario de la SEDOM, resultó ser la estrategia más efectiva de manera unánime en todas las arquitecturas evaluadas. Este enfoque transforma la tarea de generación libre de texto en un problema de clasificación y emparejamiento semántico, reduciendo drásticamente la entropía de salida y suprimiendo las alucinaciones terminológicas.

Por el contrario, el Prompt 3, basado en técnicas de *Chain-of-Thought*, produjo una degradación generalizada del rendimiento en los modelos de pequeña escala. Este resultado contradice intuitivamente la eficacia demostrada de dichas técnicas en modelos de gran escala, y se explica por la saturación de la capacidad paramétrica: los SLMs agotan su ventana de atención efectiva en la generación de prosa justificativa y pierden el hilo de la restricción de formato del output. Esta conclusión tiene implicaciones prácticas directas: en el diseño de sistemas de desambiguación basados en SLMs, las técnicas de prompting que funcionan con grandes modelos no pueden trasladarse directamente sin validación experimental.

6.1.3. Impacto del ajuste fino con LoRA

El ajuste fino mediante adaptadores LoRA arrojó resultados heterogéneos que dependen tanto de la escala del modelo como de su familia arquitectónica. En la familia Qwen se observa una degradación de precisión que decrece a medida que aumenta la escala: $-20{,}1$ puntos porcentuales en *Qwen3.5-0.8B* (de $0{,}816$ a $0{,}615$), $-3{,}7$ puntos en *Qwen3.5-2B* y solo $-1{,}0$ puntos en *Qwen3.5-4B*. En el caso de *Qwen3.5-0.8B*, el adaptador altera el equilibrio geométrico preexistente en las capas de atención del modelo e impone un sesgo de frecuencias absolutas que destruye la flexibilidad cognitiva necesaria para seguir prompts complejos.

Sin embargo, este patrón de "a menor escala, mayor degradación" no es universal: *Gemma-3-1b-it*, pese a ser el modelo de menor escala del subconjunto ajustado, es el único que **mejora** con LoRA ($+13{,}4$ puntos porcentuales, de $0{,}418$ a $0{,}552$). Esta excepción resulta coherente con el sesgo de chat identificado en su comportamiento base: el entrenamiento supervisado con LoRA corrige específicamente la tendencia de Gemma a generar respuestas discursivas fuera del formato esperado, un problema de alineamiento con la tarea más que de conocimiento clínico subyacente. En consecuencia, el efecto de LoRA sobre la precisión no está determinado únicamente por un umbral de tamaño, sino que interactúa con las características de alineamiento propias de cada familia de modelos.

En las variantes de 2B y 4B de la familia Qwen, más allá de la ganancia moderada en precisión bruta, el ajuste fino LoRA aportó una reducción drástica del formateo erróneo de respuestas y una mayor consistencia en la adherencia a las restricciones del prompt, y en el caso del modelo de 4B, una inmunidad prácticamente total frente a la inyección de ruido sintáctico. El output generado por el modelo *LoRA_Qwen3.5-4B* es directamente consumible por sistemas informáticos hospitalarios descendentes sin postprocesamiento adicional, aunque su precisión bruta (90,6 %) resulta ligeramente inferior a la del modelo base equivalente (91,6 %).

6.1.4. Heterogeneidad entre familias arquitectónicas

La comparativa entre las familias Qwen, Gemma e IBM Granite pone de manifiesto diferencias cualitativas de comportamiento que van más allá de la escala de parámetros. La familia Qwen demostró la mayor robustez nativa en el entorno clínico, fundamentada en la alta densidad de tokens médicos de su corpus de preentrenamiento multilingüe. Los modelos Gemma-3 exhibieron un marcado *sesgo de chat* en su configuración base: priorizaron la elocuencia y la estructura conversacional sobre la precisión extractiva estricta, lo que se manifestó como un lastre en los prompts

que exigen respuestas atómicas. El modelo Granite-1B presentó tiempos de inferencia anómalos superiores a media hora, un comportamiento específico de su arquitectura hipercompacta no replicable en otras familias de tamaño equivalente y que lo excluye de cualquier escenario de uso clínico en tiempo real.

6.1.5. Contribuciones a la auditoría de errores

El análisis forense de los fallos de inferencia identificó patrones recurrentes de confusión semántica que arrojan luz sobre los límites actuales de los SLMs en el procesamiento del lenguaje clínico. Las abreviaturas con mayor resistencia predictiva fueron aquellas que presentaban alguna de las siguientes características: polisemia contextual extrema (como `III`, que puede referirse al tercer par craneal, a un grado de bloqueo auriculoventricular o a un ordinal numérico), fragmentación destructiva en la tokenización (como `UPDRS`, descompuesto en sub-tokens inconexos por el algoritmo BPE), o presencia de caracteres especiales no estándar en los vocabularios generalistas (como `Tª` y `ºC`). Estos hallazgos constituyen una agenda de mejora concreta para futuras iteraciones del sistema.

6.1.6. El papel del prototipo de herramienta de anotación en el proceso experimental

El desarrollo del prototipo de herramienta de anotación web, aunque de naturaleza instrumental, resultó indispensable para la fase experimental del proyecto. La herramienta permitió construir el Corpus Médicos (299 instancias de desambiguación anotadas con supervisión de especialistas clínicos del servicio de cardiología del Hospital Universitario de Jaén) que constituye el conjunto de evaluación de mayor rigor del trabajo. La arquitectura cliente-servidor implementada, con soporte para la reanudación de sesiones, la autoasignación de abreviaturas unívocas y la incorporación de nuevas entradas al diccionario, podría adaptarse en un desarrollo futuro como base de una plataforma de anotación reutilizable para otros proyectos de investigación en PLN clínico en español.

6.1.7. Cumplimiento de objetivos

Los objetivos específicos formulados en la introducción se han alcanzado de la siguiente forma:

- **Análisis del repertorio de siglas y modelos capaces de interpretarlas:** cubierto mediante la revisión del estado del arte y la construcción del diccionario de abreviaturas médicas a partir de la SEDOM.
- **Evaluación comparativa de distintos modelos:** satisfecho con el diseño experimental de 25 configuraciones (7 modelos $\times$ 3 prompts + 4 modelos con ajuste fino LoRA).
- **Construcción de una biblioteca de abreviaciones clínicas:** logrado con el fichero `abreviaciones.csv` que integra el diccionario de referencia y las contribuciones de los facultativos anotadores.
- **Perfeccionamiento del prompting:** abordado con el diseño, evaluación y análisis comparativo de tres estrategias de estructuración de instrucciones.
- **Implementación de un pipeline experimental reproducible:** logrado mediante el script de evaluación parametrizable mediante fichero de configuración, que permitió lanzar y comparar de forma automática y sistemática las 25 configuraciones modelo-prompt evaluadas.
- **Evaluación mixta del sistema:** cubierto con la combinación de métricas automáticas de clasificación y la evaluación sobre corpus con anotación experta humana.
- **Documentación del proceso y reflexión sobre el impacto clínico:** recogido en la presente memoria.

6.2. Líneas de trabajo futuro

Los resultados obtenidos abren un conjunto de líneas de trabajo que se presentan en orden decreciente de dependencia con respecto a las aportaciones inmediatas de este TFG.

6.2.1. Ampliación y enriquecimiento del corpus

La principal limitación del diseño experimental es el tamaño de los conjuntos de evaluación. El Corpus Médicos, compuesto por 299 instancias distribuidas en siete documentos de cardiología, permite extraer conclusiones metodológicas sólidas, pero no es suficiente para estimar con garantías estadísticas el comportamiento del sistema ante el espectro completo de abreviaturas que aparecen en la práctica clínica real.

Una línea de trabajo prioritaria es la expansión del corpus hacia otros servicios hospitalarios (urgencias, oncología, nefrología, neurología), incrementando tanto el número de documentos como la diversidad de abreviaturas cubiertas.

En paralelo, sería deseable estudiar la concordancia inter-anotador entre distintos especialistas clínicos sobre los mismos documentos, lo que permitiría cuantificar la ambigüedad inherente de ciertas abreviaturas incluso para expertos humanos y establecer una cota superior realista para el rendimiento del sistema automático.

6.2.2. Estrategias avanzadas de ajuste fino

Los resultados sobre LoRA sugieren al menos dos extensiones metodológicas relevantes. En primer lugar, la exploración de variantes de la técnica de adaptación paramétrica eficiente, como QLoRA con cuantización en 4 bits, DoRA (*Weight-Decomposed Low-Rank Adaptation*) o IA^3 (*Infused Adapter by Inhibiting and Amplifying Inner Activations*), que podrían ofrecer perfiles de rendimiento distintos, especialmente en el régimen de modelos ultra-compactos donde LoRA estándar produce sobreajuste. En segundo lugar, la experimentación con distintas configuraciones de rango y alfa en los adaptadores, así como con técnicas de mezcla de expertos (*Mixture of Experts*) en el ajuste fino, podría abrir vías para superar el umbral mínimo de masa paramétrica identificado en los experimentos.

6.2.3. Tokenización y vocabulario médico especializado

La auditoría forense de errores identificó la fragmentación destructiva de acrónimos médicos por parte del tokenizador BPE como uno de los factores de fallo más relevantes. Una línea de mejora directa es la extensión del vocabulario del tokenizador con los acrónimos del diccionario de la SEDOM y otros recursos léxicos biomédicos en español, garantizando que siglas como `UPDRS`, `TTPA` o `IECA` se indexen como tokens unitarios con identidad conceptual propia. Esta modificación, conocida como *vocabulary augmentation*, requiere un reentrenamiento del embedding de entrada pero puede realizarse sin reentrenar el cuerpo del modelo, lo que la hace técnicamente accesible.

6.2.4. Integración en sistemas de soporte a la decisión clínica

La validación de la viabilidad técnica del sistema en condiciones de hardware hospitalario estándar allana el camino hacia su integración como módulo en sistemas de soporte a la decisión clínica. Una línea de trabajo natural es el desarrollo de una API

REST que encapsule el pipeline de desambiguación (detección de abreviaturas, consulta al diccionario, inferencia del modelo y generación del informe expandido) y pueda conectarse con sistemas de Historia Clínica Electrónica (HCE) como OpenMRS u Orion Health mediante estándares de interoperabilidad sanitaria (HL7 FHIR, CDA R2). Este paso elevaría la aportación del trabajo del plano de la investigación al de la transferencia tecnológica.

6.2.5. Extensión a nuevos tipos de terminología

El sistema desarrollado se ha centrado en la desambiguación de abreviaturas y siglas, pero el texto clínico presenta otras formas de terminología ambigua que podrían abordarse con el mismo marco metodológico. La extensión del sistema al reconocimiento y normalización de nombres de medicamentos en sus distintas presentaciones (principio activo, nombre comercial, abreviatura farmacológica), a la expansión de símbolos de unidades de medida no estándar y a la detección de acrónimos propios de servicios o centros hospitalarios concretos (que no figuran en ningún diccionario general) representan tres vías de ampliación del alcance del proyecto con alto impacto clínico potencial.

6.2.6. Evaluación con métricas de impacto clínico

La métrica de evaluación principal empleada en este trabajo es la *accuracy* o precisión (proporción de instancias correctamente desambiguadas), estándar en tareas de desambiguación formuladas como clasificación multi-clase. El uso de métricas complementarias como F1-macro, que pondera por igual las clases independientemente de su frecuencia, y BERTScore, que captura la similitud semántica entre la expansión generada y la de referencia, queda propuesto como extensión metodológica.

6.2.7. Arquitecturas multimodales

Los informes hospitalarios no son documentos exclusivamente textuales: incluyen tablas de laboratorio, resultados de pruebas de imagen, electrocardiogramas digitalizados y datos estructurados de monitores. Una línea de trabajo de mayor envergadura, posibilitada por la reciente maduración de los modelos de lenguaje visión-lenguaje (VLMs) de pequeña escala, es la integración de la desambiguación textual con la interpretación de estas fuentes de información multimodal. La correlación entre la abreviatura en el texto y el valor numérico correspondiente en una tabla de constantes vitales,

por ejemplo, podría resolver ambigüedades que el sistema textual actual no puede deshacer por sí solo.

Capítulo 7

Uso responsable de IA

En coherencia con los principios de transparencia e integridad académica que deben regir todo trabajo de investigación, este capítulo detalla de forma pormenorizada cómo se han empleado las herramientas de Inteligencia Artificial (IA) generativa durante la realización del proyecto. El objetivo es dejar constancia explícita del alcance real de dicho uso, de los controles de calidad aplicados a sus resultados y de la asunción plena de responsabilidad, por parte del autor, sobre la totalidad del trabajo presentado.

Antes de entrar en el detalle, resulta imprescindible establecer una precisión fundamental que enmarca todo lo que se expone a continuación: **la redacción de esta memoria es obra íntegra y exclusiva del autor**. Ni un solo párrafo, argumento o razonamiento de este documento ha sido redactado por una herramienta de IA. El contenido intelectual, la estructura, la interpretación de los resultados y la totalidad de las decisiones de diseño experimental son fruto del trabajo personal del autor. El empleo de la IA se ha circunscrito de manera estricta a las tareas instrumentales de apoyo que se enumeran en las secciones siguientes y, en el caso concreto de la memoria, se ha limitado exclusivamente a la corrección ortográfica, sin intervención alguna en la escritura del texto.

7.1. Fases del proyecto apoyadas por IA

A lo largo del desarrollo del proyecto se han utilizado cuatro herramientas de IA generativa con propósitos claramente diferenciados: GitHub Copilot, empleado como asistente de programación integrado en el entorno de desarrollo; Google Gemini y ChatGPT (OpenAI), utilizados como apoyo conversacional para la resolución de dudas técnicas y la exploración de enfoques; y Claude (Anthropic), empleado principalmente en tareas de programación y en la corrección ortográfica de la memoria. En todos los

casos, estas herramientas actuaron como un instrumento de apoyo bajo la supervisión y el criterio del autor, nunca como sustituto del trabajo propio. Las fases concretas en las que intervinieron fueron las siguientes:

- **Desarrollo del prototipo de herramienta de anotación:** se recurrió a GitHub Copilot, Claude y Google Gemini como asistentes de programación durante la construcción del prototipo web descrito en el Capítulo 5, tanto en la generación de fragmentos de código repetitivos como en la sugerencia de estructuras de implementación.
- **Resolución de problemas durante el desarrollo del prototipo:** ante errores y bloqueos técnicos surgidos en la fase de implementación, se emplearon GitHub Copilot, Google Gemini y Claude como apoyo para el diagnóstico y la depuración (*debugging*) del código.
- **Creación de scripts auxiliares de formateo de datos:** GitHub Copilot asistió en la escritura de los scripts encargados de reformatear y preparar los conjuntos de datos, así como de otras utilidades auxiliares de apoyo al flujo de trabajo experimental.
- **Configuración de los scripts de ajuste fino y evaluación de modelos:** Google Gemini y Claude prestaron apoyo en la configuración y la escritura de los scripts empleados para el ajuste fino (*fine-tuning*) mediante LoRA y para el análisis de los modelos, descritos en el Capítulo 4.
- **Exploración de posibles análisis de los resultados:** Google Gemini, ChatGPT y Claude se utilizaron como apoyo para plantear y contrastar posibles enfoques de análisis sobre los resultados obtenidos. La interpretación final de dichos resultados, no obstante, corresponde en su totalidad al autor.
- **Generación de gráficas y formato de tablas:** Claude y Google Gemini asistieron en la creación de las gráficas presentadas en la memoria y en la aplicación de formato a las tablas en formato LaTeX.
- **Corrección ortográfica de la memoria:** Claude se empleó únicamente para la detección y corrección de erratas y errores ortográficos del texto ya redactado por el autor. Este uso **no incluyó en ningún momento la redacción ni la reescritura de contenido**, sino exclusivamente la revisión ortográfica de un texto previamente escrito en su totalidad por el autor.

7.2. Validación y seguridad de los resultados

Las herramientas de IA generativa presentan limitaciones conocidas: pueden cometer errores (*alucinaciones*), introducir sesgos o proponer código con fallos o vulnerabilidades. Consciente de estos riesgos, el autor aplicó de manera sistemática una serie de protocolos de verificación sobre todas las salidas proporcionadas por la IA antes de incorporarlas al proyecto:

- **Revisión y comprensión de todo el código generado:** ningún fragmento de código sugerido por estas herramientas se incorporó de forma automática. El autor revisó, comprendió y adaptó manualmente cada aportación, verificando que su comportamiento se ajustaba a la lógica prevista y descartando aquellas propuestas que no encajaban con el diseño del sistema.
- **Ejecución de pruebas sobre el código:** el código apoyado por IA se sometió a pruebas de funcionamiento en condiciones reales de ejecución, comprobando que producía los resultados esperados sobre los datos del proyecto antes de darlo por válido.
- **Contraste de la información técnica:** las sugerencias de carácter conceptual o técnico proporcionadas por las herramientas conversacionales se contrastaron con la documentación oficial de las librerías empleadas y con fuentes académicas fiables, evitando asumir como ciertas afirmaciones no verificadas.
- **Verificación de gráficas y tablas:** las gráficas y tablas elaboradas con apoyo de IA se cotejaron directamente con los datos experimentales de origen, garantizando que los valores representados se correspondían fielmente con los resultados obtenidos en los experimentos.
- **Revisión manual de la corrección ortográfica:** cada corrección ortográfica propuesta se revisó de forma individual para asegurar que respetaba el sentido original del texto y no introducía alteraciones no deseadas en el contenido redactado por el autor.
- **Protección de datos sensibles:** en ningún momento se introdujeron datos clínicos reales, información confidencial de pacientes ni datos identificativos en las consultas (*prompts*) realizadas a las herramientas de IA. El corpus hospitalario empleado en el proyecto se encontraba previamente anonimizado, y las interacciones con la IA se limitaron a cuestiones de índole técnica, sin exponer material sensible, en cumplimiento de las obligaciones de privacidad descritas en el Capítulo 3.

7.3. Beneficios aportados al proyecto

El uso responsable y supervisado de estas herramientas ha tenido un impacto positivo en el desarrollo global del trabajo, sin que ello reste en modo alguno mérito ni autoría al esfuerzo personal invertido. Los principales beneficios percibidos fueron:

- **Mejora de la eficiencia en tareas mecánicas:** el apoyo en la generación de código repetitivo, el formateo de datos y la maquetación de tablas permitió reducir el tiempo dedicado a tareas rutinarias y de escaso valor intelectual, liberando tiempo para concentrarlo en las decisiones de diseño experimental y en el análisis de los resultados.
- **Superación de bloqueos técnicos:** la asistencia en la depuración de errores facilitó la resolución de problemas puntuales que, de otro modo, habrían supuesto una detención prolongada del desarrollo, agilizando así el avance del prototipo y de los experimentos.
- **Aprendizaje de nuevas tecnologías:** la interacción con estas herramientas expuso al autor a enfoques, librerías y patrones de implementación que enriquecieron su formación técnica, actuando como un complemento al proceso de aprendizaje propio del Trabajo Fin de Grado.
- **Mejora de la calidad formal de la memoria:** la corrección ortográfica contribuyó a depurar erratas y a elevar la pulcritud del texto final, siempre sobre una redacción íntegramente elaborada por el autor.

En síntesis, la IA se ha empleado en este proyecto como una herramienta de apoyo instrumental, sujeta en todo momento al criterio, la supervisión y la responsabilidad última del autor. El valor intelectual del trabajo, la redacción completa de esta memoria y la interpretación de los resultados constituyen una aportación personal y original, sobre la que el autor asume plena responsabilidad.

Bibliografía

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[2] Nithushi R Samaranayake, Dixon ST Cheung, May PS Lam, Tommy T Cheung, William CM Chui, Ian CK Wong, and Bernard MY Cheung. The effectiveness of a 'do not use'list and perceptions of healthcare professionals on error-prone abbreviations. *International journal of clinical pharmacy*, 36(5):1000–1006, 2014.

[3] Katherine Mondaca-Gómez and Naldy Febré Vergara. Uso de abreviaturas inseguras en la prescripción médica y errores de medicación: una revisión narrativa. *Revista médica de Chile*, 148(6):842–848, 2020.

[4] Francisco Soto Arnáez et al. Análisis de la incidencia de riesgo de la utilización de acrónimos en la historia clínica digital en pacientes hospitalizados. 2015.

[5] Jorge M Porras-Garzón and Rosa Estopà. Escalas de legibilidad aplicadas a informes médicos: límites de un análisis cuantitativo formal. *Círculo de Lingüística Aplicada a la Comunicación*, 83:25, 2020.

[6] Miluska.Jara. Objetivos y metas de desarrollo sostenible - Desarrollo Sostenible, 12 2020. URL `https://www.un.org/sustainabledevelopment/es/sustainable-development-goals/`.

[7] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 3645–3650, 2019.

[8] Roy Schwartz, Jesse Dodge, Noah A Smith, and Oren Etzioni. Green ai. *Communications of the ACM*, 63(12):54–63, 2020.

[9] Pengfei Li, Jianyi Yang, Mohammad A Islam, and Shaolei Ren. Making ai less "thirsty": Uncovering and addressing the secret water footprint of ai models. arxiv, 10.48550. *arXiv preprint arXiv.2304.03271*, 2023.

[10] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240, 2020.

[11] Kexin Huang, Jaan Altosaar, and Rajesh Ranganath. Clinicalbert: Modeling clinical notes and predicting hospital readmission. *arXiv preprint arXiv:1904.05342*, 2019.

[12] Sonia Plasencia Martínez and Javier Moliner Lahoz. Uso y abuso de las abreviaturas en los informes del hospital clínico universitario lozano blesa. *Rev Paraninfo Digital*, 16, 2012.

[13] Ander Intxaurrondo, Juan Carlos de la Torre, H Rodríguez Betanco, Montserrat Marimon, Jose Antonio Lopez-Martin, Aitor Gonzalez-Agirre, J Santamarıa, Marta Villegas, and Martin Krallinger. Resources, guidelines and annotations for the recognition, definition resolution and concept normalization of spanish clinical abbreviations: the barr2 corpus. In *Proceedings of SEPLN*, pages 1–9, 2018.

[14] Areej Mustafa Mahmoud Jaber. Word sense disambiguation for clinical abbreviations. 2022.

[15] Marc Weeber, James G Mork, and Alan R Aronson. Developing a test collection for biomedical word sense disambiguation. In *Proceedings of the AMIA Symposium*, page 746, 2001.

[16] Ariel S. Schwartz and Marti A. Hearst. *A SIMPLE ALGORITHM FOR IDENTIFYING ABBREVIATION DEFINITIONS IN BIOMEDICAL TEXT*. doi: $10.1142/9789812776303_0042$. URL `https://doi.org/10.1142/9789812776303_0042`.

[17] Hongfang Liu, Virginia Teller, and Carol Friedman. A multi-aspect comparison study of supervised word sense disambiguation. *Journal of the American Medical Informatics Association*, 11(4):320–331, 2004.

[18] Mark Stevenson, Yikun Guo, Abdulaziz Alamri, and Robert Gaizauskas. Disambiguation of biomedical abbreviations. In *Proceedings of the BioNLP 2009 Workshop*, pages 71–79, 2009.

[19] Sociedad Española de Documentación Médica (SEDOM). Diccionario de siglas médicas. `http://diccionario.sedom.es/`, 2024. Accedido: 2026-06-08.

[20] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 260–270, 2016.

[21] Michihiro Yasunaga, Jure Leskovec, and Percy Liang. Linkbert: Pretraining language models with document links. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8003–8016, 2022.

[22] Monica Agrawal, Stefan Hegselmann, Hunter Lang, Yoon Kim, and David Sontag. Large language models are few-shot clinical information extractors. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 1998–2022, 2022.

[23] Aitor Gonzalez-Agirre, Montserrat Marimon, Ander Intxaurrondo, Obdulia Rabal, Marta Villegas, and Martin Krallinger. Pharmaconer: Pharmacological substances, compounds and proteins named entity recognition track. In *Proceedings of The 5th Workshop on BioNLP Open Shared Tasks*, pages 1–10, 2019.

[24] Antonio Miranda-Escalada, Eulàlia Farré, and Martin Krallinger. Named entity recognition, concept normalization and clinical coding: Overview of the cantemist track for cancer text mining in spanish, corpus, guidelines, methods and results. *IberLEF@ SEPLN*, pages 303–323, 2020.

[25] IBM Granite Team. Granite 3.0 language models, October 2024. URL `https://github.com/ibm-granite/granite-3.0-language-models/`.

[26] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[27] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

[28] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.

[29] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, and Sayak Paul. PEFT: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

[30] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023.

[31] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.

[32] Chiara Zucco et al. DataTools4Heart: A european health data toolbox for cardiovascular diseases. In *Studies in Health Technology and Informatics (Proceedings of MIE 2023)*, volume 305, pages 123–126. IOS Press, 2023.

[33] Yonghui Wu, Jun Xu, Yaoyun Zhang, and Hua Xu. Clinical abbreviation disambiguation using neural word embeddings. In *Proceedings of BioNLP 15*, pages 171–176, 2015.

[34] Zeljko Kraljevic, Thomas Searle, Anthony Shek, Lukasz Roguski, Kawsar Noor, Daniel Bean, Aurelie Mascio, Leilei Zhu, Amos A Folarin, Angus Roberts, et al. Multi-domain clinical natural language processing with medcat: the medical concept annotation toolkit. *Artificial intelligence in medicine*, 117:102083, 2021.

[35] Tianyu Han, Lisa C Adams, Jens-Michalis Papaioannou, Paul Grundmann, Tom Oberhauser, Alexei Figueroa, Alexander Löser, Daniel Truhn, and Keno K Bressem. Medalpaca–an open-source collection of medical conversational ai models and training data. *arXiv preprint arXiv:2304.08247*, 2023.

[36] Equipo CLARA-Med. El proyecto CLARA-Med: Simplificación de textos médicos en español. In *Proceedings of the International Conference on Language Resources and Evaluation (LREC)*, 2023. Proyecto coordinado CSIC-UNED-UCM.

Apéndice A

Prompts diseñados para la desambiguación de siglas

En este anexo se detallan los tres tipos de *prompts* estructurados que se almacenan en el archivo `prompts.json`. Estos han sido diseñados siguiendo técnicas de ingeniería de *prompts* mediante la asignación de roles específicos, tareas acotadas e instrucciones de restricción de formato de salida en formato JSON.

A.1. Prompt 1: Enfoque de análisis lingüístico y clínico

Este *prompt* asigna al modelo el rol de médico especialista con competencias lingüísticas, priorizando el análisis contextual y el área médica global del informe.

```
Rol: Eres un médico especialista con alta capacidad de análisis lingüístico.
Tarea: Selecciona la definición correcta para la abreviación objetivo.

Contexto completo: {texto_completo}
Frase donde aparece: {frase}
Abreviación: {abreviatura}
Opciones posibles:
{opciones}

Instrucciones:
Analiza el área médica del "Contexto completo".
Evalúa cada una de las "Opciones" dentro de la "Frase clave".
Elige la opción que tenga sentido clínico en ese escenario.
```

```
IMPORTANTE: Responde ÚNICAMENTE generando un diccionario JSON. La clave
debe llamarse "eleccion" y el valor debe ser tu respuesta. No uses
plantillas vacías ni puntos suspensivos.
```

A.2. Prompt 2: Enfoque de sustitución semántica

Este *prompt* induce al modelo a realizar una sustitución interna del término y descartar opciones por incoherencia gramatical o clínica directa en la frase afectada.

```
Rol: Eres un experto en terminología clínica y desambiguación de siglas médicas.
Tarea: Sustituye mentalmente la abreviación por cada opción y descarta las
que no tengan sentido en la frase.

Contexto completo: {texto_completo}
Frase donde aparece: {frase}
Abreviación: {abreviatura}
Opciones posibles:
{opciones}

Instrucciones:
Analiza el área médica del "Contexto completo".
Sustituye la abreviación por cada opción en la "Frase donde aparece" y
descarta las que no tengan sentido clínico.
Elige la opción que mejor encaje tras la sustitución.

IMPORTANTE: Responde ÚNICAMENTE generando un diccionario JSON. La clave
debe llamarse "eleccion" y el valor debe ser tu respuesta. No uses
plantillas vacías ni puntos suspensivos.
```

A.3. Prompt 3: Enfoque de auditoría y coherencia global

Este tercer *prompt* adopta una perspectiva global de historia clínica, forzando al modelo a buscar consistencia cruzada con otros elementos y siglas presentes en el documento.

```
Rol: Eres un médico auditor con experiencia en normalización de historias clínicas.
```

```
Tarea: Elige la definición más coherente con el resto del documento clínico.

Contexto completo: {texto_completo}
Frase donde aparece: {frase}
Abreviación: {abreviatura}
Opciones posibles:
{opciones}

Instrucciones:
Analiza el área médica del "Contexto completo" buscando otras siglas o
términos relacionados.
Evalúa cada una de las "Opciones" dentro de la "Frase donde aparece".
Elige la opción más coherente con el conjunto del relato clínico.

IMPORTANTE: Responde ÚNICAMENTE generando un diccionario JSON. La clave
debe llamarse "eleccion" y el valor debe ser tu respuesta. No uses
plantillas vacías ni puntos suspensivos.
```

Apéndice B

Manual de usuario del prototipo de herramienta de anotación

Este anexo tiene como objetivo servir de guía práctica para instalar, poner en marcha y utilizar el prototipo de herramienta de anotación web descrita en el Capítulo 5. Está redactado para que cualquier persona, independientemente de sus conocimientos técnicos, pueda desplegar la herramienta en un equipo local y emplearla para anotar informes clínicos con abreviaturas médicas.

B.1. Requisitos previos

Antes de instalar la herramienta es necesario disponer de:

- **Python 3**, preferiblemente en una versión reciente (3.10 o superior), por compatibilidad con las bibliotecas empleadas.
- **pip**, el gestor de paquetes de Python, habitualmente incluido en la instalación de Python.
- Un **navegador web moderno** (Google Chrome, Mozilla Firefox, Microsoft Edge, etc.), ya que la herramienta no requiere instalar ningún software adicional en el equipo del anotador.
- Espacio en disco suficiente (varios gigabytes), dado que las dependencias relacionadas con el modelo de detección neuronal (`torch`, `transformers`, etc.) son voluminosas.
- **Opcionalmente**, una GPU compatible con CUDA para acelerar la inferencia del modelo de detección de abreviaturas. La herramienta funciona igualmente sin

GPU: en ese caso el modelo se ejecuta sobre CPU o, si no puede cargarse, se activa automáticamente el modo de respaldo basado en el diccionario CSV (véase la Sección B.9).

- Conexión a internet durante la primera ejecución, únicamente si se desea que el servidor descargue el modelo `lmolino/extractor_abreviaciones` desde Hugging Face. Si no se dispone de conexión, la herramienta sigue siendo completamente funcional gracias al modo de respaldo por diccionario.

B.2. Estructura del proyecto

La herramienta se distribuye como una carpeta autocontenida (`plataformaAnotacion`) con la siguiente organización:

```
plataformaAnotacion/
|-- modeloBusqueda/
|   |-- buscador_abreviaciones.py
|   |-- extractor_abreviaciones.py
|   |-- main.py
|   ‘-- prompt_finetunig_terminos.txt
|-- scripts/
|-- static/
|   |-- app.js
|   ‘-- styles.css
|-- abreviaciones.csv
|-- index.html
|-- tagger_api.py
‘-- requirements.txt
```

A continuación se describe brevemente la función de cada elemento:

- `tagger_api.py`: servidor Flask que expone la API REST y sirve la interfaz web. Es el fichero que debe ejecutarse para arrancar la herramienta.
- `index.html`, `static/app.js` y `static/styles.css`: componentes del cliente (frontend) que implementan la interfaz de anotación.
- `abreviaciones.csv`: diccionario de siglas médicas empleado como fuente de opciones de expansión (véase la Sección B.8). Debe residir siempre en la raíz de `plataformaAnotacion/`, junto a `tagger_api.py`.

- `modeloBusqueda/`: módulo con el modelo de extracción neuronal de abreviaturas (`extractor_abreviaciones.py`), la búsqueda por diccionario (`buscador_abreviaciones.py`), los *prompts* de inferencia (`prompt_finetunig_terminos.txt`) y un script de demostración independiente (`main.py`).
- `scripts/`: scripts auxiliares de apoyo al desarrollo; no son necesarios para el funcionamiento básico de la herramienta.
- Las carpetas `__pycache__` son cachés de bytecode generadas automáticamente por Python y pueden ignorarse o eliminarse sin ningún efecto sobre el funcionamiento de la herramienta.

B.3. Instalación

B.3.1. Obtención del proyecto

Copie o descargue la carpeta completa `plataformaAnotacion` (con todo su contenido) en el equipo donde se vaya a ejecutar el servidor. Es recomendable que dicha carpeta se sitúe dentro de un directorio de proyecto más amplio que contenga también la carpeta `Documentacion/`, empleada para el modo revisión (véase la Sección B.3.4).

B.3.2. Creación de un entorno virtual (recomendado)

Para evitar conflictos con otras instalaciones de Python del sistema, se recomienda crear un entorno virtual antes de instalar las dependencias. Desde una terminal, situados en la carpeta `plataformaAnotacion`:

```
# Crear el entorno virtual
python -m venv venv

# Activar el entorno virtual
# En Windows:
venv\Scripts\activate

# En Linux / macOS:
source venv/bin/activate
```

B.3.3. Instalación de las dependencias

Con el entorno virtual activado, instale las dependencias listadas en `requirements.txt`:

```
pip install -r requirements.txt
```

Este fichero incluye tanto las dependencias básicas del servidor (`Flask`, `flask-cors`) como las necesarias para ejecutar el modelo de detección neuronal (`torch`, `transformers`, `accelerate`, `peft`, entre otras). Algunas de estas dependencias están fijadas a una compilación específica de CUDA (por ejemplo, `torch==2.9.1+cu130`). Si el equipo no dispone de una GPU NVIDIA compatible con esa versión de CUDA, la instalación de `torch` puede fallar; en ese caso, sustituya esa línea del fichero por la versión de CPU indicada en la web oficial de PyTorch[1], o elimine el sufijo `+cu130`. Esto no impide el uso de la herramienta, ya que, en ausencia de GPU o de estas dependencias, el sistema activa automáticamente el modo de respaldo basado en el diccionario CSV.

B.3.4. Preparación de las carpetas de datos

El servidor guarda y lee los documentos anotados en formato JSON desde una carpeta situada *fuera* de `plataformaAnotacion`, concretamente en:

```
<carpeta_padre_de_plataformaAnotacion>/Documentacion/EDA/JSON_Nuevos/
```

Esta carpeta se crea automáticamente al arrancar el servidor si no existe, por lo que no requiere ninguna acción para guardar nuevas anotaciones. Sin embargo, si se desea utilizar el **modo revisión** (Sección B.5.2) para cargar documentos ya preprocesados, los ficheros `<nombre_documento>.json` correspondientes deben colocarse manualmente en esa ruta antes de acceder a la herramienta.

B.4. Puesta en marcha del servidor

Con el entorno virtual activado y las dependencias instaladas, sitúese en la carpeta `plataformaAnotacion` y ejecute:

```
python tagger_api.py
```

[1] `https://pytorch.org/get-started/locally/`

Durante el arranque, la consola mostrará información sobre el dispositivo de cómputo detectado (GPU/CUDA, MPS o CPU) y el resultado de la carga del modelo de detección de abreviaturas. Si el modelo no puede cargarse (por ausencia de GPU, de dependencias o de conexión a internet), se mostrará un aviso y la herramienta continuará funcionando en modo de respaldo, basado exclusivamente en el diccionario `abreviaciones.csv`. En ambos casos, el servidor queda operativo en:

```
http://localhost:5000/
```

Basta con abrir esa dirección en el navegador para acceder a la interfaz.

Nota sobre despliegue en red: el fichero `static/app.js` tiene la dirección del servidor codificada en la constante `API_URL` (por defecto, `http://localhost:5000`). Si la herramienta se despliega en un servidor de la red interna del hospital para que varios facultativos accedan desde otros equipos, esta constante debe actualizarse con la dirección IP o el nombre de host real del servidor.

B.5. Modos de acceso a la herramienta

B.5.1. Modo normal: anotación desde cero

Accediendo a la URL raíz (`http://localhost:5000/`) se obtiene el modo normal, pensado para introducir manualmente el texto de un informe clínico y anotarlo desde cero.

B.5.2. Modo revisión: reanudar o corregir un documento

Accediendo a `http://localhost:5000/<nombre_documento>` (sustituyendo `<nombre_documento>` por el identificador del documento, sin extensión) se obtiene el modo revisión. Este modo busca el fichero `<nombre_documento>.json` en la carpeta descrita en la Sección B.3.4 y precarga el texto junto con las anotaciones que ya contuviera, permitiendo completar o corregir una sesión de anotación previa. Si el fichero no existe en esa ruta, el servidor devuelve un mensaje indicando que no se ha encontrado.

B.6. Guía de uso paso a paso

B.6.1. Carga del diccionario

Al abrir la herramienta, el apartado *Estado del Diccionario* indica si `abreviaciones.csv` se ha cargado correctamente y cuántas abreviaturas contiene. Esta carga es automática y no requiere ninguna acción por parte del usuario; hasta que no se complete, el botón de procesar texto permanece deshabilitado.

B.6.2. Introducción y procesamiento del texto (modo normal)

En el modo normal, el anotador debe:

1. Indicar, opcionalmente, un **identificador de documento** en el campo correspondiente (se usará como nombre del fichero JSON de salida; si se deja vacío se asigna un nombre por defecto).
2. Pegar o escribir el texto del informe clínico en el área de texto.
3. Pulsar el botón **Identificar Abreviaciones**.

El texto es enviado al servidor, que lo tokeniza y detecta las abreviaturas conocidas (combinando el modelo neuronal, cuando está disponible, con el diccionario CSV). El resultado se muestra en el panel de resultados, con cada abreviatura resaltada mediante un color de fondo.

B.6.3. Código de colores

Cada abreviatura detectada se resalta con un color que indica su estado:

- **Amarillo (pendiente):** la abreviatura tiene varias opciones de expansión posibles y aún no se ha seleccionado ninguna. Requiere la intervención del anotador.
- **Azul (auto-asignada):** la abreviatura solo tenía una opción de expansión en el diccionario y el sistema se la ha asignado automáticamente. Se recomienda revisarla y confirmarla.
- **Verde (confirmada):** el anotador ha seleccionado manualmente el significado, o ha confirmado una asignación automática.

B.6.4. Selección del significado de una abreviatura

Al hacer clic sobre cualquier abreviatura resaltada se abre un modal con las opciones de expansión disponibles en el diccionario. El anotador puede:

- Pulsar sobre una de las opciones listadas para seleccionarla; el modal se cierra automáticamente y la abreviatura pasa a mostrarse en verde.
- Escribir una definición personalizada en el campo de texto inferior y pulsar **Guardar otra definición** (o la tecla `Intro`) si ninguna de las opciones propuestas es correcta. La nueva definición se añade al diccionario y queda disponible para futuras anotaciones.
- Pulsar **Ignorar Abreviación** si el fragmento resaltado no debe anotarse (por ejemplo, un falso positivo). El token quedará marcado como ignorado y no se incluirá en el fichero JSON final.

B.6.5. Incorporación de nuevas abreviaturas no detectadas

Si el anotador observa en el texto una abreviatura que *no* ha sido resaltada automáticamente, puede seleccionarla con el ratón (arrastrando el cursor sobre el fragmento correspondiente). Al soltar el botón del ratón se abre un modal para introducir su definición; al guardarla, el fragmento se incorpora tanto a la anotación del documento actual como al diccionario `abreviaciones.csv`, quedando registrado como una entrada añadida por el facultativo.

B.6.6. Guardado de la anotación

Una vez revisadas todas las abreviaturas necesarias, el botón **Guardar como JSON** se habilita. Al pulsarlo, el cliente construye el objeto de anotación (véase la Sección B.7) y lo envía al servidor, que lo almacena en la carpeta de salida (Sección B.3.4). Si por algún motivo el servidor no está disponible en ese momento, el navegador descargará automáticamente el fichero JSON de forma local, como medida de contingencia para no perder el trabajo realizado.

B.7. Formato del fichero de anotación (JSON)

Cada documento anotado se guarda con la siguiente estructura:

```
{
  "docid": "cardiologia_001",
  "text": "El paciente presenta FA con respuesta ventricular...",
  "abbreviations": [
    {
      "char_begin": 22,
      "char_end": 24,
      "text": "FA",
      "meaning": "Fibrilacion auricular"
    },
    ...
  ]
}
```

donde `char_begin` y `char_end` son las posiciones de carácter de la abreviatura dentro del campo `text`, y `meaning` es el significado seleccionado por el anotador.

B.8. El diccionario de abreviaturas: `abreviaciones.csv`

El diccionario que alimenta las opciones de expansión mostradas al anotador es el fichero `abreviaciones.csv`, con tres columnas:

- **Abreviacion**: la forma abreviada.
- **Definicion**: su significado.
- **Facultativo**: origen de la entrada. El valor `NO` indica que pertenece al diccionario de referencia original; `SI` indica que fue añadida por un médico desde la propia interfaz; `IA` indica que fue sugerida por el modelo de lenguaje durante la construcción del corpus y añadida posteriormente.

El fichero puede editarse manualmente con cualquier editor de texto o de hojas de cálculo (respetando la codificación UTF-8 y la estructura de columnas) para incorporar entradas de forma masiva. Los cambios realizados manualmente se aplicarán la próxima vez que se reinicie el servidor, ya que el diccionario se carga en memoria al arrancar.

B.9. Resolución de problemas comunes

- **El botón «Identificar Abreviaciones» permanece deshabilitado.** El diccionario no se ha podido cargar. Compruebe que el fichero `abreviaciones.csv` existe en la raíz de `plataformaAnotacion` y que el servidor sigue en ejecución; revise el mensaje de error mostrado en el apartado *Estado del Diccionario*.
- **La consola indica «Modelo NO disponible» al arrancar.** El modelo neuronal no ha podido descargarse o cargarse (falta de GPU, dependencias incompletas o sin conexión a internet). No es necesario realizar ninguna acción: la herramienta sigue siendo completamente funcional en modo de respaldo por diccionario, aunque la detección automática de abreviaturas será algo menos exhaustiva.
- **Al acceder al modo revisión aparece «Archivo no encontrado».** Verifique que el identificador de la URL coincide exactamente con el nombre del fichero JSON (sin la extensión `.json`) y que dicho fichero está ubicado en `Documentacion/EDA/JSON_Nuevos/`, tal y como se describe en la Sección B.3.4.
- **Al guardar aparece un aviso de que no se pudo guardar en el servidor.** El servidor no ha podido persistir el fichero (por ejemplo, por permisos de escritura o por haberse detenido). El navegador descargará automáticamente una copia local del JSON como medida de seguridad; puede colocarse manualmente en la carpeta de salida una vez resuelto el problema.
- **El puerto 5000 ya está en uso por otra aplicación.** Edite la última línea de `tagger_api.py` (`app.run(debug=True, port=5000)`) para indicar un puerto libre, y actualice también la constante `API_URL` en `static/app.js` para que coincida.

B.10. Recomendaciones de despliegue y privacidad

Dado que la herramienta procesa informes clínicos, se recomienda desplegarla exclusivamente en un equipo o servidor de la red interna del centro sanitario, sin exponer el puerto del servidor a internet. Todo el procesamiento se realiza localmente y ningún texto introducido se envía a servicios externos, lo que permite cumplir con los requisitos de privacidad de los datos clínicos (véase la Sección 5.1). Al tratarse de un prototipo académico de uso interno, la herramienta no incorpora sistema de autenticación de usuarios, por lo que se recomienda restringir el acceso a la dirección del servidor a las personas autorizadas para la tarea de anotación.